Diploma Engineering

Laboratory Manual

(Software Testing)
(DI05000111)

[Computer Engineering, 5th semester]

Enrollment No	
Name	
Branch	Computer
Academic Term	2026-2027(Term-261)
Institute	GPG, Surat (GTU Code-615)



Directorate of Technical Education
Gandhinagar - Gujarat

Software Testing (DI05000111)

Laboratory Manual

Prepared By

Shri Alpeshkumar. R. Thaker
Lecturer – Diploma in Computer Engineering
Government Polytechnic, Ahmedabad

Shri Umang D. Shah
Lecturer – Diploma in Computer Engineering
Government Polytechnic, Ahmedabad

Branch Coordinator

Shri B. H. Kantevala
HOD- Diploma in Computer Engineering
Government Polytechnic, Ahmedabad

Committee Chairman

Shri R. D. Raghani
(HOD-EC)
Principal (I/C)
Government Polytechnic, Gandhinagar

CTE's Vision:

To facilitate quality technical and professional education having relevance for both industry and society, with moral and ethical values, giving equal opportunity and access, aiming to prepare globally competent technocrats.

CTE's Mission:

1. Quality technical and professional education with continuous improvement of all the resources and personnel
2. To promote conducive ecosystem for Academic, Industry, Research, Innovations and Startups
3. To provide affordable quality professional education with moral values, equal opportunities, accessibility and accountability
4. To allocate competent and dedicated human resources and infrastructure to the institutions for providing world-class professional education to become a Global Leader ("Vishwa Guru")

Institute's Vision:

Institute's Mission:

Department's Vision:

Department's Mission:

Certificate

This is to certify that Ms. _____

Enrollment No. _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ of _____ Semester

Of **Government Polytechnic for Girls, Surat** Institute (GTU Code: **615**) has completed the term work satisfactorily in Subject **Software Testing - DI050001111** for the academic year: 2026-2027, Term: **261** as Prescribed in the curriculum.

Place: ……

Date: ……..

Subject Faculty **Head of the Department**

Preface

The primary aim of any laboratory/Practical/field work is enhancement of required skills as well as creative ability amongst students to solve real time problems by developing relevant competencies in psychomotor domain. Keeping in view, GTU has designed competency focused outcome-based curriculum-NEP-2024 for Diploma engineering programmes. In this more time is allotted to practical work than theory. It shows importance of enhancement of skills amongst students and it pays attention to utilize every second of time allotted for practical amongst Students, Instructors and Lecturers to achieve relevant outcomes by performing rather than writing practice in study type. It is essential for effective implementation of competency focused outcome- based Green curriculum-NEP-2024. Every practical has been keenly designed to serve as a tool to develop & enhance relevant industry needed competency in each and every student. These psychomotor skills are very difficult to develop through traditional chalk and board content delivery method in the classroom. Accordingly, this lab manual has been designed to focus on the industry defined relevant outcomes, rather than old practice of conducting practical to prove concept and theory.

By using this lab manual, students can read procedure one day in advance to actual performance day of practical experiment which generates interest and also, they can have idea of judgement of magnitude prior to performance. This in turn enhances predetermined outcomes amongst students. Each and every Experiment /Practical in this manual begins by competency, industry relevant skills, course outcomes as well as practical outcomes which serve as a key role for doing the practical. The students will also have a clear idea of safety and necessary precautions to be taken while performing experiment.

This manual also provides guidelines to lecturers to facilitate student-centered lab activities for each practical/experiment by arranging and managing necessary resources in order that the students follow the procedures with required safety and necessary precautions to achieve outcomes. It also gives an idea that how students will be assessed by providing Rubrics.

Software testing is an indispensable process in the software development life cycle, serving as a critical quality assurance measure. Its primary objective is to identify and rectify defects, ensuring that the software meets specified requirements and functions as intended. Through a systematic and rigorous examination of the software, testing helps detect errors, inconsistencies, and potential vulnerabilities, thereby enhancing the reliability and robustness of the final product. By validating each component and feature, software testing contributes to the overall improvement of software quality, fosters user satisfaction, and reduces the likelihood of post-deployment issues. Additionally, it provides stakeholders with confidence in the software's performance, functionality, and security, ultimately supporting the delivery of a high-quality and dependable product.

By using this course, the student will develop underpinning knowledge, adequate software testing skills of competency for testing various applications using different testing techniques.

Although we try our level best to design this lab manual, but always there are chances of improvement. We welcome any suggestions for improvement.

Programme Outcomes (POs):

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the *engineering* problems.

2. **Problem analysis**: Identify and analyse well-defined *engineering* problems using codified standard methods.

3. **Design/ development of solutions:** Design solutions for *engineering* well-defined technical problems and assist with the design of systems components or processes to meet specified needs.

4. **Engineering Tools, Experimentation and Testing:** Apply modern *engineering* tools and appropriate technique to conduct standard tests and measurements.

5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.

6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.

7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes *in field of engineering.*

Practical Outcome - Course Outcome matrix

Course Outcomes (COs):

a) CO1: Describe importance of Software Testing to enhance the quality of software products.
b) CO2: Apply black-box and white-box testing techniques effectively.
c) CO3: Develop effective test cases based on specifications, utilizing appropriate templates and methodologies.
d) CO4: Apply bug identification skills to create a comprehensive defect report for the software application.
e) CO5: Utilize popular testing tools (e.g., Selenium, JUnit, TestNG) for creating and executing automated test scripts.

S. No.	Practical Outcome/Title of experiment	CO1	CO2	CO3	CO4	CO5
1.	Create a glossary of at least five software testing terminologies with explanations.	√	-	-	-	-
2.	Describe why both SDLC and STLC are Essential in the software development process.	√	-	-	-	-
3.	Enlist and present at least three popular Testing methodology (e.g., Agile, Waterfall) with its advantages and disadvantages.	√	-	-	-	-
4.	Write program and design test cases for the following Control and decision-making statement. 1) For... Loop 2) Switch...case 3) Do... While 4) If...else	-	√	-	-	-
5.	Design test cases for different tasks (OTP Verification, Image upload, Age verification in Registration) in any software modules using Equivalence partitioning, boundary value analysis, and decision table testing techniques of Black Box Testing.	-	√	-	-	-
6.	Identify system specification & design test cases for Sales Invoice Management.	-	√	-	-	-

7.	Design Test Cases for Flight Ticket Booking System.					
8.	Develop test scenarios and test cases for the Login functionality of a social media application.	-	-	√	-	-
9.	Develop an RTM and measure testing metrics for any two dynamic web pages of an e-commerce website.	-	-	√	-	-
10.	Execute test cases for a travel booking app And prepare a test summary report.	-	-	√	-	-
11.	Prepare defect report after executing test cases for registration page.	-	-	-	√	-
12.	Prepare defect report after executing test Cases for Withdrawn of amount from ATM Machine.	-	-	-	√	-
13.	A. Install and set up the Selenium WebDriver and necessary drivers (e.g., ChromeDriver, GeckoDriver) on your system. B. Install JUnit, TestNG using an IDE (Integrated Development Environment) like Eclipse or IntelliJ.	-	-	-	-	√
14.	Design and run test script for a registration Page using Selenium tool and JUnit.	-	-	-	-	√
15.	Design and run test script for a Login page and Home page using Selenium tool and TestNG.	-	-	-	-	√

Industry Relevant Skills

The following industry relevant skills are expected to be developed in the students by performance of experiments of this course.

Proficient in identifying and resolving software defects through comprehensive testing methodologies and tools to ensure the delivery of high-quality and reliable software products.

Guidelines to Course Faculty

1. Couse faculty should demonstrate experiment with all necessary implementation strategies described in curriculum.
2. Couse faculty should explain industrial relevance before starting of each experiment.
3. Course faculty should involve& give opportunity to all students for hands on experience.
4. Course faculty should ensure mentioned skills are developed in the students by asking.
5. Utilize 2 hrs. of lab hours effectively and ensure completion of write up with quiz also.
6. Encourage peer to peer learning by doing same experiment through fast learners.

Instructions for Students

1. Organize the work in the group and make record of all observations.
2. Students shall develop maintenance skill as expected by industries.
3. Student shall attempt to develop related hand-on skills and build confidence.
4. Student shall develop the habits of evolving more ideas, innovations, skills etc.
5. Student shall refer technical magazines and data books.
6. Student should develop habit to submit the practical on date and time.
7. Student should well prepare while submitting write-up of exercise.

Continuous Assessment Sheet

Enrolment No: **Name:**

Date: **Term:**

Sr. No	Practical Outcome/Title of experiment	Page	Date	Marks (25)	Sign
1	Create a glossary of at least five software testing terminologies with explanations.				
2	Describe why both SDLC and STLC are Essential in the software development process.				
3	Enlist and present at least three popular Testing methodology (e.g., Agile, Waterfall) with its advantages and disadvantages.				
4	Write program and design test cases for the following Control and decision-making statement. 1) For... Loop 2) Switch...case 3) Do... While 4) If...else				
5	Design test cases for different tasks (OTP Verification, Image upload, Age verification in Registration) in any software modules using Equivalence partitioning, boundary value analysis, and decision table testing techniques Of Black Box Testing.				
6	Identify system specification & design test cases for Sales Invoice Management.				
7	Design Test Cases for Flight Ticket Booking System.				
8	Develop test scenarios and test cases for the Login functionality of a social media application.				
9	Develop an RTM and measure testing metrics for any two dynamic web pages of an e-commerce website.				
10	Execute test cases for a travel booking app And prepare a test summary report.				
11	Prepare defect report after executing test cases for registration page.				

12	Prepare defect report after executing test Cases for Withdrawn of amount from ATM Machine.				
13	A. Install and set up the Selenium WebDriver and necessary drivers (e.g., ChromeDriver, GeckoDriver) on your system. B. Install JUnit, TestNG using an IDE (Integrated Development Environment) like Eclipse or IntelliJ.				
14	Design and run test script for a registration Page using Selenium tool and JUnit.				
15	Design and run test script for a Login page and Home page using Selenium tool and TestNG.				

Practical No.1:

Create a glossary of at least five software testing terminologies with explanations.

A. Objective:

To know about fundamental of software testing. It contains number of frequently used terms, terminologies and various life cycles that will help students to familiar with words and phrases commonly used in testing and requirements work.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

2. **Problem analysis**: Identify and analyze well-defined engineering problems using codified standard methods.

3. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

4. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

This practical is expected to develop the following skills.

1. Understand importance of software testing and terminologies in application development.
2. Familiarity with SDLC and STLC methodologies ensures a systematic and organized approach to software development, contributing to efficient and high-quality outcomes

D. Expected Course Outcomes (Cos)

Describe importance of Software Testing to enhance the quality of software products.

E. Practical Outcome (PRo)

Describe various software testing terminologies, SDLC & STLC lifecycle which will be helpful in software testing.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

Software testing is a systematic process of evaluating a software application to identify any defects, errors, or discrepancies between expected and actual outcomes. It involves the execution of a program or system with the intent of finding errors and ensuring that the software behaves as intended. Testing is a crucial part of the software development life cycle (SDLC) and is performed at various stages to guarantee the delivery of a high-quality and reliable software product.

1.1 Importance of Software Testing:

1. Identification of Defects:
    - Testing helps in identifying defects or bugs in the software early in the development process. This early detection allows for timely correction, reducing the cost and effort required for fixing issues in later stages.
2. Ensuring Quality:
    - Quality assurance is a fundamental objective of software testing. Through a systematic testing process, developers and testers can ensure that the software meets specified requirements and functions correctly, leading to a high-quality product.
3. Customer Satisfaction:
    - Thorough testing contributes to customer satisfaction by delivering a software product that meets or exceeds user expectations. A reliable and bug-free application enhances user experience and confidence in the software.
4. Risk Management:
    - Testing helps in identifying and mitigating risks associated with software development. By uncovering potential issues early, development teams can proactively address and manage risks, reducing the likelihood of critical failures in the live environment.
5. Compliance with Requirements:
    - Software testing ensures that the developed software aligns with the specified requirements. By validating each component against its intended functionality, testing helps in preventing deviations from the project's objectives.
6. Cost-Effectiveness:
    - Early detection and resolution of defects are more cost-effective than fixing issues after the software is deployed. Testing contributes to overall cost savings

By minimizing the expenses associated with post-release bug fixes and maintenance.

7. Improved Decision-Making:
    - Test results provide valuable information to project stakeholders, allowing them to make informed decisions regarding the software's readiness for release. Testing metrics and reports offer insights into the software's performance and quality.
8. Enhanced Security:
    - Security testing is a critical aspect of software testing, ensuring that the application is robust against potential security threats. Identifying and addressing security vulnerabilities is essential for protecting sensitive data and maintaining user trust.
9. Regulatory Compliance:
    - In certain industries, there are regulatory standards and compliance requirements that software must meet. Testing helps in ensuring that the software complies with these regulations, avoiding legal issues and penalties.
10. Continuous Improvement:
    - Testing contributes to continuous improvement by providing feedback to the development process. Lessons learned from testing activities can be used to refine development practices, leading to better software quality in future projects.

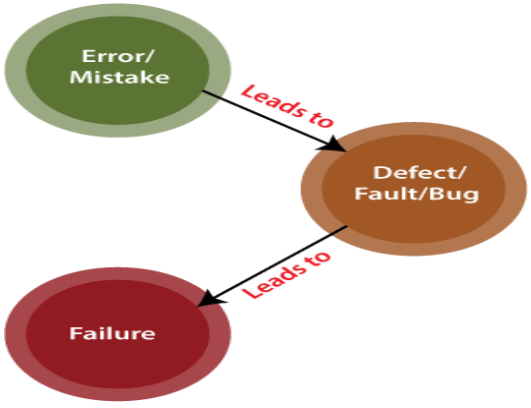
1.2 Software Testing – Bug vs Defect vs Error vs Fault vs Failure

In software testing and development, terms like "Bug," "Defect," "Error," "Fault," and "Failure" are often used to describe different aspects of issues or problems. While these terms are sometimes used interchangeably, they have specific meanings in the context of software engineering.

1. **Bug:**
    - **Definition:** A bug is a term often used interchangeably with "defect" or "error." It refers to a flaw, mistake, or unintended behavior in a software application. Bugs can occur in the code, design, or system architecture and may lead to incorrect or unexpected results.
2. **Defect:**
    - **Definition:** A defect is a deviation or flaw in the software product that does not conform to its specifications or requirements. It is a broader term that encompasses any problem, whether it is in the code, design, documentation, or other aspects of the software.

3. **Error:**
    - **Definition:** An error is a human action that produces an incorrect or unintended result. Errors are mistakes made by developers during the software development process. These mistakes may lead to defects or bugs in the software.
4. **Fault:**
    - **Definition:** A fault is a defect or abnormality in a software component or system. It is the manifestation of an error in the software. When a fault is encountered during the execution of a program, it may lead to a failure.
5. **Failure:**
    - **Definition:** A failure occurs when a software application or system does not perform its intended function or produces incorrect results. Failures are observable and measurable deviations from expected behavior during actual system operation.

Relationships between these Terms:



- An error made by a developer (human action) can result in a fault in the code or system.
- A fault, when executed or triggered, may lead to a defect in the system.
- A defect is a broad term encompassing any problem in the software, whether it originated from an error or a fault.
- Bugs are manifestations of defects in the software that may cause failures during system operation.

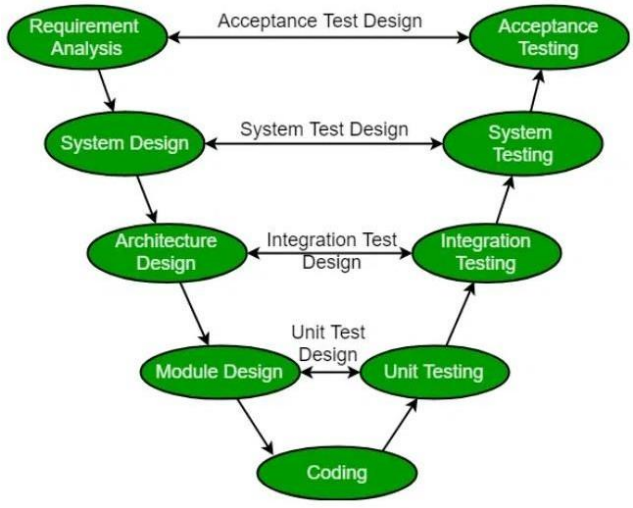
Example Scenario:

1. **Developer Error:** A developer makes a coding mistake (error) while implementing a new feature.
2. **Fault:** The coding mistake results in a fault in the code.
3. **Defect:** The fault manifests as a defect in the software.
4. **Bug:** Users report unexpected behavior or incorrect results, indicating the presence of a bug.
5. **Failure:** The observed incorrect behavior during actual system use is a failure.

Understanding these terms and their relationships is crucial in software testing and debugging processes, as it helps in effectively communicating issues within the development and testing teams.

1.3 Verification and Validation (V Model)

The V Model, also known as the Verification and Validation Model, is a software development and testing framework that emphasizes the relationship between development phases and corresponding testing activities. It is called the V Model because of its characteristic V-shaped representation. The left side of the "V" represents the development phases, and the right side represents the corresponding testing phases. The model highlights the interdependence of verification and validation processes.



Verification:

- *Definition:* Verification is the process of ensuring that the product is designed and implemented correctly based on the specified requirements.
- *In the V Model:* The left side of the V represents the verification process. Each stage of the development phase has a corresponding stage in the testing phase.
- *Activities:*
    - **Requirements Analysis:** Verification of requirements to ensure clarity, completeness, and consistency.
    - **System Design:** Verification of the system design against the requirements.
    - **Architecture Design:** Verification of the software architecture design.
    - **Module Design:** Verification of the detailed module-level design.
    - **Coding:** Verification of the source code against the design.

Validation:

- *Definition:* Validation is the process of evaluating the product during or at the end of the development process to determine whether it satisfies the specified requirements.
- *In the V Model:* The right side of the V represents the validation process. Each testing phase is associated with a specific development phase, and validation activities are performed.
- *Activities:*
    - **Unit Testing:** Validation of individual units or components of the software.
    - **Integration Testing:** Validation of interactions between integrated components or systems.
    - **System Testing:** Validation of the entire software system.
    - **User Acceptance Testing (UAT):** Validation by end-users to ensure the software meets their requirements.

Characteristics of the V Model:

1. **Parallel Development and Testing:** Unlike the traditional waterfall model, where testing is a separate phase after development, the V Model involves parallel development and testing activities.
2. **Early Detection of Defects:** Defects are detected and addressed at an early stage, reducing the cost and effort of fixing issues later in the development process.
3. **Traceability:** The model emphasizes traceability between the development and testing phases, ensuring that each component of the software is linked to its corresponding testing activity.
4. **Comprehensive Testing:** The model promotes comprehensive testing by including various levels of testing (unit, integration, system, and acceptance) throughout the development process.

Advantages of the V Model:

1. **Early Defect Detection:** Defects are identified early in the development process, leading to more efficient bug fixing.
2. **Improved Traceability:** The model provides a clear traceability between the requirements, design, and testing phases.
3. **Parallel Development and Testing:** Development and testing activities can occur concurrently, reducing the overall project timeline.

The V Model is particularly useful for projects with well-defined requirements and where changes are less likely to occur. It offers a systematic approach to software development and testing, ensuring that the final product meets specified requirements.

1.4 Quality Assurance and Quality Control

Quality Assurance

Quality assurance (QA) in software testing is a systematic process that ensures that the software development and testing processes are well-defined, efficient, and result in the delivery of high-quality software. QA activities aim to prevent defects, verify that processes are followed, and provide confidence in the reliability of the software being developed. Here's a detailed description of quality assurance in software testing:

1. **Definition of Quality Assurance:**

   - Quality assurance is a set of planned and systematic actions to ensure that the software development and testing processes result in a product that meets specified requirements and is free of defects.

2. **Objectives of Quality Assurance:**

   - Ensure that the software development and testing processes are well-defined, repeatable, and effective.
   - Identify and eliminate defects early in the development life cycle to reduce the cost of fixing issues.
   - Establish and enforce standards and processes to improve consistency and reliability.
   - Provide confidence to stakeholders that the software meets quality expectations.

3. **Key Components of Quality Assurance in Software Testing:**
   - Process Definition and Compliance: QA involves defining and documenting processes for software development, testing, and quality control. It also includes ensuring that team members follow these processes consistently.
   - Standards and Best Practices: Establishing standards and best practices for coding, testing, documentation, and other development activities. These standards serve as guidelines for the development team to produce high-quality software.
   - Training and Skill Development: QA includes training team members on the defined processes, tools, and methodologies. Continuous skill development ensures that the team stays updated with industry best practices.
   - Reviews and Audits: QA involves conducting reviews and audits of project documentation, code, and test cases to identify discrepancies, non-compliance with standards, and potential issues.
   - Metrics and Measurements: Establishing and monitoring key performance indicators (KPIs) and metrics to measure and improve the effectiveness and efficiency of the software development and testing processes.
   - Process Improvement: QA is focused on continuous improvement. Regularly analyzing processes, identifying areas for improvement, and implementing changes to enhance overall efficiency and quality.

4. **Integration with Testing:**
   - QA is closely integrated with the testing process. It involves defining testing strategies, test plans, and test cases based on the project's requirements and objectives.
   - Ensuring that testing activities are aligned with the overall project goals and that the testing process is capable of identifying and verifying defects.
5. **Benefits of Quality Assurance:**
   - Improved software quality and reliability.
   - Early detection and prevention of defects.
   - Reduced rework and development costs.
   - Enhanced stakeholder confidence in the software.

In summary, quality assurance in software testing is a holistic and proactive approach to ensuring that software development processes are well-defined, efficient, and capable of producing high-quality software. It involves a combination of process definition, adherence to standards, training, reviews, and continuous improvement to achieve the desired level of quality in the final product.

Quality Control

Quality control in software testing refers to the process of ensuring that the software being developed or maintained meets the specified quality standards and requirements. It involves systematic activities and methodologies aimed at identifying and correcting defects or deviations from the desired quality. Here's a detailed description of quality control in software testing:

1. **Objective of Quality Control:**
   - The primary goal of quality control in software testing is to ensure that the software product meets the defined quality criteria, adheres to specifications, and satisfies user expectations.
2. **Key Components of Quality Control:**
   - Test Planning: Quality control begins with the creation of a comprehensive test plan. This plan outlines the testing approach, resources, schedule, and objectives to ensure systematic and organized testing activities.
   - Test Design: The development of test cases and test scenarios based on requirements and specifications. Test design aims to verify that the software functions as intended under various conditions.

- Test Execution: Running the test cases to execute the planned testing activities. This involves comparing the actual behavior of the software with the expected results to identify any discrepancies.
- Defect Logging and Tracking: When discrepancies or defects are identified, they are logged in a defect tracking system. Quality control involves not only finding defects but also managing them effectively, including tracking their status, severity, and resolution.
- Regression Testing: Ensuring that changes or fixes do not negatively impact existing functionalities. This involves re-executing previously executed test cases to verify that the modifications have not introduced new defects.
- Test Reporting: Regular reporting on the progress of testing activities, including test execution status, defect metrics, and other relevant information. This helps stakeholders make informed decisions about the software's readiness for release.
- Test Closure: A formal process to assess whether the testing objectives have been met and to obtain approval to release the software. It involves generating a test summary report and archiving test ware for future reference.
- Testing Techniques in Quality Control:
- Black Box Testing: Focuses on the external behavior of the software without knowledge of its internal implementation. Examples include functional testing, usability testing, and acceptance testing.
- White Box Testing: Examines the internal logic and structure of the software. Examples include unit testing, code coverage analysis, and structural testing.
- Gray Box Testing: Combines elements of both black box and white box testing, where the tester has partial knowledge of the internal workings of the software.

3. **Quality Control Tools:**

- Test Management Tools: Tools that help in planning, designing, executing, and tracking tests. Examples include TestRail, HP ALM, and Jira.
- Defect Tracking Tools: Tools used to log, track, and manage defects throughout the testing process. Examples include Bugzilla, Jira, and Mantis.
- Automation Testing Tools: Tools that enable the automation of repetitive and time-consuming testing tasks. Examples include Selenium, Appium, and JUnit.

4. **Continuous Improvement:**

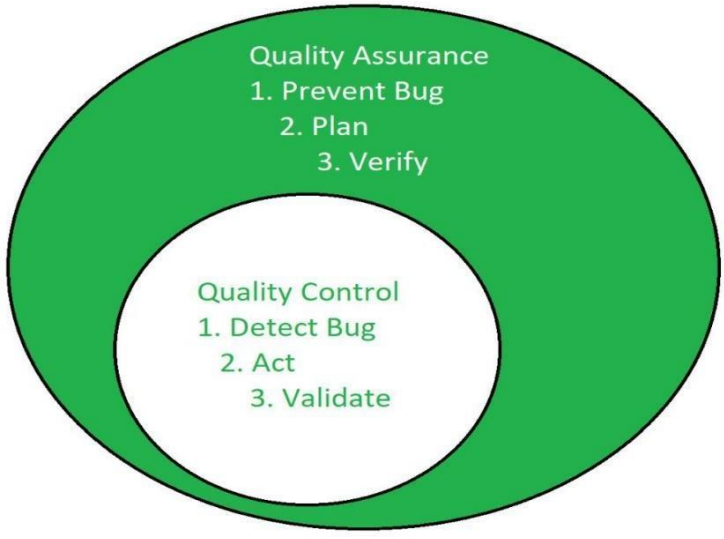
- Quality control is an iterative process, and continuous improvement is a fundamental aspect. Feedback from testing activities, retrospective analyses, and post-release reviews contribute to improving future testing processes and overall software quality.

5. **Standards and Compliance:**
   - Adherence to industry standards and compliance requirements is an integral part of quality control. For example, following ISTQB (International Software Testing Qualifications Board) standards and guidelines can help ensure best practices in software testing.

6. **Collaboration and Communication:**
   - Effective communication and collaboration between different stakeholders, including developers, testers, and business analysts, are crucial for successful quality control. Clear documentation and reporting facilitate a shared understanding of quality objectives.

In summary, quality control in software testing is a comprehensive process that involves planning, designing, executing, and managing testing activities to ensure that software meets specified quality standards. It plays a vital role in delivering reliable and high-quality software products to end-users.

Relation between QA and QC:



QA and QC are complementary elements of a comprehensive quality management strategy in software development. QA focuses on preventing defects by improving processes, while QC involves the actual testing and identification of defects in the product. Both are crucial for delivering high-quality software to end-users. The integration of QA and QC practices contributes to a robust quality management framework.

H. Practical Outcome:

Write down description of at least five software testing terminologies.

Software Testing Terminologies :

1. Software Testing

Software testing is the process of evaluating a software application to identify defects and verify that it meets specified requirements. It ensures that the software functions correctly, reliably, and efficiently before being released to users.

2. Bug

A bug is a flaw or mistake in a software application that causes incorrect or unexpected behavior. Bugs may occur due to coding mistakes, design issues, or incorrect implementation.

3. Defect

A defect is any deviation in the software from its specified requirements or expected behavior. It may exist in the code, design, documentation, or other parts of the software system.

4. Error

An error is a human mistake made during software development, such as writing incorrect code or misunderstanding a requirement. Errors often lead to defects in the software.

5. Fault

A fault is an incorrect piece of code or design introduced because of an error. When executed, a fault may cause the software to fail.

6. Failure

A failure occurs when the software does not perform its intended function or produces incorrect results during execution. It is the visible effect of one or more faults.

7. Validation

Validation is the process of checking whether the developed software satisfies user needs and requirements. It answers the question, "Are we building the right product?"

8. Verification

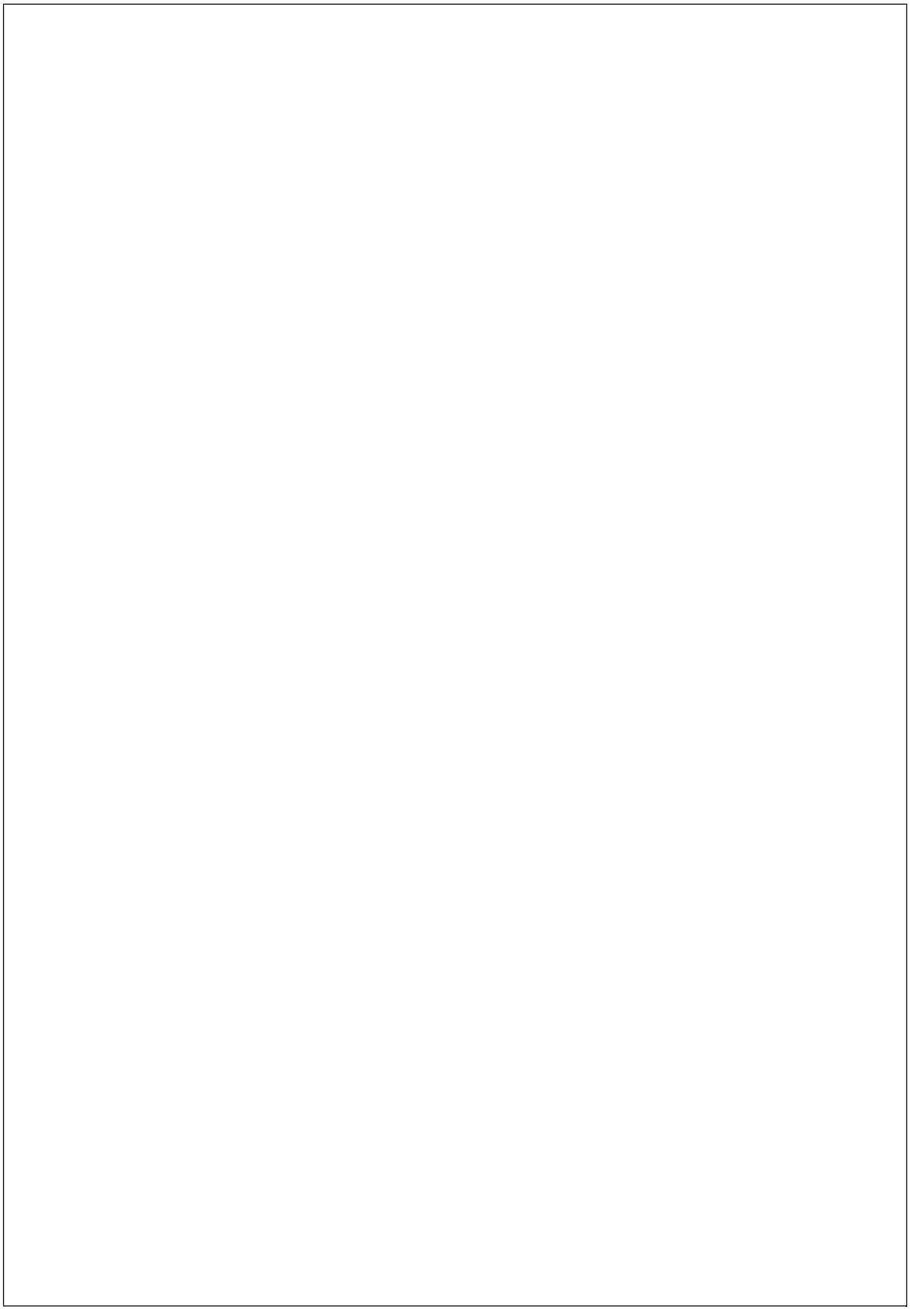
Verification is the process of checking whether the software is being developed according to the specified requirements and design documents. It answers the question, "Are we building the product right?"

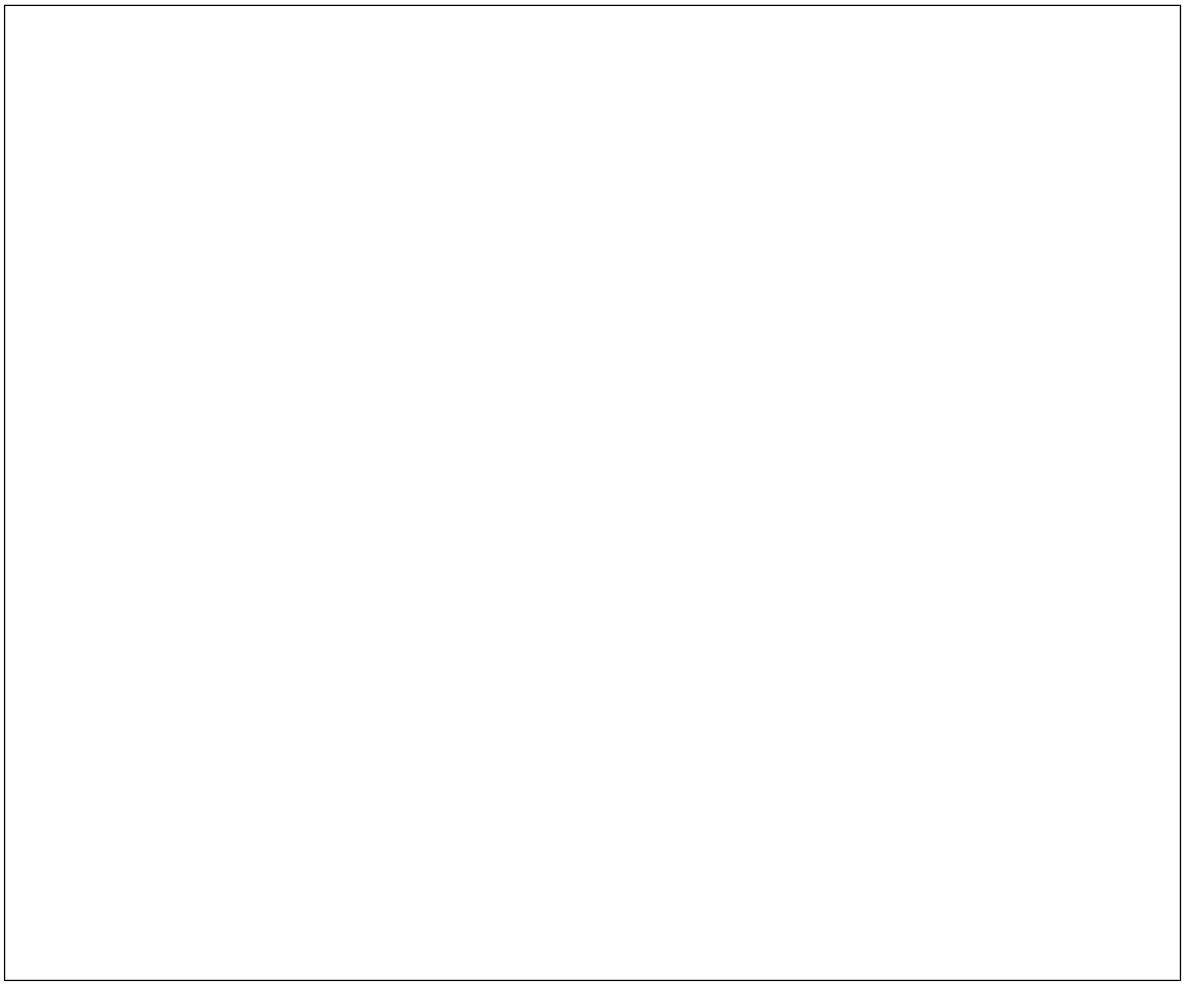
9. Quality Assurance (QA)

Quality Assurance (QA) is a process-oriented activity that focuses on preventing defects by improving software development and testing processes. Its objective is to ensure that the software development process produces high-quality software.

10. Quality Control (QC)

Quality Control (QC) is a product-oriented activity that focuses on identifying defects in the software through testing. It ensures that the final software product meets the required quality standards before release.





I. Practical related Questions.

1. What is the primary goal of software testing?

a) To develop software
b) To find defects and ensure the software meets requirements
c) To write code
d) To design user interfaces

2. Which phase of the Software Testing Life Cycle (STLC) involves creating detailed test cases based on requirements and design specifications?

a) Test Planning
b) Test Execution
c) Test Case Design
d) Test Closure

3. In Software Development Life Cycle (SDLC), what does the "Feasibility Study" phase involve?

a) Writing code
b) Assessing the technical and economic feasibility of the project

c) Creating detailed test cases
d) Deploying the software

4. What is the primary goal of software testing?

a) Verification
b) Validation
c) Both a and b
d) Neither a nor b

5. What is the main objective of Validation in software testing?

a) Checking the syntax of the source code
b) Verifying the software against user expectations
c) Identifying defects in the software
d) Executing test cases to ensure code quality

J. References Links

1. https://www.geeksforgeeks.org/software-testing-basics/
2. https://www.tutorialspoint.com/difference-between-bug-defect-error-fault- and-failure
3. https://www.geeksforgeeks.org/software-testing-bug-vs-defect-vs-error-vs-fault-vs-failure/
4. https://www.guru99.com/sdlc-vs-stlc.html
5. https://www.softwaretestinghelp.com/quality-assurance-vs-quality-control/

K. Assessment-Rubrics

Sr .No.	Criteria	Obtain Marks
1	Correctness of Practical Implementation	**/15**
2	Timely Submission of Report and Answer to Questions	**/10**
	Total	**/25**

Signature with Date

Practical No.2:

Describe why both SDLC and STLC are Essential in the software development process.

A. Objective:

To know about fundamental of software testing. It contains number of frequently used terms, terminologies and various life cycles that will help students to familiar with words and phrases commonly used in testing and requirements work.

B. Expected Program Outcomes (POs)

a. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

b. **Problem analysis**: Identify and analyse well-defined engineering problems using codified standard methods.

c. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

d. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

This practical is expected to develop the following skills.

a. Understand importance of software testing and terminologies in application development.
b. Familiarity with SDLC and STLC methodologies ensures a systematic and organized approach to software development, contributing to efficient and high-quality outcomes

D. Expected Course Outcomes (Cos)

Describe importance of Software Testing to enhance the quality of software products.

E. Practical Outcome (PRo)

Describe SDLC & STLC lifecycle which will be helpful in software testing.

F. Expected Affective domain Outcome (ADos)

a. Follow Testing standards and practices.
b. Maintain tools and equipment.
c. Follow safety practices.
d. Follow ethical practices

G. Prerequisite Theory:

2.1 Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a systematic process used by software developers to design, develop, test, and deploy software applications. It is a structured Approach that helps ensure the quality and effectiveness of the software. The SDLC consists of several phases, each with its own set of activities and deliverables.



Here is an overview of the typical SDLC life cycle:

1. **Planning:**
   - **Objective:** Define the project scope, goals, and requirements. Identify constraints such as budget and timeline.
   - **Activities:**
     - Conduct feasibility studies
     - Define project scope
     - Create a project plan
     - Define roles and responsibilities
2. **Feasibility Study:**
   - **Objective:** Assess the technical and economic feasibility of the project.
   - **Activities:**
     - Evaluate technical feasibility
     - Assess economic feasibility

- Analyze legal and ethical considerations

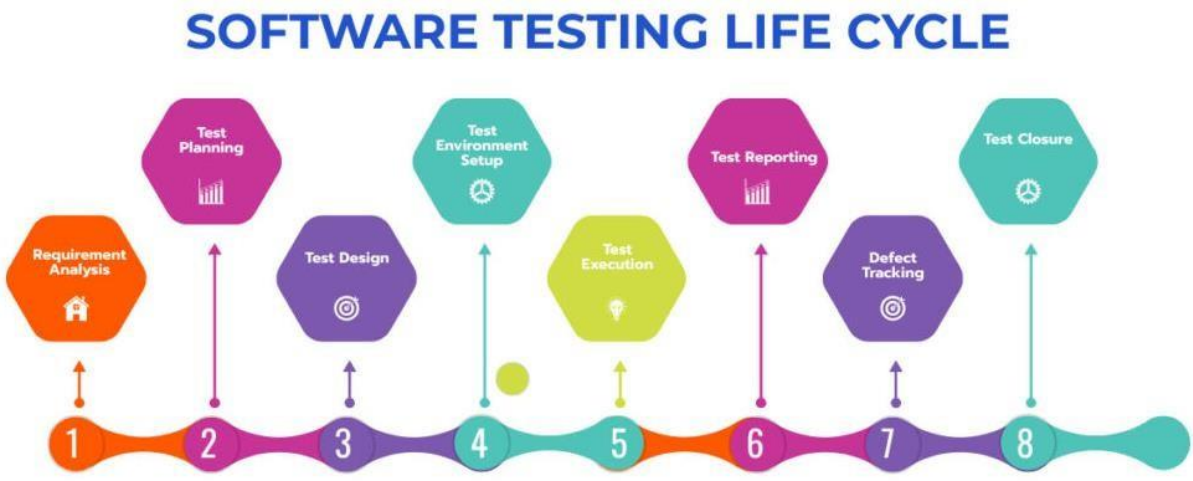
3. **System Design:**
   - **Objective:** Define the architecture, components, modules, data, and interfaces of the system.
   - **Activities:**
     - Create a high-level design
     - Define system architecture
     - Develop detailed specifications
     - Design user interface
4. **Implementation (Coding):**
   - **Objective:** Transform the design into actual code.
   - **Activities:**
     - Write code according to design specifications
     - Conduct unit testing
     - Debugging and fixing issues
5. **Testing:**
   - **Objective:** Verify that the software works as intended and meets the specified requirements.
   - **Activities:**
     - Conduct various types of testing (unit testing, integration testing, system testing, acceptance testing)
     - Identify and fix defects
     - Verify software meets requirements
6. **Deployment:**
   - **Objective:** Release the software to end-users.
   - **Activities:**
     - Develop deployment plans
     - Install the software on target environments
     - Monitor for issues during initial use
7. **Maintenance and Support:**
   - **Objective:** Address issues, update features, and provide ongoing support.
   - **Activities:**
     - Fix defects and issues
     - Implement updates and enhancements
     - Provide user support

It's important to note that the SDLC is not a strictly linear process, and iterations or feedback loops may occur between phases. Agile methodologies, for example, embrace iterative development and may involve cycles of planning, implementation, and testing in shorter timeframes.

Additionally, various SDLC models, such as Waterfall, Agile, and Spiral, offer different approaches to managing the software development process. The choice of SDLC model depends on factors like project size, complexity, and requirements.

2.2 Software Testing Life Cycle (STLC)

The Software Testing Life Cycle (STLC) is a set of processes and activities performed by testing teams to ensure the quality and reliability of a software product. The STLC outlines the steps and phases that testers go through from the initial planning To the final release of the software. While the specific activities and phases may vary based on the testing approach and methodologies adopted, here is a generalized breakdown of the Software Testing Life Cycle:



1. **Requirement Analysis:**
   - **Objective:** Understand and analyze the requirements from a testing perspective.
   - **Activities:**
     - Review and analyze the software requirements.
     - Identify testable requirements and potential risks.
     - Create a Test Plan outlining the testing strategy, scope, resources, schedule, and deliverables.
2. **Test Planning:**
   - **Objective:** Plan and define the testing approach based on the project requirements and scope.
   - **Activities:**
     - Define test objectives and entry/exit criteria.
     - Identify test deliverables.
     - Allocate resources, roles, and responsibilities.
     - Develop a test schedule.
3. **Test Case Design:**
   - **Objective:** Create detailed test cases based on the requirements and design specifications.
   - **Activities:**
     - Develop test scenarios and test cases.
     - Define test data and expected results.

- Ensure test cases cover various scenarios, including positive and negative test cases.

4. **Test Environment Setup:**
   - **Objective:** Establish the test environment to execute the test cases.
   - **Activities:**
     - Set up the necessary hardware and software.
     - Configure the test environment to replicate the production environment.
     - Verify that the test environment is stable and ready for testing.
5. **Test Execution:**
   - **Objective:** Execute the test cases on the prepared test environment.
   - **Activities:**
     - Execute test cases as per the test plan.
     - Record test results, including any defects or issues.
     - Monitor and log system behavior during testing.
6. **Defect Reporting:**
   - **Objective:** Report and manage defects identified during testing.
   - **Activities:**
     - Log defects with detailed information.
     - Prioritize and categorize defects based on severity and impact.
     - Communicate defects to the development team for resolution.
7. **Defect Tracking:**
   - **Objective:** The testing team logs and tracks the defects or issues identified during the testing process and works with the development team to resolve them.
8. **Test Closure:**
   - **Objective:** Summarize testing activities and make decisions about further testing.
   - **Activities:**
     - Review and assess test results against exit criteria.
     - Prepare test summary reports.
     - Obtain stakeholders' approval for completing the testing phase.

STLC is adaptable and may vary based on the project, organization, or testing methodologies used. The goal of STLC is to systematically and efficiently conduct testing activities to ensure the quality and reliability of the software being developed. It is often closely integrated with the development process, ensuring that testing is performed at each appropriate stage of the SDLC.

Summary of SDLC (Software Development Life Cycle):

SDLC, or Software Development Life Cycle, is a systematic process that outlines the steps or phases involved in developing software. It provides a structured framework for

Planning, designing, coding, testing, deploying, and maintaining software applications. The key phases of SDLC include planning, feasibility study, system design, implementation (coding), testing, deployment, and maintenance. SDLC aims to deliver high-quality software that meets user requirements, is developed within budget and timeline constraints, and can be maintained effectively over time. It helps in managing resources efficiently, controlling costs, and providing a structured approach to software development.

Summary of STLC (Software Testing Life Cycle):

STLC, or Software Testing Life Cycle, is a series of systematic and predefined processes involved in testing a software application. It is an integral part of SDLC and includes stages such as requirement analysis, test planning, test case design, test environment setup, test execution, defect tracking and reporting, test closure, retesting, regression testing, and optional phases like performance testing and automation testing. STLC aims to ensure the quality and reliability of the software by systematically identifying and fixing defects, verifying functionality, and validating the software against specified requirements. It plays a crucial role in improving the overall quality of the software and contributes to customer satisfaction by delivering a reliable and defect-free product.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

Practical Outcome:

Describe why both SDLC and STLC are essential in the software development process.

Software Development Life Cycle (SDLC) and Software Testing Life Cycle (STLC) are two important processes in software development. SDLC focuses on planning, designing, developing, testing, deploying, and maintaining software, while STLC focuses on verifying and validating the software through systematic testing. Both are essential for developing reliable, high-quality software.

Importance of SDLC :

- Provides a structured approach for software development.
- Helps in proper planning and resource management.
- Ensures the software is developed according to user requirements.
- Reduces development cost and time by following a defined process.
- Improves software quality and maintainability.

Important of STLC :

- Ensures that the software is thoroughly tested before release.
- Helps identify and fix defects at an early stage.
- Verifies that the software meets functional and non-functional requirements.
- Improves the reliability, security, and performance of the software.
- Increases customer satisfaction by delivering a defect-free product.

Why Both SDLC and STLC are Essential ?

SDLC provides the roadmap for developing software, whereas STLC ensures that every phase of development is properly tested. SDLC helps build the software correctly, while STLC verifies that the software works correctly and satisfies user requirements. Together, they reduce project risks, improve software quality, minimize maintenance costs, and ensure timely delivery of a reliable software product.

Conclusion :

Both SDLC and STLC play an important role in software development. SDLC manages the complete development process, and STLC ensures the quality of the final product through systematic testing. Together, they help deliver reliable, secure, and high-quality software that meets user expectations.

I. References / Suggestions

1. https://www.javatpoint.com/software-engineering-agile-model
2. https://www.geeksforgeeks.org/agile-software-testing/
3. https://testsigma.com/blog/agile-model-in-software-testing/
4. https://www.geeksforgeeks.org/waterfall-software-testing/
5. https://testsigma.com/blog/spiral-model/

Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Correctness of Practical Implementation	**/15**
2	Timely Submission of Report and Answer to Questions	**/10**
	Total	**/25**

Signature with Date

Practical No.3: Enlist and present at least three popular testing methodologies (e.g., Agile, Waterfall etc.) with its advantages and disadvantages.

A. Objective:

To know about how to various phases of model related to testing action that ensures the delivery of high-quality software that meets specified requirements and satisfies user expectations using various software testing methodologies.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

2. **Problem analysis**: Identify and analyse well-defined engineering problems using codified standard methods.

3. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

4. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

1. Understand testing criteria of each phase of various testing methodologies.
2. Utilize various testing methodologies as per need of software development.

D. Expected Course Outcomes (Cos)

Describe importance of Software Testing to enhance the quality of software products.

E. Practical Outcome (PRo)

Describe various software testing methodologies with pros and cons.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

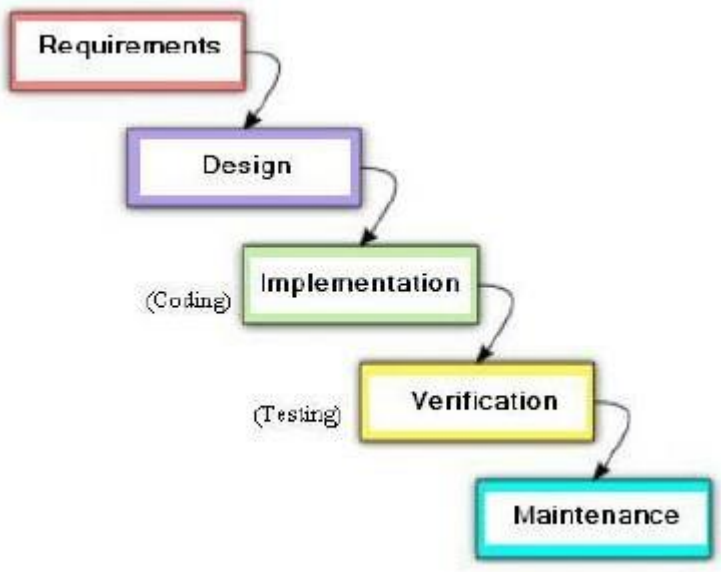
Software Testing methodologies are the approaches and strategies used for testing a precise product to make sure it is fit for purpose. Software testing methodologies are essential as they play a critical role in ensuring the quality, reliability, and effectiveness of software applications.

Some well-accepted testing methodologies comprise – Waterfall model, Agile methodology, iterative model, verification and validation Methodology (V-Model), Spiral Model, Extreme Programming Model, RAD (Rapid Action Development) Methodology, and many others.

Let's explore the different types of testing methodologies below.

Waterfall Model

The Waterfall Model is a traditional and linear approach to software development and testing. In the context of software testing methodologies within the Waterfall Model, testing is typically performed in a sequential and phase-dependent manner. Here's how software testing is integrated into each phase of the Waterfall Model:



1. **Requirements Phase:**
   - **Testing Focus:** In this phase, the focus is on reviewing and validating the requirements documentation to ensure they are complete, clear, and unambiguous.
   - **Testing Activities:** Testers may engage in requirements reviews, walkthroughs, and inspections to identify potential issues early on**.**

2. **Design Phase:**
    - **Testing Focus:** The design phase involves creating detailed design specifications based on the requirements. Testing activities in this phase aim to ensure that the design meets the specified requirements and is feasible.
    - **Testing Activities:** Design reviews, architectural reviews, and inspections may be conducted to validate the design.
3. **Implementation (Coding) Phase:**
    - **Testing Focus:** The coding phase involves actual software development based on the design specifications. Testing activities aim to identify and correct coding errors and ensure compliance with coding standards.
    - **Testing Activities:** Unit testing is the primary focus during this phase, where individual units or modules are tested in isolation to verify their correctness.
4. **Testing Phase:**
    - **Testing Focus:** This is a dedicated testing phase where the entire software system is tested as a whole. The objective is to identify and fix defects and ensure that the software meets the specified requirements.
    - **Testing Activities:** Integration testing, system testing, and acceptance testing are performed sequentially. Each level of testing verifies different aspects of the software's functionality and integration.
5. **Deployment Phase:**
    - **Testing Focus:** The deployment phase involves delivering the tested and validated software to the end-users or customers.
    - **Testing Activities:** User acceptance testing (UAT) may be conducted in collaboration with end-users to ensure that the software meets their expectations and is ready for deployment.
6. **Maintenance Phase:**
    - **Testing Focus:** The maintenance phase involves fixing defects, addressing issues, and making enhancements to the software based on user feedback and changing requirements.
    - **Testing Activities:** Regression testing is a crucial activity during maintenance to ensure that new changes do not introduce new defects or negatively impact existing functionality.

Advantages of the Waterfall Model for Testing:

- Well-structured and easy to understand.
- Clear documentation at each phase.
- Straightforward project management and progress tracking.

Disadvantages of the Waterfall Model for Testing:

- Limited flexibility for changes after the project has started.
- Late detection of defects as testing is performed after development.
- Long development and testing cycles.

It's important to note that while the Waterfall Model has been widely used in the past, many modern development methodologies, such as Agile, emphasize iterative and more flexible approaches to software development and testing. These methodologies aim to address some of the limitations of the Waterfall Model by incorporating continuous feedback and adaptation throughout the development process.

Agile Model

In the Agile model, testing is an integral and continuous part of the software development process. Agile methodologies, such as Scrum, Kanban, and Extreme Programming (XP), emphasize collaboration, adaptability, and customer feedback. Below is an overview of how testing is integrated into the Agile model:

- **User Stories and Product Backlog:**
  - Agile development starts with a product backlog containing user stories, each representing a specific functionality or feature.
  - **Testing Focus**: Testers collaborate with the product owner and development team to understand user stories and define acceptance criteria.
- **Sprint Planning:**
  - Agile projects are divided into fixed-length iterations called sprints, usually 2-4 weeks.
  - **Testing Focus**: Testers participate in sprint planning meetings to discuss testing requirements for the upcoming user stories. They help estimate the testing effort and plan test activities for the sprint.
- **Daily Stand-ups:**
  - Daily stand-up meetings facilitate communication within the team, providing updates on progress and discussing any impediments.
  - **Testing Focus**: Testers share their progress, discuss testing challenges, and collaborate with developers to address any issues promptly.
- **Continuous Integration**:
  - Agile teams often use continuous integration to merge code changes frequently, ensuring that the software is continually built and tested.
  - **Testing Focus**: Automated tests, including unit tests, integration tests, and other automated checks, are integrated into the continuous integration process to provide rapid feedback on code changes.
- **Incremental Development**:
  - Agile projects emphasize delivering a potentially shippable product increment at the end of each sprint.
  - **Testing Focus**: Testers continuously verify and validate features throughout the sprint, ensuring that each increment meets the acceptance criteria and is free of critical defects.

- **Continuous Testing**:
  - Agile teams practice continuous testing to identify defects early in the development process.
  - **Testing Focus**: Automated testing, exploratory testing, and other testing activities are performed continuously throughout the development cycle, promoting a sustainable pace for the team.
- **Sprint Review:**
  - At the end of each sprint, the team conducts a sprint review to demonstrate the completed work to stakeholders.
  - **Testing Focus**: Testers showcase the tested features, and stakeholders provide feedback. Any necessary adjustments are discussed for future sprints.
- **Sprint Retrospective**:
  - A sprint retrospective is conducted to reflect on the sprint, identify areas for improvement, and make adjustments for the next iteration.
  - **Testing Focus**: Testers provide insights into testing processes and suggest improvements, contributing to the team's continuous improvement.
- **Release and Deployment**:
  - Agile projects can release increments to production at the end of each sprint or combine multiple increments for larger releases.
  - **Testing Focus**: Final acceptance testing and validation are performed before releasing the product increment to production.

Advantages of Agile Testing:

- Early and continuous feedback from stakeholders.
- Rapid identification and resolution of defects.
- Flexibility to adapt to changing requirements.
- Collaboration between development and testing teams.

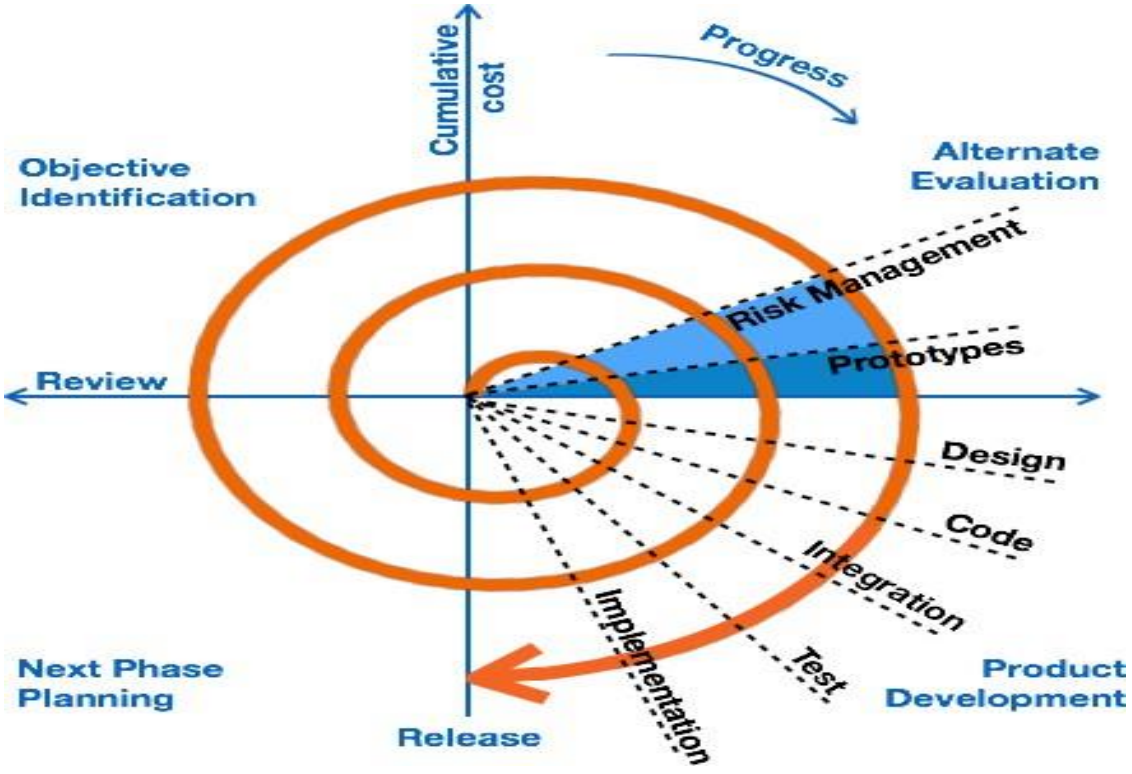
Challenges of Agile Testing:

- Requires strong collaboration and communication.
- Continuous time and resource constraints.
- Relies heavily on automated testing for efficiency.

Agile testing methodologies aim to deliver high-quality software iteratively, with a focus on customer satisfaction and adaptability to changing requirements. The close collaboration between development and testing teams ensures that feedback is integrated quickly, leading to a more responsive and effective development process.

Spiral Model

The Spiral Model is primarily a software development methodology, not specifically a testing methodology. However, it is often used in conjunction with various testing methodologies to ensure that testing activities are integrated into the overall software development process. The Spiral Model is an iterative and incremental model that combines elements of both waterfall and prototype models.



Here's a general overview of how the Spiral Model can be adapted for software testing:

1. **Planning:**
   - Identify the objectives, risks, and alternatives.
   - Plan the testing activities, including defining test objectives, scope, and criteria.
2. **Risk Analysis:**
   - Evaluate potential risks and uncertainties associated with the project.
   - Prioritize and determine the impact of each risk on the project.
3. **Engineering:**
   - Develop a prototype or initial version of the software.
   - Conduct unit testing on individual components/modules.
4. **Evaluation:**
   - Review the prototype with stakeholders, including testers, to gather feedback.
   - Evaluate the prototype in terms of functionality, performance, and other relevant criteria.
5. **Test Planning:**
   - Based on feedback and evaluation, plan the testing for the next iteration.
   - Update the test strategy and test plan accordingly.

6. **Testing:**
    - Conduct various types of testing (unit, integration, system, etc.) during each iteration.
    - Identify and document defects, if any, and prioritize them for resolution.
7. **Risk Resolution:**
    - Address and mitigate the identified risks.
    - Update the risk assessment based on the progress of the project.
8. **Re-evaluation:**
    - Review and evaluate the modified software after addressing the risks and making changes.
    - Gather feedback from stakeholders, including users and testers.
9. **Iterate:**
    - Repeat the spiral for subsequent iterations, incorporating improvements and changes based on feedback and testing results.

Key Characteristics of Spiral Model in Testing:

1. **Early Testing:**
    - Testing activities are initiated early in the development process.
    - Incremental releases of the software are tested and validated in each iteration.
2. **Risk-Driven:**
    - The model emphasizes risk analysis and management.
    - Testing activities focus on areas with high levels of risk, ensuring they are thoroughly tested.
3. **Flexibility:**
    - The model is adaptable to changes in requirements.
    - Testing efforts are adjusted based on the evolving understanding of the system.
4. **Customer Involvement:**
    - Regular customer feedback is sought, and the software is evaluated by the customer at the end of each iteration.
5. **Iterations:**
    - The development and testing process is carried out in iterative cycles.
    - Each iteration results in a new version of the software.
6. **Documentation:**
    - Documentation is maintained and updated throughout the process, reflecting the current state of the software.

While the Spiral Model offers flexibility and accommodates changes during the development process, it requires a thorough understanding of the application domain and continuous risk assessment. Additionally, the model may involve more time and resources

Compared to some other software development methodologies. The Spiral Model allows for flexibility and responsiveness to changes throughout the development process. It helps in accommodating changes in requirements, addressing risks, and ensuring that testing activities are integrated at various stages. It's crucial to adapt the model according to the specific needs of the project and the testing methodology being used, such as agile testing or traditional testing approaches.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

I. Implementation:

Enlist and present at least three popular testing methodologies (e.g., Agile, Waterfall etc.) with its advantages and disadvantages.

Popular Software Testing Methodologies :

1. Waterfall Methodology
2. Agile Methodology
3. Spiral Methodology

(Detailed description on next page)

1. Waterfall Methodology :

Introduction

The Waterfall Model is one of the oldest and simplest software development methodologies. It follows a linear and sequential approach, where each phase must be completed before moving to the next phase. Testing is performed only after the development phase is completed.

Phases

- Requirement Analysis
- System Design
- Implementation (Coding)
- Testing
- Deployment
- Maintenance

Advantages

- Simple and easy to understand.
- Well-defined phases and documentation.
- Easy to manage because each phase has specific deliverables.
- Suitable for small projects with fixed requirements.
- Progress can be easily tracked.

Disadvantages

- Difficult to accommodate changing requirements.
- Testing begins only after coding is completed.
- Defects are discovered late in the development cycle.
- Not suitable for large or complex projects.
- High risk if requirements are not clearly defined.

2. Agile Methodology :

Introduction

Agile is an iterative software development methodology where software is developed in small parts called iterations or sprints. Development and testing are performed simultaneously, allowing continuous feedback and improvement.

Phases

- Requirement Gathering
- Sprint Planning
- Design
- Development
- Testing
- Review
- Release

Advantages

- Continuous testing improves software quality.
- Early detection and correction of defects.
- Easily adapts to changing customer requirements.
- Faster delivery of working software.
- Better communication between developers, testers, and customers.
- Higher customer satisfaction.

Disadvantages

- Requires active customer involvement.
- Less documentation compared to Waterfall.
- Difficult to estimate project cost and timeline.
- Requires experienced team members.
- Not suitable for projects with fixed requirements.

3. Spiral Methodology

Introduction

The Spiral Model combines the features of the Waterfall Model and Prototyping. It focuses on risk analysis and develops the software in multiple iterations called spirals. Each spiral includes planning, risk analysis, development, testing, and customer evaluation.

Phases

- Planning
- Risk Analysis
- Engineering (Design & Development)
- Testing and Customer Evaluation

Advantages

- Excellent for large and complex projects.
- Risks are identified and managed early.
- Customer feedback is obtained after every iteration.
- Changes can be easily incorporated.
- Produces high-quality software.

Disadvantages

- Expensive due to repeated risk analysis.
- Complex to manage.
- Requires skilled professionals.
- Time-consuming.
- Not suitable for small projects.

Conclusion

Different software testing methodologies are used depending on the project requirements. Waterfall is suitable for projects with fixed requirements, Agile is best for projects requiring continuous development and frequent changes, and Spiral is ideal for large, complex, and high-risk projects. Choosing the appropriate methodology helps improve software quality, reduce risks, and ensure successful project completion.

J. Practical related Exercises.

1. **Which testing approach is characterized by short development cycles, frequent releases, and continuous integration?**
   - A) Waterfall testing
   - B) Agile testing
   - C) V-Model testing
   - D) Incremental testing
2. **Which testing methodology is characterized by a series of repeating cycles, each involving planning, risk analysis, engineering, testing, and evaluation?**
   - A) Waterfall
   - B) Spiral
   - C) Agile
   - D) V-Model
3. **In which testing methodology is the development process divided into small, fixed-length iterations or sprints?**
   - A) Waterfall
   - B) Incremental
   - C) RAD (Rapid Application Development)
   - D) Scrum (Agile Framework)
4. **Which testing methodology involves the development of a working model of the system at an early stage and refining it iteratively?**
   - A) Waterfall
   - B) Prototype
   - C) Incremental

D) RAD (Rapid Application Development)

5. **Which testing methodology is known for its emphasis on risk analysis and mitigation throughout the development process?**
   A) Agile
   B) Spiral
   C) Waterfall
   D) Incremental

K. References / Suggestions

1. https://www.javatpoint.com/software-engineering-agile-model
2. https://www.geeksforgeeks.org/agile-software-testing/
3. https://testsigma.com/blog/agile-model-in-software-testing/
4. https://www.geeksforgeeks.org/waterfall-software-testing/
5. https://testsigma.com/blog/spiral-model/

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Correctness of Practical Implementation	**/15**
2	Timely Submission of Report and Answer to Questions	**/10**
	Total	**/25**

Signature with Date

Practical No.4: Write program and design test cases for the following Control and decision-making statement.

1) For... Loop 2) Switch...case 3) Do... While 4) If...else.

A. Objective:

To assess the software's behavior under various conditions, helping identify and address potential issues in the application's functionality.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

2. **Problem analysis**: Identify and analyze well-defined engineering problems using codified standard methods.

3. **Design/ development of solutions:** Design solutions for engineering well-defined technical problems and assist with the design of systems components or processes to meet specified needs.

4. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.

6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.

7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

- To create well-structured, comprehensive, and effective test cases for control and decision-making statements in software development.
- Design various test cases based on specification of application.

D. Expected Course Outcomes (Cos)

Apply black-box and white-box testing techniques effectively.

E. Practical Outcome (PRo)

Design test cases for the looping structure and decision-making statement.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

Test Case:

A test case is a detailed set of conditions, inputs, and expected outcomes that are designed to verify whether a particular feature or aspect of a software application functions as intended. Test cases help ensure that software meets the specified requirements and behave correctly under various scenarios. Each test case represents a unique testing scenario and provides a way to systematically validate the functionality of the software.

Let's take an example to illustrate the concept of a test case:

Example: Login Functionality

1) Test Case: Valid Login

Objective: Verify that a user can successfully log in with valid credentials.

Test Case Steps:

1. Open the application login page.
2. Enter a valid username (e.g., "john_doe") in the username field.
3. Enter a valid password (e.g., "P@ssw0rd") in the password field.
4. Click the "Login" button.
5. Observe the system response.

Expected Outcome:

- The system should authenticate the user with the provided credentials.
- The user should be successfully logged into the application.

Additional Considerations:

- Verify that the user is directed to the correct landing page after successful login.

- Check that the application displays the user's name or relevant information on the dashboard.
- Confirm that the user receives appropriate feedback, such as a welcome message or successful login notification.

2) Test Case: Invalid Login

Objective: Verify that the system handles invalid login attempts appropriately.

Test Case Steps:

1. Open the application login page.
2. Enter an invalid username (e.g., "invalid_user") in the username field.
3. Enter an invalid password (e.g., "wrong_password") in the password field.
4. Click the "Login" button.
5. Observe the system response.

Expected Outcome:

- The system should reject the login attempt and display an error message.
- The error message should indicate that the login credentials are incorrect.

Additional Considerations:

- Check that the system does not allow access to the application for invalid credentials.
- Verify that the error message is clear and user-friendly.
- Ensure that the system does not reveal specific details about whether the username or password is incorrect to prevent security risks.

These examples illustrate two different test cases for the login functionality of a software application. Each test case defines a specific scenario, the steps to be executed, and the expected outcomes.

Control and decision-making statement in php

In PHP, control and decision-making statements are used to control the flow of execution based on certain conditions. The main control structures include if, else, elseif, switch, while, do-while, for, and foreach. Here are some examples:

For Loop: The for loop is used when you know the number of iterations.

```
<?php
for ($i = 1; $i <= 5; $i++) {
echo "The value of i is: $i <br>";
}
```

?>

If Statement: The if statement is used to execute a block of code only if a specified condition is true.

```
<?php
$x = 10;
if ($x > 0) {
echo "The value of x is positive.";
}
?>
```

If-Else Statement: The else statement is used in conjunction with if to execute a block of code if the condition is false.

```
<?php
$x = 5;
if ($x > 0) {
echo "The value of x is positive.";
} else {
echo "The value of x is non-positive.";
}
?>
```

Switch Statement: The switch statement is used to perform different actions based on different conditions.

```
<?php
$day = "Monday";
switch ($day) {
case "Monday":
echo "It's the start of the week.";
break;
case "Friday":
```

```
echo "It's almost the weekend.";
break;
default:
echo "It's a regular day.";
}
?>
```

do...while loop

This is another type of loop that is similar to the while loop. The key difference is that the do...while loop always executes the code block at least once, and then it continues to execute as long as the specified condition is true. Here's the basic syntax of the do...while loop:

```
<?php
$counter = 1;
do {
echo "This is iteration $counter <br>";
   $counter++;
} while ($counter <= 5);
?>
```

Example :

Below is a simple PHP program that uses a **for** loop to calculate the sum of numbers from 1 to a given input. Here, I've included some test cases to help you understand how to design test cases for a **for** loop.

```
<?php
function calculateSum($n) {
   $sum = 0;
for ($i = 1; $i <= $n; $i++) {
      $sum += $i;
   }
return $sum;
```

```
}

$number = 5;
$result = calculateSum($number);
echo "Sum of numbers from 1 to $number is: $result\n";
?>
```

Test Cases:

1. **Valid Positive Input:**
    - Input: **5**
    - Expected Output: **15** (1 + 2 + 3 + 4 + 5)
2. **Valid Zero Input:**
    - Input: **0**
    - Expected Output: **0** (Sum of no numbers)
3. **Valid Large Input:**
    - Input: **10**
    - Expected Output: **55** (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10)
4. **Negative Input (Assuming non-negative integers):**
    - Input: **-3**
    - Expected Output: **Error** or **Undefined** (Depending on how you handle non-positive input)
5. **Decimal Input (Assuming the program only handles integers):**
    - Input: **3.5**
    - Expected Output: **Error** or **Undefined**
6. **String Input (Assuming the program only accepts numeric input):**
    - Input: **"Hello"**
    - Expected Output: **Error** or **Undefined**
7. **Boundary Test - Maximum Integer Value:**

- Input: **2147483647** (Max value for a 32-bit signed integer)
- Expected Output: **Error** or **Undefined** (Depending on system limitations)

8. **Boundary Test - Large Input:**
    - Input: **100000**
    - Expected Output: *(The actual sum value needs to be calculated)*

Feel free to modify the test cases based on the specific requirements of your program and the constraints of your system.

Note: **Students have to write test cases for remaining control and decision making statement.**

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

I. Implementation:

1. While Loop: Calculate the sum of first N natural numbers

</> Source Code:

```php
<?php
function sumNaturalNumbers($n) {
    $sum = 0;
    $i = 1;

    while($i <= $n) {
        $sum += $i;
        $i++;
    }

    return $sum;
}

echo sumNaturalNumbers(5);
?>
```

Test Cases

1. Valid Input
   Input: 5
   Expected Output: 15

2. Small Input
   Input: 1
   Expected Output: 1

3. Zero Input
   Input: 0
   Expected Output: 0

---

2. Switch Case: Display the day of the week

</> Source Code:

```php
<?php
function getDay($day) {
    switch($day) {
        case 1: return "Monday";
        case 2: return "Tuesday";
        case 3: return "Wednesday";
        case 4: return "Thursday";
        case 5: return "Friday";
        case 6: return "Saturday";
        case 7: return "Sunday";
        default: return "Invalid Day";
    }
}
echo getDay(3);
?>
```

Test Cases

1. Valid Input
   Input: 3
   Expected Output: "Wednesday"

2. Boundary Input
   Input: 7
   Expected Output: "Sunday"

3. Invalid Input
   Input: 10
   Expected Output: "Invalid Day"

---

3. If...Else Statement: Check whether a number is Even or Odd

</> Source Code:

```
<?php
function checkEvenOdd($num) {
   if($num % 2 == 0)
      return "Even";
   else
      return "Odd";
}

echo checkEvenOdd(8);
?>
```

Test Cases

1. Even Number

Input: 8
Expected Output: "Even"

2. Odd Number

Input: 11
Expected Output: "Odd"

3. Zero

Input: 0
Expected Output: "Even"

J. Practical related Exercises.

1. **What is the primary purpose of test cases in software development?**
   a. To write code faster
   b. To detect and fix bugs
   c. To impress clients
   d. To increase file size
2. **What is a test case in the context of software testing?**
   a. A bug in the code.
   b. A condition that is always true.
   c. A set of conditions and inputs to verify the correct behavior of a program.
   d. A statement that cannot be tested.
3. **Why is it important to design test cases for different branches of decision-making statements?**
   a. To confuse developers.
   b. To increase code complexity.
   c. To ensure that all possible code paths are tested.
   d. To make the code longer.
4. **Which of the following statements is true about boundary testing in decision-making statements?**
   a. It is not necessary for robust testing.
   b. It focuses on values outside the normal range.
   c. It only tests the most common cases.
   d. It is performed after the software is deployed.
5. **What is the benefit of designing test cases for loops?**
   a. To make the code shorter.
   b. To avoid using loops altogether.
   b. To ensure that the loop terminates properly and produces correct results.
   d. Test cases are not relevant for loops.

K. References / Suggestions

1. https://www.geeksforgeeks.org/software-engineering-white-box-testing/
2. https://www.javatpoint.com/white-box-testing
3. https://www.browserstack.com/guide/white-box-testing
4. https://www.guru99.com/white-box-testing.html
5. https://dotnettutorials.net/lesson/white-box-testing/

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	**/5**
2	Test Case Design	**/8**
3	Test Case Execution	**/4**
4	Correctness of Practical Implementation	**/4**
5	Timely Submission of Report and Answer to Questions	**/4**
	Total	**/25**

Signature with Date

Practical No.5: Design test cases for different tasks (OTP Verification, Image upload, Age verification in Registration) in any software modules using Equivalence partitioning, boundary value analysis, and decision table testing techniques of Black Box Testing.

A. Objective:

Cover different aspects of the software's functionality, ensuring that it behaves as expected under various conditions. Equivalence Partitioning focuses on grouping input values into classes, Boundary Value Analysis tests values at the edges of these classes, and Decision Table Testing addresses combinations of input conditions and their expected results. These techniques help maximize test coverage and identify potential issues early in the testing process.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

2. **Problem analysis**: Identify and analyze well-defined engineering problems using codified standard methods.

3. **Design/ development of solutions:** Design solutions for engineering well-defined technical problems and assist with the design of systems components or processes to meet specified needs.

4. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.

6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.

7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

- To validate software functionality without knowing its internal code.
- Utilize various methods of black box testing to design test cases for function requirements.

D. Expected Course Outcomes (Cos)

Apply black-box and white-box testing techniques effectively.

E. Practical Outcome (PRo)

Design test cases for tasks such as OTP Verification, Image upload, Age verification using various methods of Black box testing.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

Black Box Testing:

Black Box Testing is a software testing method where the tester examines the functionality of a system without having knowledge of its internal code, structure, or implementation details. The tester focuses solely on the inputs and outputs of the software under test, treating it as a "black box" where the internal workings are not visible or known.

Black Box Testing helps identify defects, errors, and discrepancies between expected and actual outcomes without being influenced by the internal structure of the system. This type of testing is essential for ensuring the overall quality and reliability of software applications.

Black Box Testing involves various methods and techniques to ensure that the software functions correctly according to its specifications without focusing on the internal code structure. There are various methods of Black Box testing.Here are some common methods of Black Box Testing:

Equivalence Partitioning:

Equivalence Partitioning is a black-box testing technique that divides the input space of a software system into classes or groups of data, where each class represents a set of equivalent inputs. The idea is that if a system behaves correctly for one input in an equivalence class, it should behave correctly for any input in that class. This technique helps

in designing effective and efficient test cases by reducing redundancy and covering a representative set of inputs.

Example:

Consider a simple login functionality where the user has to enter a username and password. The requirements specify that the username must be between 5 and 15 characters, and the password must be at least 8 characters long. Equivalence Partitioning can be applied to design test cases for valid and invalid inputs.

1. **Equivalence Classes for Username:**
   - Valid Usernames: "john_doe", "user123"
   - Invalid Usernames (Too Short): "ab"
   - Invalid Usernames (Too Long): "thisisaverylongusername"
2. **Equivalence Classes for Password:**
   - Valid Passwords: "secure123", "myp@ssword"
   - Invalid Passwords (Too Short): "pass123"
   - Invalid Passwords (Valid Length but Weak): "weakpwd"

Equivalence Partitioning Test Cases:

Based on the equivalence classes defined, we can design test cases to cover each class:

1. **Valid Username and Valid Password:**
   - Input: Username = "john_doe", Password = "secure123"
   - Expected Result: Successful login
2. **Invalid Username (Too Short) and Valid Password:**
   - Input: Username = "ab", Password = "myp@ssword"
   - Expected Result: Login failure due to an invalid username length
3. **Valid Username and Invalid Password (Too Short):**
   - Input: Username = "user123", Password = "pass123"
   - Expected Result: Login failure due to an invalid password length
4. **Invalid Username (Too Long) and Invalid Password (Valid Length but Weak):**
   - Input: Username = "thisisaverylongusername", Password = "weakpwd"
   - Expected Result: Login failure due to both invalid username length and weak password
5. **Valid Username and Invalid Password (Valid Length but Weak):**
   - Input: Username = "john_doe", Password = "weakpwd"
   - Expected Result: Login failure due to a weak password

These test cases cover different equivalence classes, ensuring that the software is tested for various scenarios and that it responds correctly to both valid and invalid inputs within each class.

Boundary Value Analysis (BVA):

Boundary Value Analysis is a black-box testing technique that focuses on testing values at the boundaries or edges of input ranges. The idea is that errors often occur at the extremes of the input domain, and testing values at these boundaries helps ensure the robustness of

The software. BVA is commonly used in scenarios where input values must fall within a specific range.

Process of Boundary Value Analysis:

1. **Identify Input Ranges:** Determine the input ranges or domains that the software operates within. This could include ranges for variables, such as age, salary, or any other numeric or alphanumeric input.
2. **Select Boundary Values:** Choose values that are at the lower, upper, and just beyond the boundaries of the identified ranges. These values are often more likely to cause errors.
3. **Design Test Cases:** Create test cases using the selected boundary values. Test cases should include both valid and invalid scenarios to ensure that the software behaves correctly in all situations.
4. **Execute Test Cases:** Run the test cases and observe the software's behavior. Check if it handles boundary values properly and produces the expected results.

Example of Boundary Value Analysis:

Let's consider a simple example of a program that accepts an age input between 18 and 65. The program determines whether a person is eligible for a certain service based on their age. Here's how you might apply Boundary Value Analysis to this scenario:

1. **Identify Input Ranges:**
    - Age should be between 18 and 65 (inclusive).
2. **Select Boundary Values:**
    - Lower Bound: 18 (just above the minimum)
    - Upper Bound: 65 (just below the maximum)
    - Values just beyond the bounds: 17 (below the minimum) and 66 (above the maximum)
3. **Design Test Cases:**
    - Test Case 1: Age = 17 (Invalid - below the minimum)
    - Test Case 2: Age = 18 (Valid)
    - Test Case 3: Age = 30 (Valid)
    - Test Case 4: Age = 65 (Valid)
    - Test Case 5: Age = 66 (Invalid - above the maximum)
4. **Execute Test Cases:**
    - Check if the program correctly identifies Test Case 1 and Test Case 5 as invalid, while Test Cases 2, 3, and 4 are valid.

Benefits of Boundary Value Analysis:

- Helps identify potential errors at the edges of input ranges.
- Provides more thorough coverage of input scenarios.
- Efficiently targets areas where errors are likely to occur.

In this example, applying Boundary Value Analysis ensures that the age-related functionality is tested comprehensively, increasing the likelihood of finding defects related to boundary conditions.

Decision Table Testing is a black-box testing technique that is used to handle complex business rules and logic in a concise and systematic manner. It involves creating a decision table that represents all possible combinations of input conditions and their corresponding actions or outcomes. This method helps ensure that every possible combination is tested, making it effective for scenarios where different combinations of inputs can lead to different outcomes.

Components of a Decision Table:

- **Conditions (Inputs):** Represent the different input conditions that the system can encounter.
- **Actions (Outputs):** Represent the different actions or outcomes based on the combinations of input conditions.
- **Rules:** Define the relationship between input conditions and the corresponding actions.

Example of Decision Table Testing:

Let's consider a simple example of a decision table for a system that determines the eligibility of a person for a discount based on their age and membership status. The conditions are "Age" and "Membership," and the action is whether the person is "Eligible" for a discount.

Condition 1 (Age)	Condition 2 (Membership)	Action (Eligibility)
Young (< 30)	Yes	Eligible
Young (< 30)	No	Not Eligible
Middle-aged (30-60)	Yes	Eligible
Middle-aged (30-60)	No	Not Eligible
Senior (> 60)	Yes	Eligible
Senior (> 60)	No	Not Eligible

In this example:

- **Conditions:**
  - Age is divided into three categories: Young (< 30), Middle-aged (30-60), and Senior (> 60).
  - Membership is divided into two categories: Yes or No.
- **Actions:**
  - Eligibility has two possible outcomes: Eligible or Not Eligible.

The decision table systematically covers all possible combinations of conditions and their corresponding outcomes. For instance, the first row represents the scenario where a person is young (< 30) and has membership, leading to the action "Eligible."

Test Cases: Based on the decision table, we can create specific test cases to validate the system's behavior:

1. Test Case 1:
    - Age: Young (< 30)
    - Membership: Yes
    - Expected Outcome: Eligible
2. Test Case 2:
    - Age: Middle-aged (30-60)
    - Membership: No
    - Expected Outcome: Not Eligible
3. Test Case 3:
    - Age: Senior (> 60)
    - Membership: Yes
    - Expected Outcome: Eligible
4. Test Case 4:
    - Age: Middle-aged (30-60)
    - Membership: Yes
    - Expected Outcome: Eligible

These test cases cover different combinations of conditions, ensuring that the system behaves as expected for various scenarios. Decision Table Testing provides a systematic way to design test cases for complex decision-making processes, helping to identify potential errors and validate the correctness of the system's logic.

In summary, Black Box Testing is an essential methodology for ensuring the functionality, reliability, and compliance of a software system with its specified requirements. It complements other testing methodologies and helps deliver a high-quality software product to end-users.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM

J. Practical related Exercises.

1. **In Equivalence Partitioning, why is it important to choose test cases from each partition?**
   A. To cover all possible input values.
   B. To ensure that only extreme values are tested.
   C. To reduce the number of test cases.
   D. To achieve higher test coverage with fewer test cases.

2. **What is the primary objective of Boundary Value Analysis in Black Box Testing?**
   A. To focus on middle values within input ranges.
   B. To identify defects related to input extremes and boundaries.
   C. To exclude extreme values from testing.
   D. To prioritize testing based on equivalence partitions.

3. **What is the primary purpose of Decision Table Testing in Black Box Testing?**
   A. To create a table of all possible inputs without considering outcomes.
   B. To systematically design test cases for complex decision-making processes.
   C. To exclude certain input conditions from testing.
   D. To focus only on the most critical input conditions.

4. **Equivalence Partitioning is a technique used to:**
   A. Test every possible input individually.
   B. Divide the input domain into classes and select representative test cases.
   C. Identify boundary values for input variables.
   D. Create a decision table for testing.

5. **Decision Table Testing is particularly useful for testing:**

A. Single input variables.
B. Complex decision-making processes with multiple input conditions.
C. Random combinations of input values.
D. Only valid input values.

K. References / Suggestions

1. https://www.tutorialspoint.com/boundary-value-analysis-and-equivalence-partitioning-with-examples
2. https://www.guru99.com/equivalence-partitioning-boundary-value-analysis.html
3. https://www.geeksforgeeks.org/software-testing-boundary-value-analysis-vs-equivalence-partitioning/
4. https://www.softwaretestinghelp.com/what-is-boundary-value-analysis-and-equivalence-partitioning/
5. https://www.javatpoint.com/boundary-value-analysis-vs-equivalence-partitioning

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	/25

Signature with Date

Practical No.6:

Identify system specification & design test cases for Sales Invoice Management.

A. Objective:

To ensure the robustness, functionality, and reliability of the Sales Invoice Management system and Flight Ticket Booking system by establishing a clear understanding of its requirements and creating effective test cases.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

2. **Problem analysis**: Identify and analyze well-defined engineering problems using codified standard methods.

3. **Design/ development of solutions:** Design solutions for engineering well-defined technical problems and assist with the design of systems components or processes to meet specified needs.

4. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.

6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.

7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

1. Identify system with its specification and get familiar with the system.
2. Use the different software testing concepts.
3. Ability to write system specification of the systems.
4. Ability to design test cases for given systems.

D. Expected Course Outcomes (Cos)

Apply black-box and white-box testing techniques effectively.

E. Practical Outcome (PRo)

List out various specification of software and design various test cases as per mentioned in specification.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

A test case template is a standardized document that outlines the conditions, inputs, expected results, and execution steps for a specific test scenario. This template provides a structured format for organizing and documenting test cases, ensuring consistency across testing efforts.

Below is a detailed description of the various components typically included in a test case template, along with example? However you can use less or more components as per need of requirements of application.

Test Case Template Components:

1. **Test Case ID:**
   - A unique identifier for each test case, often a combination of letters and numbers.

   Example: TC_SIM_001
2. **Test Case Title:**
   - A concise and descriptive title that summarizes the objective of the test case.

   Example: Verify Creation of Sales Invoice
3. **Description:**
   - A brief description providing context and background information about the test case.

   Example: This test case aims to verify the correct creation of a sales invoice in the Sales Invoice Management system.
4. **Preconditions:**
   - Conditions that must be met before executing the test case. This ensures a consistent starting point for the test.

   Example: The user is logged into the Sales Invoice Management system with the necessary permissions.

5. **Inputs:**
    - Detailed information about the data or parameters required for the test.

   Example:
    - Customer information
    - Product details
    - Invoice amount
6. **Test Steps:**
    - Step-by-step instructions for executing the test, including the necessary actions and inputs.

   Example:

   Navigate to the "Create Invoice" section.
   Enter customer details (e.g., name, address).
   Add products/services to the invoice.
   Input invoice amount and relevant details.
   Click on the "Submit" button.

   Expected Results:
    - The anticipated outcome or behavior of the system after executing the test steps.

   Example: A new sales invoice is created and saved successfully, and the system displays a confirmation message.

   Actual Results:
    - The actual outcome observed during test execution. Testers record the results after completing the test.

   Example: The sales invoice is successfully created, and the confirmation message appears as expected.

   Pass/Fail Criteria:
    - Criteria for determining whether the test case has passed or failed based on the actual results.

   Example:
    - Pass: The invoice is created without errors, and the confirmation message is displayed.
    - Fail: The system generates an error message, or the invoice creation process fails.

   Notes:
    - Additional comments, observations, or notes that provide context or insight into the test execution.

   Example: The test was performed on the latest version of the system, and no issues were encountered during the test execution.

   Test Environment/Setup:
    - Information about the testing environment, including hardware, software, and configurations used during testing.

Example: Tested on the Sales Invoice Management system version 2.0 in a Windows 10 environment.

Tested By:

- The name or identifier of the tester who executed the test case.

Example: John Tester

Date Tested:

- The date when the test case was executed.

Example: January 15, 2024

Suggested template for design Test case is given below.

Test Case ID	Test Case Title	Description	Test Data	Test Steps	Expected Output	Actual Output	Status	Date	Created By

Explanation of Columns:

1. **Test Case ID:** A unique identifier for each test case.
2. **Test Case Title:** A concise title describing the purpose of the test case.
3. **Test Case Description:** A detailed explanation of the test scenario, including the context and objectives.
4. **Test Data:** Detailed information about the data or parameters required for the test.
5. **Test Steps:** Sequential steps to execute the test case.
6. **Expected Result:** The anticipated outcome when the test case is executed successfully.
7. **Actual Result:** The actual outcome observed during test execution (filled in after execution).
8. **Status (Pass/Fail):** Indication of whether the test case passed or failed based on the comparison of actual and expected results.
9. **Date:** The name or identifier of the tester who executed the test case
10. **Created By:** The date when the test case was executed.

Sales Invoice Management:

Sales Invoice Management is a critical component of business operations that involves the creation, tracking, and processing of invoices for goods or services sold to customers. The system typically includes features for generating invoices, managing customer information, tracking payment status, and generating reports. Ensuring the functionality, accuracy, and security of a Sales Invoice Management system is crucial for maintaining financial records and sustaining smooth business operations.

1. Specification: Invoice Creation
Description:

- The system should allow users to create new invoices with accurate and complete information.
- Mandatory fields include customer details, product details, quantity, and amount.

2. Specification: Invoice Editing
Description:

- Users with the appropriate permissions should be able to edit existing invoices.
- The system must maintain accurate records and provide audit trails for changes.

3. Specification: Duplicate Invoice Prevention
Description:

- The system should prevent the creation of invoices with duplicate invoice numbers.

4. Specification: Invoice Deletion
Description:

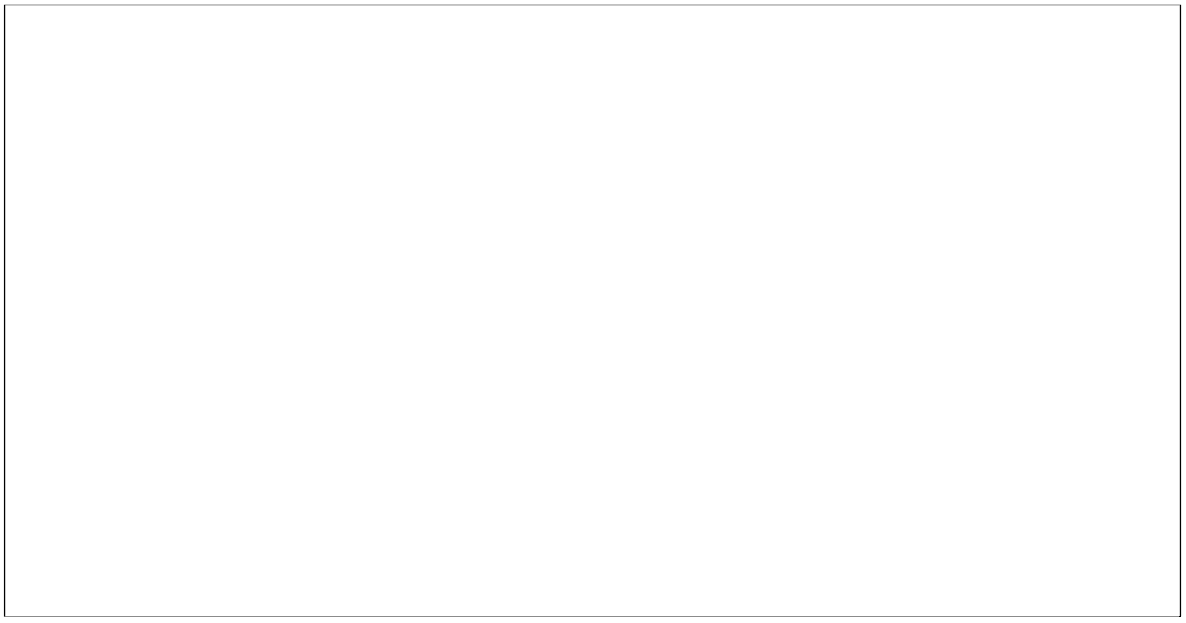
- Users with the necessary permissions should be able to delete invoices.

These specifications related to invoice creation, editing, duplicate prevention, and deletion. **Additional specifications can be identified** based on specific business requirements and functionalities desired in the Sales Invoice Management system.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

Note: **students have to design various test cases consider above and other needed specification using given above test cases format.**



J. Practical related Exercises.

1. **What is the role of a test case in the software development life cycle (SDLC)?**
   a. To replace code reviews
   b. To document software requirements
   c. To ensure the quality of the software
   d. To create user interfaces
2. **Which of the following is NOT a component of a test case?**
   a. Test Data
   b. Test Environment
   c. Test Plan
   d. Expected Result
3. **When is the best time to create test cases in the SDLC?**
   a. After the software is deployed
   b. During the design phase
   c. After coding is complete
   d. During the requirements phase
4. **Which of the following is not a component of a test case?**
   a. Test Steps
   b. Expected Result
   c. Bug Report
   d. Actual Result

5. **What is the purpose of test data in a test case?**
   a. To make the test case more complex
   b. To confuse the tester
   c. To provide input values for the test case
   d. To slow down the testing process

K. References / Suggestions

1. https://www.geeksforgeeks.org/test-case/
2. https://www.guru99.com/test-case.html
3. https://www.tutorialspoint.com/software_testing_dictionary/test_case.htm
4. https://testsigma.com/guides/what-is-test-case/

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	/25

Signature with Date

Practical No.7
Design Test Cases for Flight Ticket Booking system.

A. Objective:

To ensure the robustness, functionality, and reliability of the Sales Invoice Management system and Flight Ticket Booking system by establishing a clear understanding of its requirements and creating effective test cases.

B. Expected Program Outcomes (POs)

a. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

b. **Problem analysis**: Identify and analyze well-defined engineering problems using codified standard methods.

c. **Design/ development of solutions:** Design solutions for engineering well-defined technical problems and assist with the design of systems components or processes to meet specified needs.

d. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

e. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.

f. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.

g. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

a. Identify system with its specification and get familiar with the system.
b. Use the different software testing concepts.
c. Ability to write system specification of the systems.
d. Ability to design test cases for given systems.

D. Expected Course Outcomes (Cos)

Apply black-box and white-box testing techniques effectively.

E. Practical Outcome (PRo)

List out various specification of software and design various test cases as per mentioned in specification.

F. Expected Affective domain Outcome (ADos)

a. Follow Testing standards and practices.
b. Maintain tools and equipment.
c. Follow safety practices.
d. Follow ethical practices

G. Prerequisite Theory:

A test case template is a standardized document that outlines the conditions, inputs, expected results, and execution steps for a specific test scenario. This template provides a structured format for organizing and documenting test cases, ensuring consistency across testing efforts.
Below is a detailed description of the various components typically included in a test case template, along with example? However you can use less or more components as per need of requirements of application.

Test Case Template Components:

1. **Test Case ID:**
   - A unique identifier for each test case, often a combination of letters and numbers.

   Example: TC_SIM_001
2. **Test Case Title:**
   - A concise and descriptive title that summarizes the objective of the test case.

   Example: Verify Creation of Sales Invoice
3. **Description:**
   a. A brief description providing context and background information about the test case.

   Example: This test case aims to verify the correct creation of a sales invoice in the Sales Invoice Management system.
4. **Preconditions:**
   a. Conditions that must be met before executing the test case. This ensures a consistent starting point for the test.

   Example: The user is logged into the Sales Invoice Management system with the necessary permissions.

5. **Inputs:**
    a. Detailed information about the data or parameters required for the test example.
    b. Customer information
    c. Product details
    d. Invoice amount
6. **Test Steps:**
    a. Step-by-step instructions for executing the test, including the necessary actions and inputs.

Example:

Navigate to the "Create Invoice" section.
Enter customer details (e.g., name, address).
Add products/services to the invoice.
Input invoice amount and relevant details.
Click on the "Submit" button.

Expected Results:

b. The anticipated outcome or behavior of the system after executing the test steps.

Example: A new sales invoice is created and saved successfully, and the system displays a confirmation message.

Actual Results:

c. The actual outcome observed during test execution. Testers record the results after completing the test.

Example: The sales invoice is successfully created, and the confirmation message appears as expected.

Pass/Fail Criteria:

d. Criteria for determining whether the test case has passed or failed based on the actual results.

Example:

e. Pass: The invoice is created without errors, and the confirmation message is displayed.
f. Fail: The system generates an error message, or the invoice creation process fails.

Notes:

g. Additional comments, observations, or notes that provide context or insight into the test execution.

Example: The test was performed on the latest version of the system, and no issues were encountered during the test execution.

Test Environment/Setup:

h. Information about the testing environment, including hardware, software, and configurations used during testing.

Example: Tested on the Sales Invoice Management system version 2.0 in a Windows 10 environment.

Tested By:

i. The name or identifier of the tester who executed the test case.

Example: John Tester

Date Tested:

j. The date when the test case was executed.

Example: January 15, 2024

Suggested template for design Test case is given below.

Test Case ID	Test Case Title	Description	Test Data	Test Steps	Expected Output	Actual Output	Status	Date	Created By

Explanation of Columns:

1. **Test Case ID:** A unique identifier for each test case.
2. **Test Case Title:** A concise title describing the purpose of the test case.
3. **Test Case Description:** A detailed explanation of the test scenario, including the context and objectives.
4. **Test Data:** Detailed information about the data or parameters required for the test.
5. **Test Steps:** Sequential steps to execute the test case.
6. **Expected Result:** The anticipated outcome when the test case is executed successfully.
7. **Actual Result:** The actual outcome observed during test execution (filled in after execution).
8. **Status (Pass/Fail):** Indication of whether the test case passed or failed based on the comparison of actual and expected results.
9. **Date:** The name or identifier of the tester who executed the test case
10. **Created By:** The date when the test case was executed.

Flight Booking System:

A Flight Booking System facilitates the reservation and management of airline tickets. It involves various functionalities such as searching for flights, selecting seats, making reservations, and managing bookings. Here are suggested specifications for a Flight Booking System:

1. **User Authentication and Authorization:**
   - *Specification:* Users should be able to log in with valid credentials, and the system should enforce proper access controls based on user roles.
2. **Flight Search and Availability:**
   a. *Specification:* Users should be able to search for available flights based on criteria such as date, destination, and class.
3. **Reservation Process:**
   a. *Specification:* Users should be able to reserve seats on selected flights, and the system should update seat availability accordingly.
4. **Payment and Billing:**
   a. *Specification:* The system should securely handle payment transactions and generate accurate billing information.
5. **Booking Modification and Cancellation:**
   a. *Specification:* Users should be able to modify or cancel their bookings within specified rules and time frames.
6. **Flight Status and Notifications:**
   a. *Specification:* The system should provide real-time flight status updates, and users should receive timely notifications about any changes.
7. **Cross-browser Compatibility:**
   a. *Specification:* The system should be accessible and function consistently across various web browsers.
8. **Mobile Responsiveness:**
   a. *Specification:* The system should provide a responsive design for smooth user experience on various devices, including smartphones and tablets.
9. **Security Measures:**
   a. *Specification:* The system should implement robust security measures to protect user data and transactions.

These specifications aim to cover a range of functionalities in a Flight Booking System, ensuring its reliability, security, and user-friendliness.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

J. References / Suggestions

a. https://www.geeksforgeeks.org/test-case/
b. https://www.guru99.com/test-case.html
c. https://www.tutorialspoint.com/software_testing_dictionary/test_case.htm
d. https://testsigma.com/guides/what-is-test-case/

K. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	/25

Signature with Date

Practical No.8 Develop test scenarios and test cases for the login functionality of a social media application.

A. **Objective:** Create comprehensive test scenarios and cases to rigorously validate the login functionality of a social media application, ensuring robust security, usability, and seamless user experience.

B. **Expected Program Outcomes (POs)**

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the *engineering* problems.
2. **Problem analysis**: Identify and analyze well-defined *engineering* problems using codified standard methods.
3. **Design/ development of solutions:** Design solutions for *engineering* well-defined technical problems and assist with the design of systems components or processes to meet specified needs.
4. **Engineering Tools, Experimentation and Testing:** Apply modern *engineering* tools and appropriate technique to conduct standard tests and measurements.
5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.
6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.
7. **Life-long learning: Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.**

C. **Expected Skills to be developed based on competency:**

This practical is expect to develop the following skills in you apply types, levels and methods of software testing on applications.

1. Identify system with its specification and get familiar with the system.
2. Use the different software testing concepts.
3. Ability to write system specification of the systems.
4. Ability to design test case for given systems.

D. **Expected Course Outcomes (Cos)**

Develop effective test cases based on specifications, utilizing appropriate templates and methodologies.

E. Practical Outcome (PRo)

Create test scenarios and cases to validate the seamless and secure login process for a social media application, covering aspects such as valid and invalid credentials, multi-factor authentication, and account recovery.

F. Expected Affective domain Outcome (ADos)

5. Follow Testing standards and practices.
6. Maintain tools and equipment.
7. Follow safety practices.
8. Follow ethical practices

G. Prerequisite Theory:

The process of developing test scenarios and test cases for the login functionality of a social media application is crucial to ensure the reliability, security, and user experience of the authentication process. This theory outlines the key aspects and methodologies involved in this testing phase. Test scenarios and test cases for the login functionality of a social media application.

1. Valid Login:

- **Scenario:** In this scenario, a user enters correct credentials to log in successfully. The primary objective is to ensure that the application allows authorized users to access their accounts. Test cases involve navigating to the login page, entering a valid username and password, and clicking the "Login" button. The expected result is a successful login, and verification includes checking if the user is redirected to the home page, and user-specific elements are present.

2. Invalid Login:

- **Scenario:** This scenario focuses on users entering incorrect credentials during login attempts. Test cases cover various scenarios such as invalid username, invalid password, and a combination of both. For instance, entering an invalid username, a valid password, and clicking "Login" should result in an error message indicating invalid credentials. Verification involves checking the user-friendliness of error messages.

3. Account Lockout:

- **Scenario:** This scenario tests the application's response to a certain number of consecutive failed login attempts, leading to an account lockout. Test cases include attempting to exceed the maximum allowed failed login attempts, ensuring the account is locked, and verifying the reset of the lockout status after a specified duration. Verification involves checking the account status in the database.

4. Password Recovery:

- **Scenario:** Here, the focus is on users who forget their passwords and initiate the recovery process. Test cases cover initiating password recovery, receiving a password reset link via email, and successfully setting a new password using the provided link. Verification

Includes checking the inbox for the reset link and ensuring successful login with the new password.

5. Social Media Integration:

- **Scenario:** This scenario involves users logging in using their social media credentials. Test cases include logging in with Facebook and Google accounts, entering valid credentials, and verifying successful login. Verification involves confirming that user details are populated from the respective social media profiles.

6. Session Management:

- **Scenario:** The objective here is to test the secure management of user sessions. Test cases include testing session timeouts and multiple device logins. For instance, logging in and waiting for the session to expire should prompt the user to log in again. Verification involves confirming that the session timeout behavior aligns with expectations.

7. UI/UX:

- **Scenario:** This scenario focuses on the user interface and user experience of the login page. Test cases include verifying the presence of necessary UI elements, testing responsiveness across different screen sizes, and checking the visibility of error messages. Verification involves visually inspecting the login page and confirming the responsiveness using browser developer tools.

8. Security:

- **Scenario:** Ensuring the security of the login process is crucial. Test cases cover potential vulnerabilities such as SQL injection and Cross-Site Scripting (XSS). For example, attempting to log in with a username containing SQL injection should result in failure without executing malicious queries. Verification involves checking logs for any suspicious activity and ensuring the application is not vulnerable to XSS.

These detailed test scenarios and test cases aim to comprehensively assess the login functionality of a social media application, covering a range of potential user interactions and security considerations. Customizing these scenarios based on the specific features and requirements of the application is essential for effective testing.

Example: Invalid Login Scenario: A user enters incorrect credentials and attempts to log in.

- **Test Cases:**
    1. Enter a valid username and an incorrect password.
    2. Enter an invalid username and a valid password.
    3. Enter an invalid username and an incorrect password.
    4. Verify that appropriate error messages are displayed for each case.

Account Lockout Scenario: After a certain number of consecutive failed login attempts, the account gets locked.

- **Test Cases:**
    1. Exceed the maximum allowed failed login attempts.
    2. Verify that the account is locked.
    3. Attempt to log in with correct credentials after the account is locked.
    4. Verify that an appropriate error message is displayed.

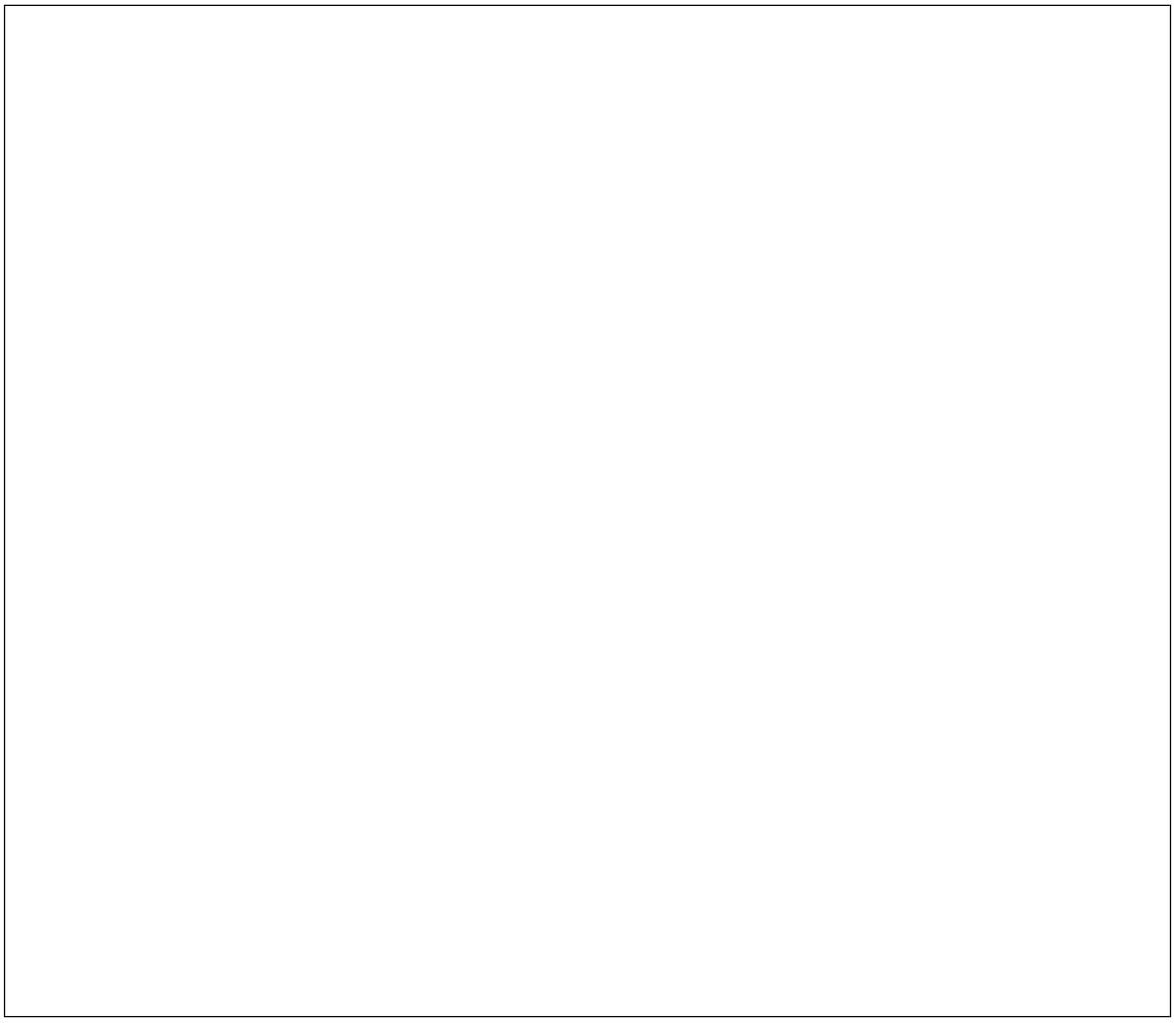
  These detailed test scenarios and test cases provide a comprehensive approach to testing the login functionality of a social media application

  Note: For login page remaining scenarios and test cases will be designed by student as exercise.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

I. Practical Implementation: Design scenarios and Test cases for remaining functionalities given in theory section.



J. Practical related Questions.

1. What is the primary purpose of a Test Scenario?
   a. To define specific test cases
   b. To identify defects in the software
   c. To describe the end-to-end functionality of a feature
   d. To document the overall testing strategy
2. Which of the following is NOT a characteristic of a good Test Case?
   a. Clear and concise
   b. Dependent on other test cases
   c. Repeatable
   d. Measurable
3. What does a Test Case typically include?
   a. Expected outcomes
   b. Test data
   c. Test steps

d. All of the above

4. Which type of testing is mainly associated with Test Cases?
   a. Unit Testing
   b. System Testing
   c. Integration Testing
   d. Acceptance Testing
5. Which of the following is a characteristic of a well-defined Test Case?
   a. Ambiguous and unclear steps.
   b. Multiple expected outcomes.
   c. A single test objective.
   d. Unstructured and lengthy description.

K. References Links

1. Test Scenario - javatpoint
2. What is Test Scenario | Purpose & How to create it (Examples) (testsigma.com)
3. Test Case vs Test Scenario – Difference Between Them (guru99.com)
4. Testscenario - Independent Software Testing Services Company

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	/25

Signature with Date

Practical No.9 Develop an RTM and measure testing metrics for any two dynamic web pages of an e-commerce website.

A. **Objective:** Create a Requirement Traceability Matrix (RTM) for two dynamic web pages of an e-commerce site, outlining the relationship between requirements and test cases. Measure testing metrics such as test coverage, defect density, and execution time to assess the effectiveness of the testing process.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the *engineering* problems.
2. **Problem analysis**: Identify and analyze well-defined *engineering* problems using codified standard methods.
3. **Design/ development of solutions:** Design solutions for *engineering* well-defined technical problems and assist with the design of systems components or processes to meet specified needs.
4. **Engineering Tools, Experimentation and Testing:** Apply modern *engineering* tools and appropriate technique to conduct standard tests and measurements.
5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.
6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.
7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering

C. Expected Skills to be developed based on competency:

This practical is expect to develop the following skills in you apply types, levels and methods of software testing on applications.

1. Identify system with its specification and get familiar with the system.
2. Use the different software testing concepts.
3. Ability to write system specification of the systems.
4. Ability to design test case for given systems.

D. Expected Course Outcomes (Cos)

Develop effective test cases based on specifications, utilizing appropriate templates and methodologies.

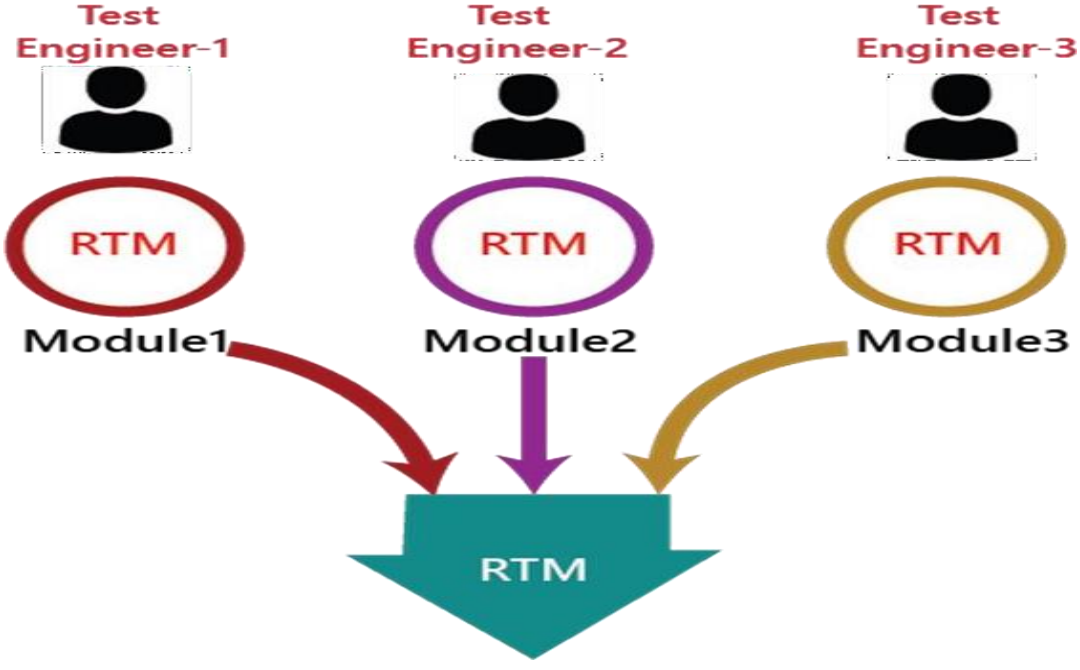
E. Practical Outcome (PRo)

Create a Requirements Traceability Matrix (RTM) and measure testing metrics, such as test coverage and defect density, for the product listing and checkout pages of an e-commerce website.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory: Requirements Traceability Matrix (RTM): Traceability matrix is a table type document that is used in the development of software application to trace requirements. It can be used for both forward (from Requirements to Design or Coding) and backward (from Coding to Requirements) tracing. It is also known as Requirement Traceability Matrix (RTM) or Cross Reference Matrix (CRM).



It is prepared before the test execution process to make sure that every requirement is covered in the form of a Test case so that we don't miss out any testing. In the RTM document, we map all the requirements and corresponding test cases to ensure that we have written all the test cases for each condition.

The test engineer will prepare RTM for their respective assign modules, and then it will be sent to the Test Lead. The Test Lead will go repository to check whether the Test Case is there or not and finally Test Lead consolidate and prepare one necessary RTM document**.**

This document is designed to make sure that each requirement has a test case, and the test case is written based on business needs, which are given by the client. It will be performed with the help of the test cases if any requirement is missing, which means that the test case is not written for a particular need, and that specific requirement is not tested because it may have some bugs. The traceability is written to make sure that the entire requirement is covered.

A Requirements Traceability Matrix (RTM) is a document that establishes a relationship between system requirements and the corresponding test cases. It is used to ensure that all requirements are covered by test cases and to trace the impact of changes in requirements throughout the testing process. The RTM helps in maintaining transparency, managing changes, and ensuring comprehensive test coverage.

Components of an RTM:

1. **Requirement ID:** A unique identifier for each requirement.
2. **Requirement Description:** A clear and concise description of each requirement.
3. **Test Case ID(s):** The test case(s) associated with each requirement.

Requirements Traceability Matrix (RTM) Template:

Requirement ID	Requirement Description	Test Case ID(s)
REQ001	User should be able to log in.	TC001, TC002
REQ002	User should be able to search for books.	TC003, TC004
REQ003	System should display available books.	TC005, TC006
REQ004	User should be able to borrow a book.	TC007, TC008
REQ005	System should notify late book returns.	TC009, TC010
...	...	...

Purpose of RTM:

- **Traceability:** Link requirements to test cases, ensuring coverage and traceability.
- **Change Management:** Track changes in requirements and update associated test cases.
- **Impact Analysis:** Understand the impact of requirement changes on testing.
- **Coverage Analysis:** Assess how well test cases cover the specified requirements.

- **Risk Management:** Identify areas with incomplete or no test coverage.

Testing Metrics in Software Testing: Software testing metrics are quantifiable indicators of the software testing process progress, quality, productivity, and overall health. The purpose of software testing metrics is to increase the efficiency and effectiveness of the software testing process while also assisting in making better decisions for future testing by providing accurate data about the testing process. A metric expresses the degree to which a system, system component, or process possesses a certain attribute in numerical terms. A weekly mileage of an automobile compared to its ideal mileage specified by the manufacturer is an excellent illustration of metrics. Here, we discuss the following points.

Software Testing Metrics are the quantitative measures used to estimate the progress, quality, productivity and health of the software testing process. The goal of software testing metrics is to improve the efficiency and effectiveness in the software testing process and to help make better decisions for further testing process by providing reliable data about the testing process.

A Metric defines in quantitative terms the degree to which a system, system component, or process possesses a given attribute. The ideal example to understand metrics would be a weekly mileage of a car compared to its ideal mileage recommended by the manufacturer. Testing metrics are quantitative measures used to assess various aspects of the testing process and the quality of the software being tested. These metrics provide insights into the effectiveness, efficiency, and progress of the testing activities. Types of Software Testing Metrics. Software testing metrics are divided into three categories:

Process Metrics: A project's characteristics and execution are defined by process metrics. These features are critical to the SDLC process's improvement and maintenance (Software Development Life Cycle).

Product Metrics: A product's size, design, performance, quality, and complexity are defined by product metrics. Developers can improve the quality of their software development by utilizing these features.

Project Metrics: Project Metrics are used to assess a project's overall quality. It is used to estimate a project's resources and deliverables, as well as to determine costs, productivity, and flaws.



Identification of correct testing metrics is very important. Few things need to be considered before identifying the test metrics

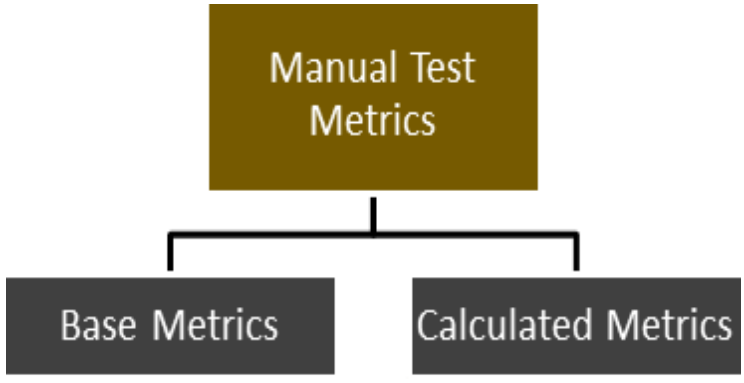
- Fix the target audience for the metric preparation
- Define the goal for metrics
- Introduce all the relevant metrics based on project needs
- Analyze the cost benefits aspect of each metrics and the project lifestyle phase in which it results in the maximum output

Manual Test Metrics: Manual testing is carried out in a step-by-step manner by quality assurance experts. Test automation frameworks, tools, and software are used to execute tests in automated testing. There are advantages and disadvantages to both human and automated testing. Manual testing is a time-consuming technique, but it allows testers to deal with more complicated circumstances. There are two sorts of manual test metrics**:**In Software Engineering, Manual test metrics are classified into two classes.

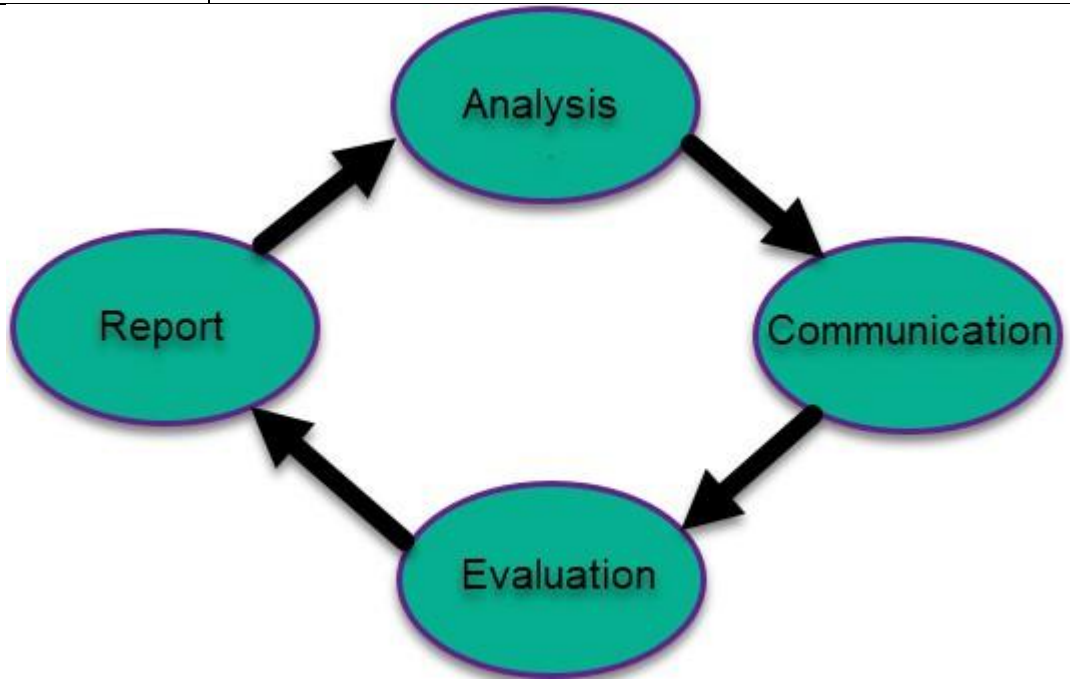
(1) Base Metrics: Analysts collect data throughout the development and execution of test cases to provide base metrics. By generating a project status report, these metrics are sent to test leads and project managers. It is quantified using calculated metrics.

- The total number of test cases
- The total number of test cases completed.

(2) Calculated Metrics: Data from base metrics are used to create calculated metrics. The test lead collects this information and transforms it into more useful information for tracking project progress at the module, tester, and other levels. It's an important aspect of the SDLC since it allows developers to make critical software changes.



Test Metrics Life Cycle: The below diagram illustrates the different stages in the test metrics life cycle.

Different stages of Metrics life cycle	Steps during each stage
Analysis	1. Identification of the Metrics 2. Define the identified QA Metrics
	
Communicate	1. Explain the need for metric to stakeholder and testing team 2. Educate the testing team about the data points to need to be captured for processing the metric
Evaluation	1. Capture and verify the data 2. Calculating the metrics value using the data captured
Report	1. Develop the report with an effective conclusion 2. Distribute the report to the stakeholder and respective representative 3. Take feedback from stakeholder

Considerations for Testing Metrics:

- Metrics should align with project goals and objectives.
- Regularly review and refine metrics to ensure relevance.
- Use metrics to drive improvement rather than as a punitive measure.
- Metrics should be complemented with qualitative assessments.

Formula for Test Metrics

To get the percentage execution status of the test cases, the following formula can be used:

Percentage test cases executed = (No of test cases executed / Total no of test cases written) x 100

Similarly, it is possible to calculate for other parameters also such as test cases that were not executed, test cases that were passed, test cases that were failed, and test cases that were blocked, and so on. Below are some of the formulas:

1. Test Case Effectiveness:

Test Case Effectiveness = (Number of defects detected / Number of test cases run) x 100

2. Passed Test Cases Percentage: Test Cases that Passed Coverage is a metric that indicates the percentage of test cases that pass.

Passed Test Cases Percentage = (Total number of tests ran / Total number of tests executed) x 100

3. Failed Test Cases Percentage: This metric measures the proportion of all failed test cases.

Failed Test Cases Percentage = (Total number of failed test cases / Total number of tests executed) x 100

4. Blocked Test Cases Percentage: During the software testing process, this parameter determines the percentage of test cases that are blocked.

Blocked Test Cases Percentage = (Total number of blocked tests / Total number of tests executed) x 100

5. Fixed Defects Percentage: Using this measure, the team may determine the percentage of defects that have been fixed.

Fixed Defects Percentage = (Total number of flaws fixed / Number of defects reported) x 100

6. Rework Effort Ratio: This measure helps to determine the rework effort ratio.

Rework Effort Ratio = (Actual rework efforts spent in that phase/ Total actual efforts spent in that phase) x 100

7. Accepted Defects Percentage: This measures the percentage of defects that are accepted out of the total accepted defects.

Accepted Defects Percentage = (Defects Accepted as Valid by Dev Team / Total Defects Reported) x 100

8. Defects Deferred Percentage: This measures the percentage of the defects that are deferred for future release.

Defects Deferred Percentage = (Defects deferred for future releases / Total Defects Reported) x 100

Example of Software Test Metrics Calculation

Let's take an example to calculate test metrics:

S No.	Testing Metric	Data retrieved during test case development
1	No. of requirements	5
2	The average number of test cases written per requirement	40
3	Total no. of Test cases written for all requirements	200
4	Total no. of Test cases executed	164
5	No. of Test cases passed	100

S No.	Testing Metric	Data retrieved during test case development
6	No. of Test cases failed	60
7	No. of Test cases blocked	4
8	No. of Test cases unexecuted	36
9	Total no. of defects identified	20
10	Defects accepted as valid by the dev team	15
11	Defects deferred for future releases	5
12	Defects fixed	12

***1. Percentage test cases executed** = (No of test cases executed / Total no of test cases written) x 100*

= (164 / 200) x 100
= 82

***2. Test Case Effectiveness** = (Number of defects detected / Number of test cases run) x 100*

= (20 / 164) x 100
= 12.2

***3. Failed Test Cases Percentage** = (Total number of failed test cases / Total number of tests executed) x 100*

*= (60 / 164) * 100*
= 36.59

***4. Blocked Test Cases Percentage** = (Total number of blocked tests / Total number of tests executed) x 100*

*= (4 / 164) * 100*
= 2.44

5. Fixed Defects Percentage = *(Total number of flaws fixed / Number of defects reported) x 100*

*= (12 / 20) * 100*

= 60

6. Accepted Defects Percentage = *(Defects Accepted as Valid by Dev Team / Total Defects Reported) x 100*

*= (15 / 20) * 100*

= 75

7. Defects Deferred Percentage = *(Defects deferred for future releases / Total Defects Reported) x 100*

*= (5 / 20) * 100*

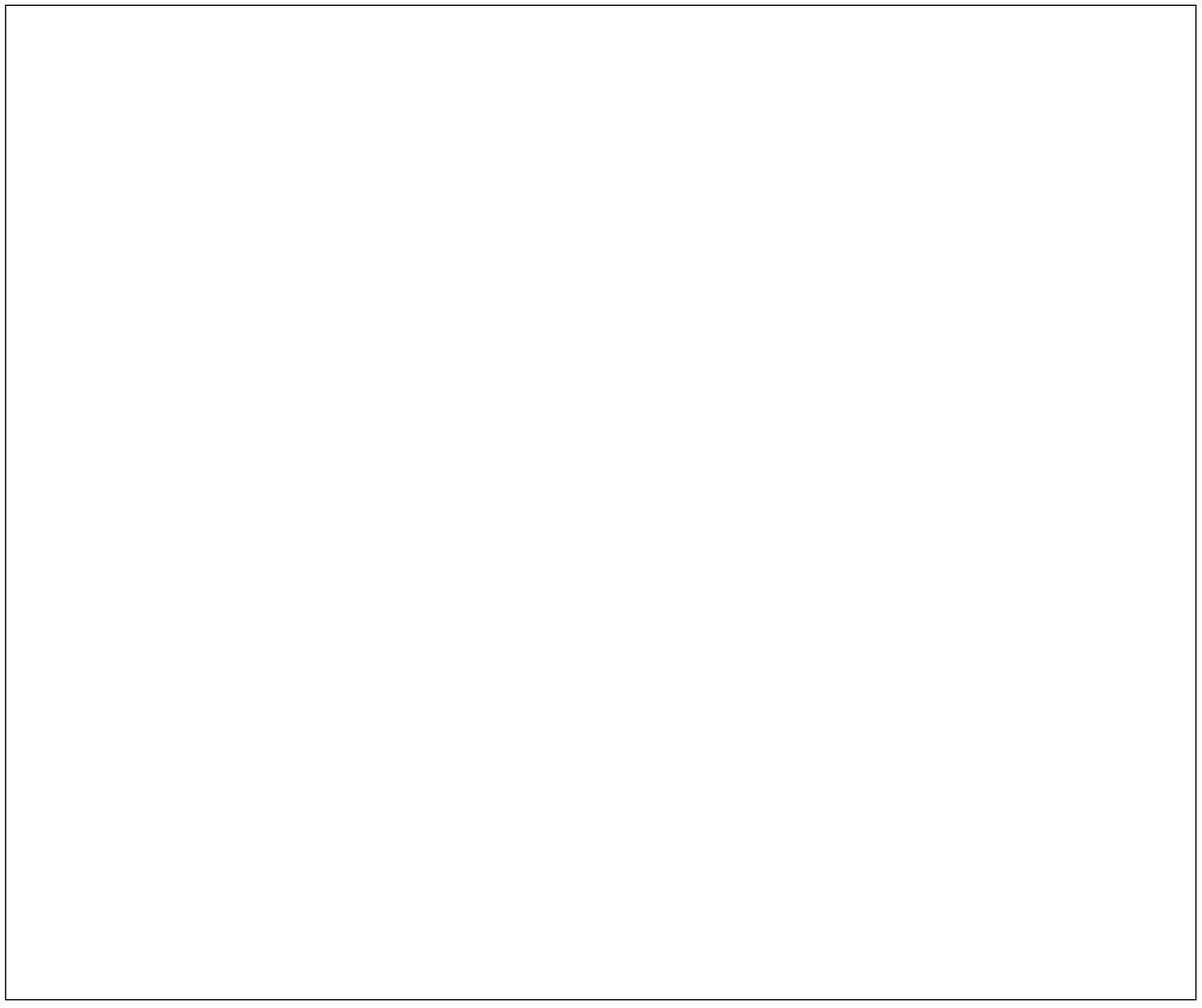
= 25

Both RTM and testing metrics play crucial roles in the testing process by providing structure, visibility, and insights into the effectiveness of testing efforts and the quality of the software under test.

I. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet Package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

J. Implementation: Design scenarios and Test cases for remaining functionalities given in theory section.



K. Practical related Exercises.

1. **What is the purpose of an RTM (Requirements Traceability Matrix) in software testing?**
   - **a)** To document test cases
   - b) To trace requirements to test cases
   - c) To track defect status
   - d) To analyze testing metrics
2. **Which testing metric measures the percentage of passed test cases out of the total executed test cases for a dynamic web page?**
   - **a)** Test Coverage
   - b) Defect Density
   - c) Pass Rate
   - d) Response Time
3. **In the context of an e-commerce website, which testing metric would be relevant for evaluating the performance of a dynamic web page?**

a) Code Coverage
b) Response Time
c) Requirement Traceability
d) Test Case Effectiveness

4. **What is the primary purpose of developing an RTM for dynamic web pages in an e-commerce website?**
   a) To track defects
   b) To ensure security
   c) To establish traceability between requirements and test cases
   d) To measure customer satisfaction
5. **Which testing metric provides insights into how thoroughly test cases cover the functionalities of dynamic web pages in an e-commerce website?**
   a) Defect Density
   b) Test Execution Time
   c) Test Case Coverage
   d) Performance Efficiency

L. References Links

1. What is Requirements Traceability Matrix (RTM) in Testing? (guru99.com)
2. Requirements Traceability Matrix - RTM - GeeksforGeeks
3. Traceability Matrix - javatpoint
4. How to Create Requirements Traceability Matrix (RTM) Example Sample Template (softwaretestinghelp.com)

M. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	/25

Signature with Date

Practical No.10 Execute test cases for a travel booking app and prepare a test Summary report.

A. **Objective:** Execute test cases for a travel booking app to validate its functionality, reliability, and usability, and compile a comprehensive test summary report.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the *engineering* problems.
2. **Problem analysis**: Identify and analyze well-defined *engineering* problems using codified standard methods.
3. **Design/ development of solutions:** Design solutions for *engineering* well-defined technical problems and assist with the design of systems components or processes to meet specified needs.
4. **Engineering Tools, Experimentation and Testing:** Apply modern *engineering* tools and appropriate technique to conduct standard tests and measurements.
5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.
6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.
7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

This practical is expect to develop the following skills in you apply types, levels and methods of software testing on applications.

1. Identify system with its specification and get familiar with the system.
2. Use the different software testing concepts.
3. Ability to write system specification of the systems.
4. Ability to design test case for given systems.

D. Expected Course Outcomes (Cos)

Develop effective test cases based on specifications, utilizing appropriate templates and methodologies.

E. Practical Outcome (PRo)

Successfully executed test cases for a travel booking app, ensuring functional requirements are met, and prepared a comprehensive test summary report.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

A Travel Booking System is a comprehensive software application designed to facilitate the seamless planning and reservation of travel-related services. This system serves as a centralized platform that allows users to effortlessly search, compare, and book various travel options, including flights, hotels, car rentals, and other accommodations. Users can access a user-friendly interface to input their travel preferences, such as destination, dates, and preferences. The system then retrieves and presents relevant information, enabling users to make informed decisions based on factors like price, availability, and reviews. Integration with secure payment gateways ensures smooth and secure transactions. Additionally, the system often incorporates features for managing user profiles, tracking itineraries, and handling cancellations or modifications. With advanced functionalities like real-time updates, notifications, and multichannel support, a Travel Booking System enhances the overall user experience, providing a convenient and efficient means for individuals to plan and book their travel arrangements.

List of scenarios for travel booking system.

1. **User Registration and Login:**
   - Verify that users can successfully register for an account.
   - Ensure users can log in using valid credentials.
   - Validate the system response for invalid login attempts.
2. **Search and Booking Functionality:**
   - Test the search functionality for flights, hotels, and other travel options.
   - Verify that search results are accurate and relevant.
   - Ensure users can select and book travel options seamlessly.
3. **Payment Process:**
   - Test the payment gateway integration for different payment methods.
   - Verify that users receive proper payment confirmation.
   - Validate error handling for failed payment transactions.
4. **Cancellation and Refund:**
   - Test the cancellation process for booked flights or accommodations.

- Verify that refunds are processed correctly and in a timely manner.
- Confirm users receive appropriate notifications for cancellations.

5. **User Profile Management:**
   - Test the functionality to update user profiles.
   - Verify that changes to user profiles are reflected accurately.
   - Ensure password reset functionality works as expected.
6. **Cross-Browser and Cross-Device Compatibility:**
   - Execute test cases on different browsers (Chrome, Firefox, Safari, etc.).
   - Test the app on various devices (desktop, tablet, mobile) to ensure responsiveness.
7. **Performance Testing:**
   - Execute performance tests to ensure the app can handle a specified number of concurrent users.
   - Check for any performance bottlenecks in the system.
8. **Security Testing:**
   - Validate that user data is securely stored and transmitted.
   - Test for vulnerabilities such as SQL injection and cross-site scripting.
9. **Integration Testing:**
   - Verify integration with third-party services like payment gateways and mapping services.
   - Test data synchronization between the app and external systems.
10. **Usability Testing:**
    - Assess the overall user experience for simplicity and intuitiveness.
    - Validate that navigation within the app is user-friendly.

After executing these test cases, the test summary report should include.

Test Summary Report: Test summary Report is a document which contains a summary of all test activities and final test results of a testing project. Test report is an assessment of how well the Testing is performed. Based on the test report, stakeholders can evaluate the quality of the tested product and make a decision on the software release.

For example, if the test report informs that there are many defects remaining in the product, stakeholders can delay the release until all the defects are fixed.

This Test Summary Report provides a comprehensive overview of the testing activities conducted for the Travel Booking App. The testing phase aimed to ensure the functionality, reliability, and performance of the app, contributing to the overall quality of the product.

The testing focused on various components of the Travel Booking App, including user registration, search and booking functionality, payment processing, cancellation and

Refund processes, user profile management, cross-browser and cross-device compatibility, performance, security, and integration with third-party services.

Testing was carried out in a controlled environment using browsers such as Chrome, Firefox, and Safari. Multiple devices, including desktops, tablets, and mobiles, were employed to ensure cross-device compatibility. The app was tested in both staging and production environments.

Below is sample Test summary report you need to prepare same for Travel Booking system.

Test Report

Test Cycle System Test

EXECUTED	PASSED			130	
	FAILED			0	
	(Total) TESTS EXECUTED (PASSED + FAILED)				130
PENDING					0
IN PROGRESS					0
BLOCKED					0
(Sub-Total) TEST PLANNED (PENDING + IN PROGRESS + BLOCKED + TEST EXECUTED)					130

Functions	Description	% TCs Executed	% TCs Passed	TCs pending	Priority	Remarks
New Customer	Check new Customer is created	100%	100%	0	*High*	
Edit Customer	Check Customer can be edited	100%	100%	0	*High*	
New Account	Check New account is added	100%	100%	0	*High*	
Edit Account	Check Account is edit	100%	100%	0	*High*	
Delete Account	Verify Account is delete	100%	100%	0	*High*	
Delete customer	Verify Customer is Deleted	100%	100%	0	*High*	
Mini Statement	Verify Ministatement is generated	100%	100%	0	*High*	
Customized Statement	Check Customized Statement is generated	100%	100%	0	*High*	

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet Package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

I. Implementation: Design test cases, execute and prepare Test summary report.

OUTPUT

J. Practical related Exercises.

1. **What is the primary purpose of a Test Summary Report?**
   a. To create test cases
   b. To identify defects
   c. To provide an overview of testing activities
   d. To execute test scripts

2. **Which section of the Test Summary Report typically includes information about the test environment?**
   a. Introduction
   b. Test Items
   c. Test Execution Overview
   d. Test Environment

3. **What does "Test Coverage" in a Test Summary Report refer to?**
   a. The percentage of test cases executed
   b. The extent to which the software has been tested
   c. The number of defects found
   d. The time taken to execute test cases

4. **In the context of a Test Summary Report, what does the "Sign-off" signify?**
   a. Approval from stakeholders to proceed with development
   b. Approval from stakeholders to proceed with testing
   c. Acknowledgment that testing is complete and the software is ready for release

d. Indication of a failed testing phase

5. **Which section of the Test Summary Report discusses challenges encountered during the testing process?**
   a. Defect Summary
   b. Test Execution Overview
   c. Test Execution Challenges
   d. Recommendations

6. **What type of testing strategy might involve both manual and automated testing approaches?**
   a. Black-box testing
   b. White-box testing
   c. Gray-box testing
   d. Hybrid testing

7. **How is the total test coverage calculated in a Test Summary Report?**
   a. Percentage of defects found
   b. Percentage of test cases executed
   c. Percentage of requirements covered
   d. Percentage of test scripts written

8. **Which deliverables are commonly mentioned in the Test Summary Report?**
   a. Test Cases and Requirements
   b. Test Scripts and Defects
   c. Test Plan and Test Cases
   d. Test Plan and Automated Test Suites

9. **What does the "Recommendations" section of a Test Summary Report typically include?**
   a. Identified defects
   b. Suggestions for improvements or future testing efforts
   c. Test execution results
   d. Test environment details

10. **What is the primary focus of the "Conclusion" section in a Test Summary Report?**
    a. Providing detailed defect information
    b. Summarizing the overall testing effort and key findings

c. Recommending improvements for future testing phases
d. Listing test execution challenges

K. References Links

1. Test Summary Reports Tutorial: Learn with Example & Template (guru99.com)
2. How To Write An Effective Test Summary Report - Software Testing Stuff
3. How To Create A Test Summary Report? (testsigma.com)

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	**/5**
2	Test Case Design	**/8**
3	Test Case Execution	**/4**
4	Correctness of Practical Implementation	**/4**
5	Timely Submission of Report and Answer to Questions	**/4**
	Total	**/25**

Signature with Date

Practical No.11 Prepare defect report after executing test cases for registration page.

A. Objective: The program objective is to systematically execute test cases on the registration page and meticulously document any identified defects, ensuring a comprehensive defect report for thorough analysis and resolution.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the *engineering* problems.
2. **Problem analysis**: Identify and analyze well-defined *engineering* problems using codified standard methods.
3. **Design/ development of solutions:** Design solutions for *engineering* well-defined technical problems and assist with the design of systems components or processes to meet specified needs.
4. **Engineering Tools, Experimentation and Testing:** Apply modern *engineering* tools and appropriate technique to conduct standard tests and measurements.
5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.
6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.
7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

This practical is expect to develop the following skills in you apply types, levels and methods of software testing on applications.

1. Identify system with its specification and get familiar with the system.
2. Use the different software testing concepts.
3. Ability to write system specification of the systems.
4. Ability to design test case for given systems.

D. Expected Course Outcomes (Cos)

Apply bug identification skills to create a comprehensive defect report for the software application.

E. Practical Outcome (PRo)

Create Efficient defect reporting enhances the registration page's functionality and user experience by addressing identified issues through executed test cases.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

Prerequisite: Registration Page developed in PHP or any other language and test cases for Registration page to execute it and generate defect report.

Defect: A defect in a software product is also known as a bug, error or fault which makes the software produce an unexpected result as per the software requirements. For example; incorrect data, system hangs, unexpected errors, missing or incorrect requirements.

Defect Report: A defect report is a document that has concise details about what defects are identified, what action steps make the defects show up, and what are the expected results instead of the application showing error (defect) while taking particular step by step actions.

Defect reports are usually created by the Quality Assurance team and also by the end-users (customers). Often customers detect more defects and report them to the support team of the software development since the majority of the customers curiously tries out every feature in the application. Now, you know what actually defect and defect reports are.

A Defect in Software Testing is a variation or deviation of the software application from end user's requirements or original business requirements. A software defect is an error in coding which causes incorrect or unexpected results from a software program which does not meet actual requirements. Testers might come across such defects while executing the test cases.

The reason behind why defect reports are created is to help developers to find out the defects easily and fix them up. A defect report is usually assigned by QA to a developer who then reads the report and reproduces the defects on the software product by following the action steps mentioned in the report. After that, the developer fixes the defects in order to get the desired outcome specified in the report.

That is why the defect reports are important and created carefully. Defect reports should be short, organized, and straight to the point and covers all the information that the

Developer needs to detect the actual defects in the report by doing what and how the one written the defect report detected the defects.

It is usual for QA teams to get defect reports from the clients that are either too short to reproduce and rectify or too long to understand what actually went wrong.
For example,

Defect Description: The application doesn't work as expected.

Now, how in the world does a developer or QA know what went wrong which doesn't meet the client expectation?

In such a case, the developer report to the QA that he couldn't find any problem or he may have fixed any other error but not the actual one client detected. So that's why it's really important to create a concise defect report to get bugs fixed.

All right. You have a pretty good idea about what, whys and how's of a defect report. So it's time for what is inside the report.
A typical defect report contains the information in an xls Sheet as follows.

1. **Defect ID:** Nothing but a serial number of defects in the report.
2. **Defect Description:** A short and clear description of the defect detected.
3. **Steps to Reproduce:** The specific steps to reproduce the defect.
4. **Action Steps:** What the client or QA did in an application that results in the defect. Step by step actions they took.
5. **Expected Result:** What results are expected as per the requirements when performing the action steps mentioned.
6. **Actual Result:** What results are actually showing up when performing the action steps?
7. **Severity:** Trivial (A small bug that doesn't affect the software product usage).
   Priority: The importance of fixing the defect (e.g., low, medium, high).
   a. **Low –** A small bug that needs to be fixed and again it's not going to affect the performance of the software.
   b. **Medium –** This bug does affect the performance. Such as being an obstacle to do a certain action. Yet there is another way to do the same thing.
   c. **High –** It highly impacts the software though there is a way around to successfully do what the bug cease to do.
   d. **Critical –** These bugs heavily impact the performance of the application. Like crashing the system, freezes the system or requires the system to restart for working properly.
8. **Attachments:** A sequence of screenshots of performing the step-by-step actions and getting the unexpected result. One can also attach a short screen recording of

Performing the steps and encountering defects. Short videos help developers and/or QA to understand the bugs easily and quickly.

9. **Additional information:** The platform you used, operating system and version. And other information which describes the defects in detail for assisting the developer understand the problem and fixing the code for getting desired results.
10. **Reported By:** The name of the person who reported the defect.
11. **Date Reported:** The date when the defect was reported.

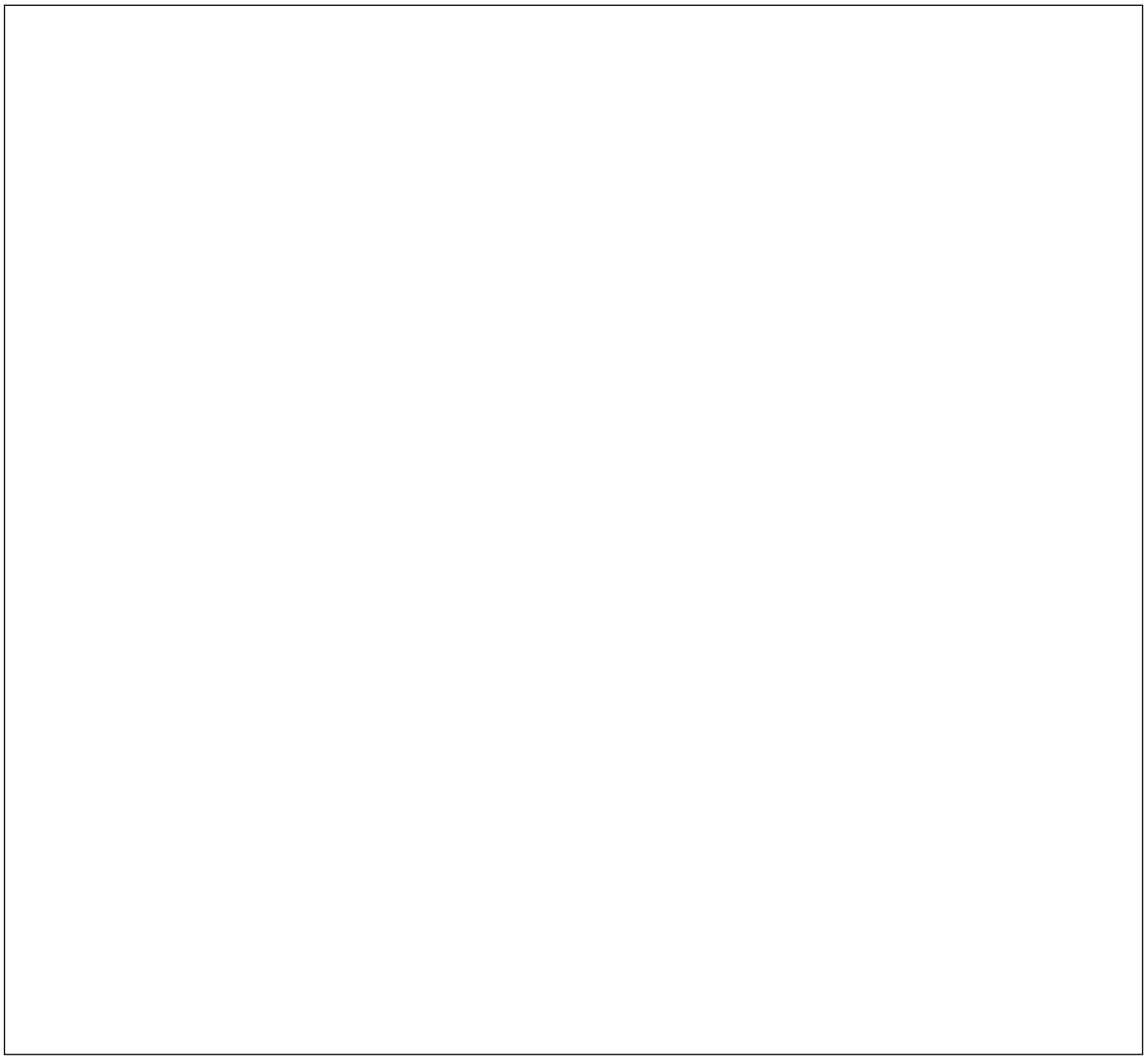
Sample Defect Report example.

Defect ID	Description	Steps to Reproduce	Expected Results	Actual Results	Severity	Priority	Reported By	Date Reported
001	Login button not functioning	1. Open the app 2. Click on the login button	The app should navigate to the user's account page	The login button does not respond	High	High	Tester A	01/10/2024

Now based on this example students can generate Defect report for registration page after executing test cases.

H. Resources/Equipment Required

Sr. No	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc



J. Practical related Questions.

1. What is a defect report in software testing?
   a. A document that outlines the testing strategy
   b. A report on the performance of the testing team
   c. A document that describes deviations from expected behavior
   d. A summary of the project timeline
2. Which of the following information is typically included in a defect report?
   a. Project timeline
   b. Test strategy
   c. Steps to reproduce the issue
   d. Team members' attendance
3. When should a defect report be raised?
   a. Only after the entire testing phase is complete
   b. As soon as a defect is identified

c. At the end of the project
d. During the development phase

4. What does the severity level in a defect report indicate?
   a. The likelihood of the defect occurring
   b. The impact of the defect on the system
   c. The priority assigned by the development team
   d. The tester's frustration level
5. In a defect report, what does the term "reproducible" mean?
   a. The defect is difficult to understand
   b. The defect can be consistently observed
   c. The defect is impossible to fix
   d. The defect is related to the user interface

K. References Links

1. Defect Management Process in Software Testing (Bug Report) (guru99.com)
2. What is Defect or Bug? Defect Reporting Template (artoftesting.com)
3. Defect Report in Software Engineering - GeeksforGeeks
4. How to write a good Defect Report? | BrowserStack

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	**/25**

Signature with Date

Practical No.12 Prepare defect report after executing test cases for Withdrawn of amount from ATM Machine.

A. Objective: To systematically document and communicate identified issues, ensuring clarity and traceability, following the execution of test cases for the withdrawal of amounts from an ATM Machine.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the *engineering* problems.
2. **Problem analysis**: Identify and analyze well-defined *engineering* problems using codified standard methods.
3. **Design/ development of solutions:** Design solutions for *engineering* well-defined technical problems and assist with the design of systems components or processes to meet specified needs.
4. **Engineering Tools, Experimentation and Testing:** Apply modern *engineering* tools and appropriate technique to conduct standard tests and measurements.
5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.
6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.
7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

This practical is expect to develop the following skills in you apply types, levels and methods of software testing on applications.

1. Identify system with its specification and get familiar with the system.
2. Use the different software testing concepts.
3. Ability to write system specification of the systems.
4. Ability to design test case for given systems.

D. Expected Course Outcomes (Cos)

Apply bug identification skills to create a comprehensive defect report for the software application.

E. Practical Outcome (PRo)

Create a comprehensive defect report following the execution of test cases for ATM Machine's withdrawal functionality.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory: Defect report definition and details already discussed in previous practical. Here in this practical ATM Withdrawal of Amount.

The withdrawal process from the ATM machine involves a series of steps to facilitate a seamless transaction for users. Upon inserting the card into the ATM, users are prompted to enter their Personal Identification Number (PIN). Following successful authentication, users are required to select the desired withdrawal amount and confirm the transaction. However, during recent testing, two notable defects were identified.

The first defect involves a critical issue where, after confirming the withdrawal, the transaction fails, and the requested amount is not dispensed. Instead, users encounter an error message indicating the unsuccessful transaction. This poses a significant inconvenience and financial disruption for users attempting to withdraw funds.

The second defect relates to a delay in processing transactions. Upon confirmation of the withdrawal, users experience a notable lag in the system's response time, hindering the prompt completion of the transaction. While this issue is not as severe as the first, it introduces an undesirable delay that may affect the overall user experience and efficiency of the ATM service.

These defects compromise the reliability and efficiency of the ATM's withdrawal functionality, necessitating immediate attention and resolution to ensure users can seamlessly access their funds without encountering errors or delays. Thorough investigation, debugging, and optimization of the transaction processing system are recommended to address these issues promptly.

Test Case: Withdrawal of Amount from ATM Machine

Test Case ID: ATM-TC-001

Test Title: Withdrawal of Amount

Verify that customers can successfully withdraw the requested amount from the ATM machine, and the system handles the transaction accurately and efficiently.

Preconditions:

- ATM machine is powered on and operational.
- Sufficient funds are available in the account linked to the inserted card.

- Cardholder has a valid PIN.

Test Steps:

1. **Insert Card:**
   - Insert a valid ATM card into the card reader.
2. **Enter PIN:**
   - Enter a valid PIN using the keypad.
3. **Select Transaction:**
   - Choose the "Withdrawal" option from the main menu.
4. **Enter Amount:**
   - Enter a valid withdrawal amount (within the account balance limits).
5. **Confirm Transaction:**
   - Press the "Confirm" or equivalent button to initiate the withdrawal.
6. **Transaction Authorization:**
   - Verify that the system prompts for authorization and displays the transaction details.
7. **Dispense Cash:**
   - Observe the ATM machine for the prompt to dispense cash.
   - Verify that the correct amount is dispensed.
8. **Receipt Confirmation:**
   - If applicable, check if the system provides an option for a transaction receipt.
   - Confirm that the receipt contains accurate information.
9. **Card Removal:**
   - Ensure the ATM prompts the user to remove the card.
10. **End Transaction:**
    - Verify that the system displays a transaction completion message.

Expected Results:

- The ATM machine should successfully process the withdrawal request.
- The correct amount of cash should be dispensed.
- The transaction receipt (if provided) should contain accurate information.
- The ATM machine should display a completion message and prompt the user to remove the card.

Post conditions:

- Customer receives the requested amount.
- Transaction details are accurately recorded.

Now based on this example students can generate Defect report for registration page after executing test cases.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet Package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

I. Practical Implementation:

Design defect report for Withdrawn of amount from ATM Machine after executing Test Cases for it.



J. Practical related Questions.

1. What is the purpose of a defect report in the context of testing ATM transactions?
   a. To outline the steps for ATM usage
   b. To summarize the ATM machine's features
   c. To identify and document issues observed during testing
   d. To provide general information about the ATM location
2. Which section of the defect report typically includes information about the person who reported the defect and the date it was raised?
   a. Summary
   b. Description
   c. Severity
   d. Reported By
3. What does "Severity" in a defect report indicate?
   a. The person responsible for fixing the defect
   b. The level of impact the defect has on the system
   c. The number of test cases executed
   d. The version of the ATM machine software
4. When should a defect report be raised?
   a. Only after the entire testing phase is complete
   b. As soon as a defect is identified

c. At the end of the project
d. During the development phase

5. Which attachment is essential in a defect report to provide visual evidence of the encountered error messages during testing?
   a. Transaction logs
   b. Screenshots of error messages
   c. ATM machine specifications
   d. User manuals

K. References Links

1. How to Write Test Cases for ATM Machine (Sample Scenarios) (softwaretestinghelp.com)
2. How To Write Test Cases for ATM (Test Scenarios ATM Machine) (softwaretestingmaterial.com)
3. ATM Simulator Project Report | PDF | Port (Computer Networking) | Network Socket (scribd.com)
4. Atm simulation mini project using Python programming language | PDF (slideshare.net)

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	**/5**
2	Test Case Design	**/8**
3	Test Case Execution	**/4**
4	Correctness of Practical Implementation	**/4**
5	Timely Submission of Report and Answer to Questions	**/4**
	Total	**/25**

Signature with Date

Practical No.13:

A) Install and set up the Selenium WebDriver and necessary drivers (e.g., ChromeDriver, GeckoDriver) on your system.

B) Install JUnit, TestNG using an IDE (Integrated Development Environment) like Eclipse or IntelliJ.

A. Objective:

To gain hands-on experience in setting up a Selenium testing environment with WebDriver, browser drivers, and popular testing frameworks in their chosen IDEs. This practical provides a solid foundation for automated testing using Selenium, Unit and TestNG

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

2. **Problem analysis**: Identify and analyze well-defined engineering problems using codified standard methods.

3. **Design/ development of solutions:** Design solutions for engineering well-defined technical problems and assist with the design of systems components or processes to meet specified needs.

4. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.

6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.

7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

- Install various software testing framework in Eclipse IDE for automation testing of application.
- Configure required client Library as per need of application testing.

D. Expected Course Outcomes (Cos)

Utilize popular testing tools (e.g., Selenium, JUnit, TestNG) for creating and executing automated test scripts.

E. Practical Outcome (PRo)

Install and Set up Eclipse IDE,configure Selenium WebDriver and browser specific drivers in your system for automation testing and Configure JUnit, TestNG testing framework in eclipse IDE.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

Selenium is a powerful and widely used open-source framework for automating web applications. It provides a suite of tools to support automated testing of web browsers across different platforms and browsers. Selenium is particularly valuable for quality assurance and software testing teams to streamline the testing process, enhance test coverage, and ensure the reliability of web applications.

Key Components of Selenium:

Selenium WebDriver:
WebDriver is the core component of Selenium, allowing users to interact with web browsers programmatically. It provides a programming interface to create and execute test scripts.
Selenium IDE (Integrated Development Environment):
Selenium IDE is a browser extension that facilitates record-and-playback functionality. It's a great tool for quick test script development, but it's primarily used as a prototyping tool.
Selenium Grid:
Selenium Grid allows the parallel execution of test scripts on multiple machines and browsers. This is useful for reducing test execution time and improving overall efficiency.
Key Features and Advantages:
Cross-Browser Compatibility:

Selenium supports various browsers such as Chrome, Firefox, Safari, and Internet Explorer, ensuring that web applications behave consistently across different environments.

Programming Language Support:

Selenium supports multiple programming languages, including Java, Python, C#, Ruby, and JavaScript. This flexibility allows teams to choose the language that best fits their expertise.

Platform Independence:

Selenium is platform-independent, meaning it can run on different operating systems like Windows, Linux, and macOS.

Open Source:

Selenium is open source, fostering a large and active community of developers. This community contributes to its continuous improvement and provides a wealth of resources and support.

Extensibility:

Selenium can be extended through the use of plugins and integrations, enabling users to customize and enhance their testing framework based on specific project requirements.

Selenium Workflow:

Identification of Elements:

Selenium allows testers to identify HTML elements on a web page using various locators such as ID, name, XPath, CSS selectors, and more.

Test Script Creation:

Test scripts are written in programming languages supported by Selenium (e.g., Java, Python). These scripts define the sequence of actions to be performed on the web application.

Execution and Verification:

Selenium executes the test scripts, interacting with the web browser to perform actions like clicking buttons, filling forms, and navigating through pages. Verification points are included to check if the application behaves as expected.

Reporting and Analysis:

Selenium provides detailed test reports, facilitating analysis of test results. Reports include information about passed and failed tests, allowing teams to identify and address issues promptly.

In conclusion, Selenium plays a crucial role in automating web application testing, contributing to the efficiency and effectiveness of software development and quality assurance processes. Its rich feature set, flexibility, and robust community support make it a preferred choice for organizations seeking reliable test automation solutions.

Selenium WebDriver Installation

Following steps will be helpful to install Selenium webdriver.
Step 1: Install Java SDK
Step 2: Install Eclipse
Step 3: Install Selenium Webdriver Files
Step 4: Configure Eclipse IDE with WebDriver

Step 1: Install Java SDK

Selenium WebDriver is a Java library, so you need to have Java installed on your machine. Follow the steps mentioned to download and install the Java Development Kit (JDK).

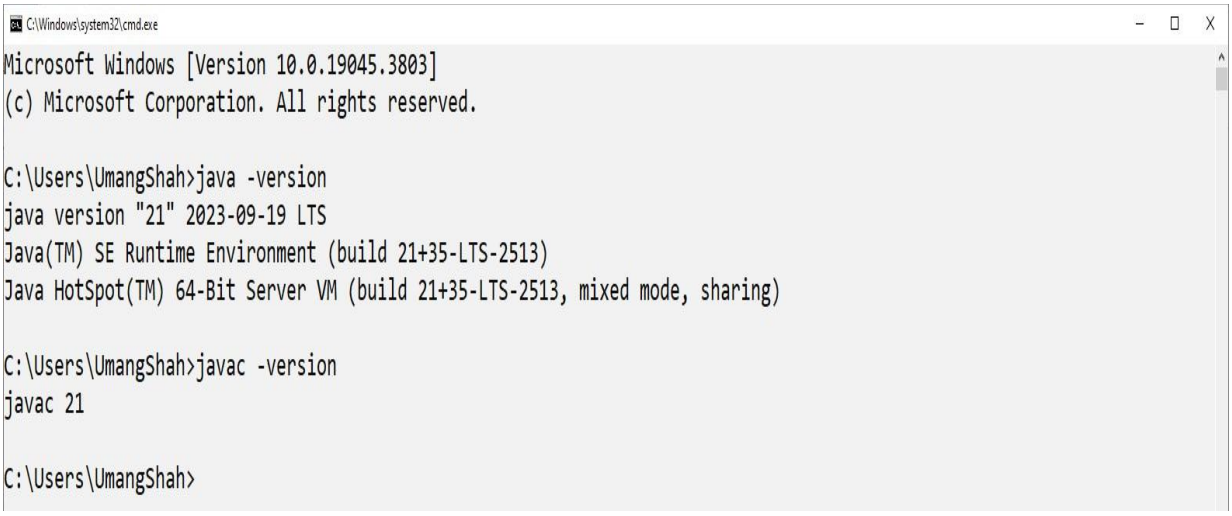
Download JDK:
Visit the official Oracle JDK download page or the OpenJDK website to download the JDK installer.(Install JDK)

Accept the license agreement.
Choose the appropriate version of JDK for your system (Windows) and download the installer.

Run the Installer:
Once the download is complete, locate the installer file (it's usually an executable file with a ".exe" extension).
Double-click on the installer to start the installation process and to complete the installation.

Set up Environment Variables:
After installation, you'll need to set up the JAVA_HOME and PATH environment variables:
Right-click on "This PC" or "Computer" on your desktop or in File Explorer.
Click on "Properties."
Click on "Advanced system settings" on the left.
Click on the "Environment Variables" button.
Under "System variables," click "New" and add a variable named JAVA_HOME with the path to your JDK installation directory (e.g., "C:\Program Files\Java\jdk-21").
Find the "Path" variable, click "Edit," and add "%JAVA_HOME%\bin" to the end of the list.
Verify the Installation:
Open a new command prompt and type the following commands:

```
java -version
javac -version
```



You should see above information about the installed Java version.

Step 2: Install Eclipse IDE

1. **Download Eclipse:**
   - Visit the official Eclipse download page: Eclipse Downloads.
   - Under "Eclipse IDE", choose the edition you want. "Eclipse IDE for Java Developers" is a popular choice for Java development.
   - Click the "Download" button.
2. **Select Installer:**
   - On the download page, you will see different download options. Choose the installer appropriate for your operating system (in this case, Windows).
3. **Download Installer:**
   - Once you've selected the installer for Windows, click on the download link.
   - The download should start, and you may need to wait for it to complete.
4. **Run Installer:**
   - Locate the downloaded installer file (e.g., **eclipse-inst-win64.exe** for 64-bit Windows) and run it.
5. **Eclipse Installer:**
   - The Eclipse Installer will open. It will ask you to select an IDE package to install. Choose the package that suits your needs. For Java development, you might choose "Eclipse IDE for Java Developers."



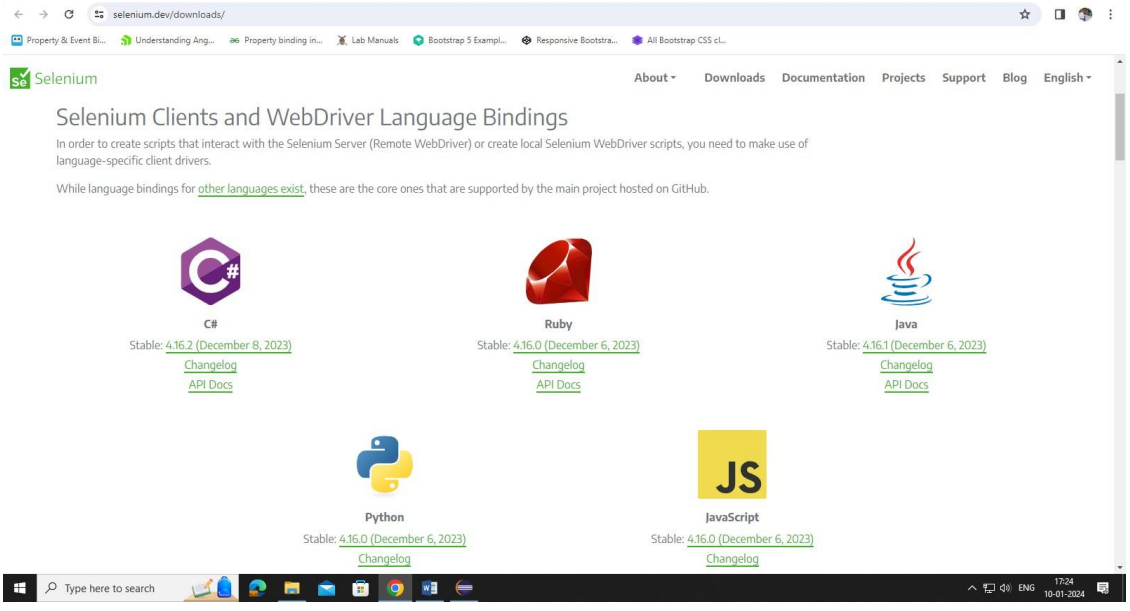
6. **Select Installation Folder:**
   - Choose the installation folder where Eclipse will be installed. The default location is usually fine, but you can choose a different one if you prefer.
7. **Select Workspace:**
   - The workspace is where your projects will be stored. You can use the default workspace or choose a different location.
8. **Install:**
   - Click on the "Install" button to start the installation process.
9. **Accept License Agreement:**
   - Read and accept the license agreement.
10. **Install Confirmation:**
    - Confirm the installation details and click on the "Launch" button.
11. **Launch Eclipse:**
    - Eclipse will now be installed, and the IDE will launch automatically. The first time it starts, it may take a little while to initialize.



12. **Set Up Development Environment:**
    - Upon launching, Eclipse will ask you to set up your development environment. Follow the on-screen instructions to configure your workspace.
13. **Eclipse Welcome Screen:**
    - Once everything is set up, you will be greeted with the Eclipse welcome screen, and you are ready to start coding.

Step 3: Install Selenium Webdriver Files

- Add the Selenium WebDriver JAR files. You can download the latest version of Selenium WebDriver from the official website and add the JAR files (e.g., **selenium-java-x.x.x.jar**, **selenium-server-standalone-x.x.x.jar**).



Step 4: Configure Eclipse IDE with WebDriver

To use Selenium WebDriver with Eclipse for Java development, you'll need to follow these steps to set up your environment:

1. **Install Eclipse:** If you haven't installed Eclipse yet, refer to the previous instructions for installing Eclipse.
2. **Open Eclipse:** Launch Eclipse IDE on your Windows machine.
3. **Create a New Java Project:**
    - Go to "File" -> "New" -> "Java Project."
    - Enter a project name and click "Finish."
4. **Configure Java Build Path:**
    - Right-click on your newly created project in the "Project Explorer."

- Select "Build Path" -> "Configure Build Path."
- In the "Libraries" tab, click on "Classpath" and then "Add External JARs."
- Add the Selenium WebDriver JAR files.

5. **Download Browser Drivers:**
   - Selenium WebDriver is a powerful tool for automating web browsers, and it provides different browser-specific implementations known as browser drivers. These drivers act as a bridge between your Selenium scripts and the browser. Selenium WebDriver interacts with web browsers using browser-specific drivers. Download the browser drivers (like ChromeDriver, GeckoDriver for Firefox, etc.) compatible with your browser versions from the respective official websites.
     Here are the commonly used WebDriver names for different browsers:
     Chrome:
     WebDriver Name: ChromeDriver
     Firefox:
     WebDriver Name: GeckoDriver
     Microsoft Edge:
     WebDriver Name: Microsoft WebDriver (or EdgeDriver for Chromium-based Edge)
     Safari:
     WebDriver Name: SafariDriver (Deprecated, not recommended for modern versions of Safari)
     Opera:
     WebDriver Name: OperaDriver (Note: Opera is not as commonly used with Selenium as Chrome or Firefox)

   - You have to download webdriver whichever you are using browser.here i am going to use chrome browser. Check your chrome browser version and according to that download chromedriver
     [Click here to download chrome web driver.]
   - Place the driver executable(s) in a directory that is included in your system's PATH variable or provide the path to the driver executable in your Selenium script.
   - When starting a WebDriver project in Eclipse, do not forget to import the Java Client Driver files onto your project. These files will constitute your Selenium Library.
6. **Write a Selenium Test Script:**
   - Create a new Java class in your Eclipse project.
   - Write a basic Selenium WebDriver script.This example opens the Google homepage and prints the title: Here's an example using Chrome:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
```

```
public class ChromeWebDriverExample {
public static void main(String[] args) {
      // Set the path to the ChromeDriver executable
System.setProperty("webdriver.chrome.driver",
"C:\\path\\to\\chromedriver.exe");

      // Create an instance of ChromeDriver
      WebDriver driver = new ChromeDriver();

      // Navigate to Google
driver.get("https://www.google.com");

      // Print the title to the console
System.out.println("Title: " + driver.getTitle());

      // Close the browser
driver.quit();
   }
}
```

Make sure to replace **"C:\\path\\to\\chromedriver.exe"** with the actual path to your **chromedriver.exe** file. This code sets the system property to locate the ChromeDriver executable, creates an instance of the ChromeDriver, navigates to Google, prints the title, and then closes the browser.

7. **Run your Java Program:**
   Run your Java program, and it should launch the browser and navigate to Google, printing the title in the console.
   Remember to keep your WebDriver executable up to date and ensure that it matches your browser version.
   That's it! You've successfully installed and configured Selenium WebDriver with Java on Windows.

JUnit

JUnit is a widely-used testing framework for Java programming language that provides support for writing and running unit tests. Unit testing is a software testing methodology where individual units or components of a software application are tested in isolation to ensure they behave as expected. JUnit facilitates the creation of automated test cases to verify the correctness of specific sections of code.

JUNIT installation and configuration

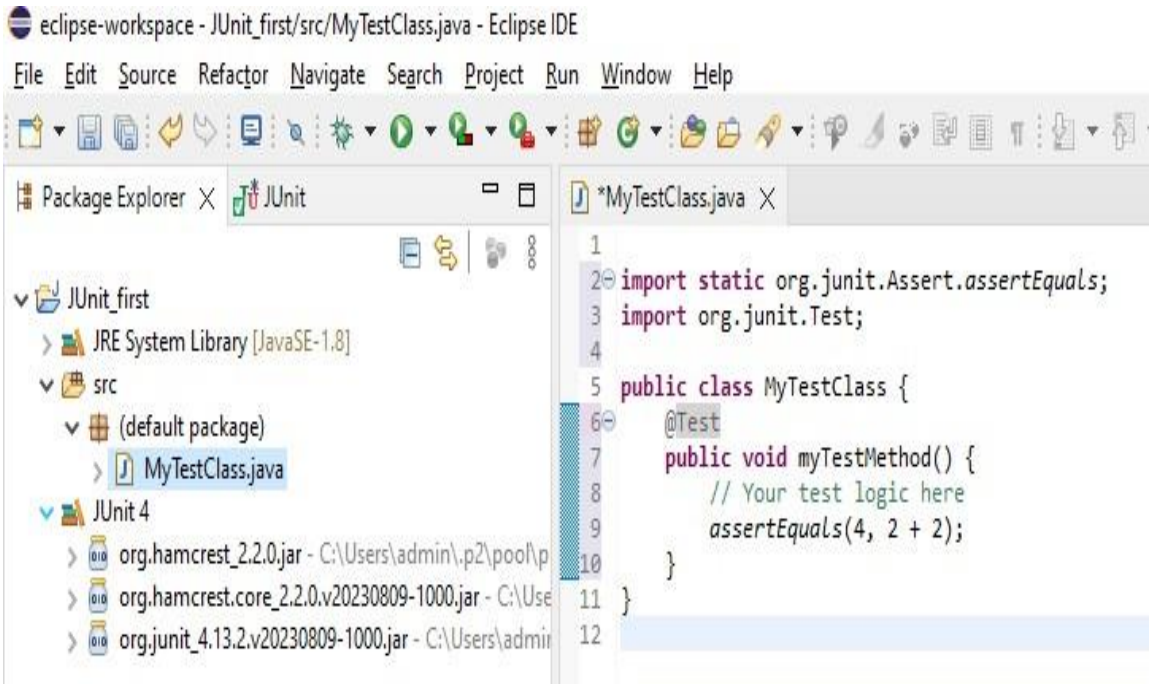
If you have successfully installed ecllips IDE then JUnitis available in it we only need to configure in it our project.If you don't have JUnit in your Eclipse workspace, you need to download it from official website.Let see how to configure it with in our project.

Configuring JUnit in Eclipse involves adding the JUnit library to the build path of your project. Here are the steps to configure JUnit in Eclipse using the Build Path options:

1. **Open Eclipse:** Start Eclipse and open your Java project.
2. **Create a New Java Class:** If you don't have a Java class or test class yet, create one. Right-click on the **src** folder in your project, select **New -> Class**, and give it a name.
3. **Write a Test Method:** In your Java class, write a method that will serve as a JUnit test. Annotate it with **@Test**.

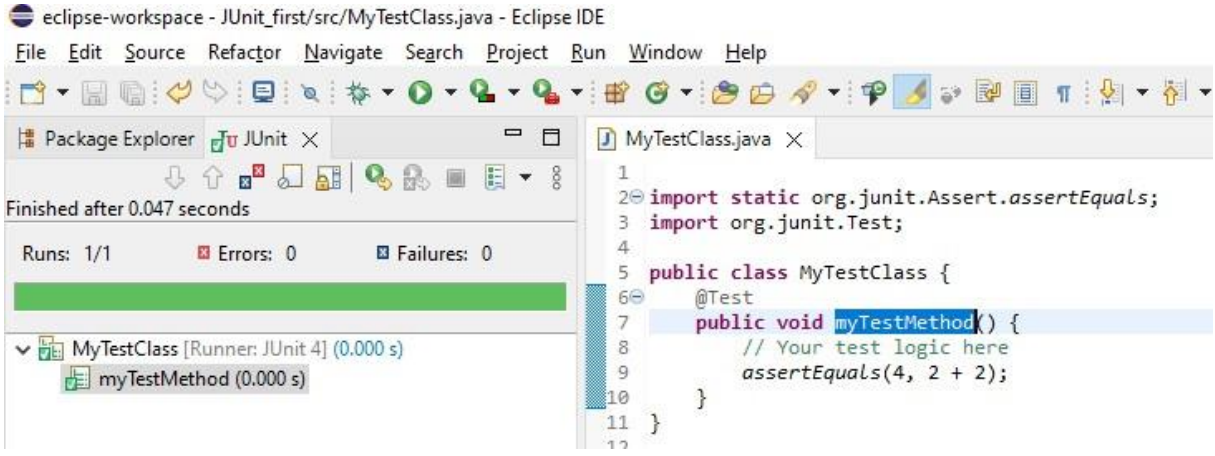
```
import static org.junit.Assert.assertEquals;
import org.junit.Test;
public class MyTestClass {
   @Test
public void myTestMethod() {
      // Your test logic here
assertEquals(4, 2 + 2);
   }
}          .
```

   Here you may get error but once you add JUnit jar files it will be solved which is given below
4. **Add JUnit Library to the Build Path:**
   - Right-click on your project in the Project Explorer.
   - Select **Properties**.
5. **Navigate to Java Build Path:**
   - In the Properties dialog, go to **Java Build Path** in the left navigation.
6. **Add JUnit Library:**
   - Click on the **Libraries** tab.
   - Click on the **Add Library** button.
7. **Select JUnit:**
   - Choose **JUnit** and click **Next**.
8. **Choose JUnit Library Version:**
   - Select the JUnit version you want to use (e.g., JUnit 4 or JUnit 5).
   - Click **Finish**.
9. **Apply and Close:**
   - Click **Apply and Close** to save the changes and close the dialog.



10. **Run Your JUnit Test:**
    - Right-click on your test method or test class.
    - Select **Run As -> JUnit Test**.

    Now, Eclipse is configured to run JUnit tests, and you can see the test results in the JUnit tab at the bottom of the Eclipse window.

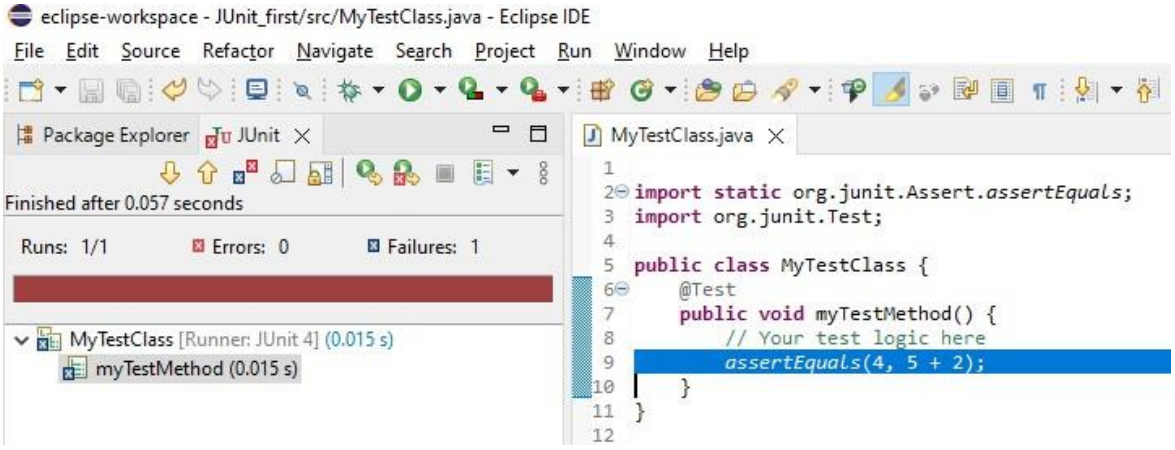


Ensure that you have the necessary JUnit JARs in your project's build path, and Eclipse will automatically recognize and run your JUnit tests.

Here you can see that we have created one test case in program for addition of 2 numbers and set actual value is 4 that is matched with expected value. Our test case is passed indicated by green color due to match both value that is our goal which is assume

Mentioned in test case. In this way we can perform various test cases of test suite for web application.

Now suppose if you are going to change value of expected value as shown below then your test case will become fail indicate by red color.



TestNG (Test Next Generation):

TestNG, which stands for "Test Next Generation," is a popular testing framework for Java that facilitates the creation and execution of test suites. Developed to address some limitations of JUnit, TestNG provides additional features and flexibility for different types of testing, including unit testing, functional testing, and integration testing.

TestNG installation and Configuration

To install and configure TestNG, you can manually download the TestNG library and add it to your Eclipse project. Here are step-by-step instructions:

Step 1: Download TestNG Jar

1. Visit the TestNG download page to download the TestNG JAR files.
2. Download the TestNG ZIP file for the latest version.
3. Extract the ZIP file to a location on your computer.

Step 2: Open Eclipse

1. Open your Eclipse IDE.

Step 3: Create a New Java Project

1. Go to **File -> New -> Java Project**.
2. Enter a project name and click **Finish**.

Step 4: Add TestNG Library

1. Right-click on your project in the Project Explorer.
2. Select **Properties**.
3. In the Properties dialog, go to **Java Build Path** in the left navigation.

4. Click on the **Libraries** tab.
5. Click on the **Add External JARs...** button.
6. Navigate to the location where you extracted the TestNG ZIP file.
7. Select the JAR files inside the "lib" folder (e.g., **testng-7.5.1.jar**), and click **Open**.
8. Click **Apply and Close** to save the changes.

Step 5: Verify TestNG Installation

1. Create a new Java class in your project.
2. Write a simple TestNG test method using annotations.

```
import org.testng.annotations.Test;
public class MyTest {
   @Test
public void myTestMethod() {
System.out.println("TestNG test method executed!");
   }
}
```

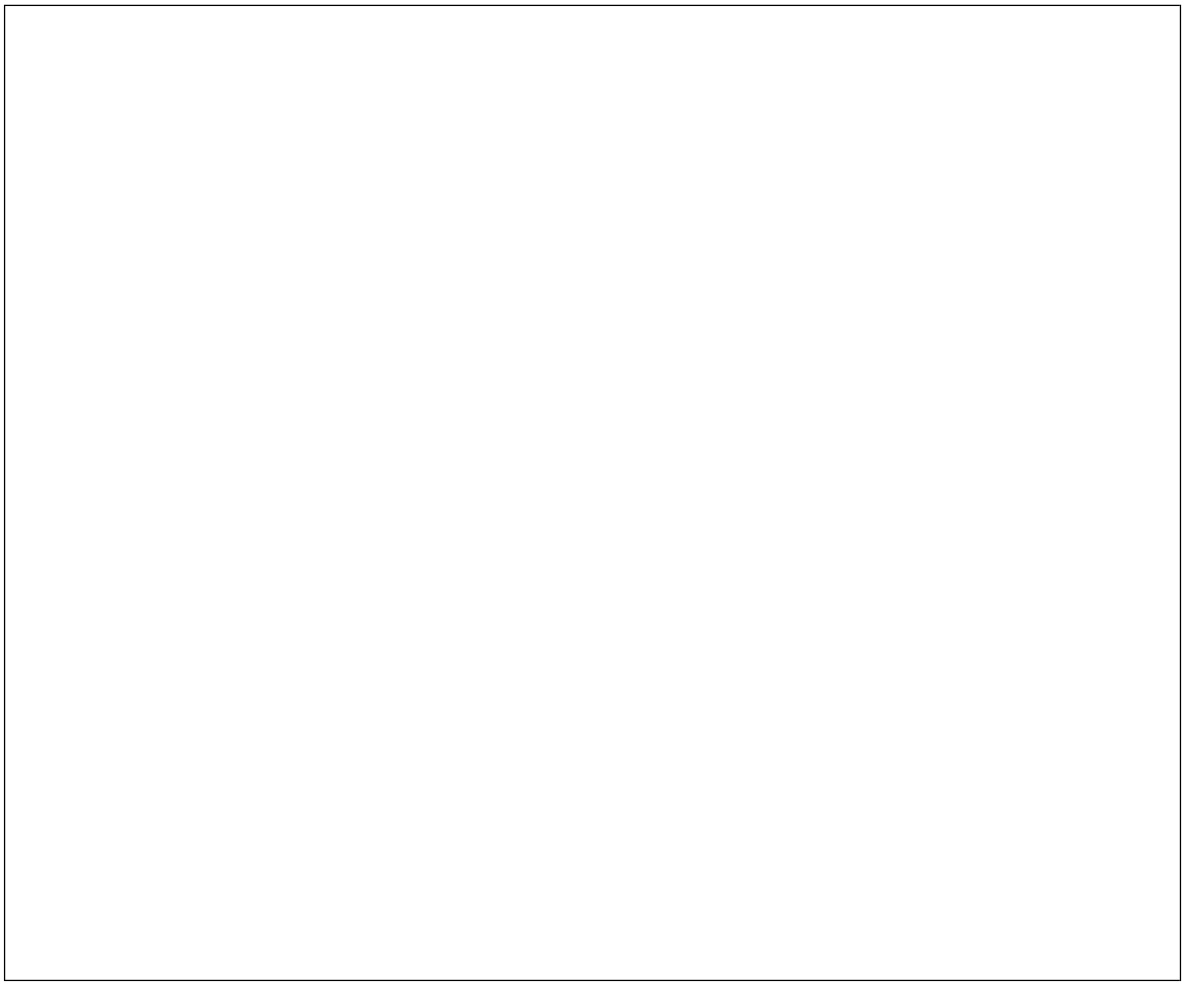
3. Right-click on the test method or the class.
4. Select **Run As -> TestNG Test**.

If everything is set up correctly, you should see the output in the Eclipse console indicating that the TestNG test method has been executed.

By following these steps, you've manually added TestNG to your Eclipse project, and you can now start writing and running TestNG.Keep in mind that when working on larger projects, using a build tool like Maven or Gradle is often recommended for better dependency management and project organization.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc



J. Practical related exercise.

1. **Which WebDriver is commonly used for automating web applications with Selenium?**
   A) FirefoxDriver
   B) ChromeDriver
   C) SafariDriver
   D) All of the above
2. **What is the purpose of a WebDriver executable (e.g., ChromeDriver, GeckoDriver) in Selenium?**
   A) To launch the browser
   B) To interact with the browser
   C) To locate web elements
   D) To execute JavaScript
3. **How do you set the system property for ChromeDriver in Java?**
   A) System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
   B) System.setProperty("chrome.driver", "path/to/chromedriver");

C) System.setChromeProperty("webdriver.chrome.driver", "path/to/chromedriver");
D) SetProperty.Chrome("path/to/chromedriver");

4. **In Eclipse IDE, how do you add JUnit to your Java project?**
   A) Right-click project > Properties > Libraries > Add Library > JUnit
   B) Right-click project > Build Path > Configure Build Path > Libraries > Add Library > JUnit
   C) Right-click project > Run As > JUnit Test
   D) Right-click project > JUnit > Add Library
5. **What is the purpose of GeckoDriver in Selenium?**
   (A).To handle Edge browser automation
   (B). To handle Internet Explorer automation
   (C). To handle Firefox browser automation
   (D). To handle Safari browser automation

K. References / Suggestions

1. https://www.javatpoint.com/selenium-webdriver-installation
2. https://www.youtube.com/watch?v=nhY_f7_j2cU
3. https://www.guru99.com/installing-selenium-webdriver.html
4. https://intellipaat.com/blog/tutorial/selenium-tutorial/installation/
5. https://www.geeksforgeeks.org/how-to-download-and-install-junit-in-eclipse/
6. https://www.guru99.com/install-testng-in-eclipse.html

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	/25

Signature with Date

Practical No 14: Design and run test script for a registration page using Selenium tool and JUnit.

A. Objective:

To ensure the functionality and reliability of the registration process. Verify that users can successfully navigate through the registration form, input valid information, submit the form, and receive confirmation. The objective is to identify and address any issues related to user registration, providing a robust and error-free experience for end-users.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the engineering problems.

2. **Problem analysis**: Identify and analyze well-defined engineering problems using codified standard methods.

3. **Design/ development of solutions:** Design solutions for engineering well-defined technical problems and assist with the design of systems components or processes to meet specified needs.

4. **Engineering Tools, Experimentation and Testing:** Apply modern engineering tools and appropriate technique to conduct standard tests and measurements.

5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.

6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.

7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

- Ability to design test case for given test scenario of systems.
- Apply browser specific driver in test script
- Utilize various elements locator types and JUnit Annotations in test script for automation testing.

D. Expected Course Outcomes (Cos)

Utilize popular testing tools (e.g., Selenium, JUnit, TestNG) for creating and executing automated test scripts.

E. Practical Outcome (PRo)

Design test cases and run test script for a registration page using Selenium tool and JUnit.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory:

Selenium Element Locator

In Selenium, element locators are used to locate web elements on a web page. There are various types of locators, and each is suitable for different scenarios. Here's overview of some commonly used Selenium locators:

Locator Type	Syntax	Example Usage
ID	By.id("idValue")	driver.findElement(By.id("username")).sendKeys("user123");
Name	By.name("nameValue")	driver.findElement(By.name("password")).sendKeys("pass456");
Class Name	By.className("classNameValue")	driver.findElement(By.className("submit-button")).click();
Tag Name	By.tagName("tagNameValue")	List<WebElement> links = driver.findElements(By.tagName("a"));
Link Text	By.linkText("linkTextValue")	driver.findElement(By.linkText("Click Here")).click();
Partial Link Text	By.partialLinkText("partialLinkTextValue")	driver.findElement(By.partialLinkText("Partial Link")).click();

XPath	By.xpath("xpathExpression")	driver.findElement(By.xpath("//input[@id='username']")).sendKeys("user123");
CSS Selector	By.cssSelector("cssSelectorValue")	driver.findElement(By.cssSelector(".login-form input[name='password']")).sendKeys("pass456");

Each locator has its own advantages and considerations. Choose the appropriate one based on the structure of the HTML document and the specific element you are trying to locate. XPath and CSS selectors are more flexible and powerful but may be slower than simpler locators like ID or Name

JUnit Annotations

In JUnit, annotations are used to provide metadata about methods and classes, enabling the framework to understand how to execute tests. Here's a table describing some commonly used annotations in JUnit with examples**:**

Annotation	Description	Example Usage
@Before	Executed before each test method.	@Before public void setUp() { /* Initialization code */ }
@After	Executed after each test method.	@After public void tearDown() { /* Cleanup code */ }
@Test	Denotes a test method.	@Test public void testMethod() { /* Test code */ }
@Ignore	Skips a test method or an entire class.	@Ignore @Test public void ignoredTest() { /* Ignored test code */ }
@BeforeClass	Executed once before any test methods.	@BeforeClass public static void setUpClass() { /* One-time setup code */ }
@AfterClass	Executed once after all test methods.	@AfterClass public static void tearDownClass() { /* One-time cleanup code */ }
@Test(timeout)	Specifies a maximum Execution time for a test.	@Test(timeout = 1000) public void testMethod() { /*

		Test code with a timeout of 1 second */ }
@Test(expected)	Expects an exception during the test.	@Test(expected = SomeException.class) public void testMethod() { /* Test code that expects an exception */ }

Here's a brief explanation of each annotation:

- @Before: Used for setup tasks that need to be performed before each test method runs.
- @After: Used for cleanup tasks that need to be performed after each test method runs.
- @Test: Denotes a method as a test method. JUnit will execute methods annotated with @Test.
- @Ignore: Skips the annotated test method or class, making it temporarily inactive.
- @BeforeClass: Executed once before any test methods in the class. Used for one-time setup.
- @AfterClass: Executed once after all test methods in the class. Used for one-time cleanup.
- @Test(timeout): Specifies the maximum execution time for a test method. If the method exceeds the specified timeout, it fails.
- @Test(expected): Specifies that the annotated test method is expected to throw a particular exception.

These annotations help organize and control the execution of test cases in a JUnit test.

The following code snippets are useful to design test script for automation testing using selenium and Junit.
Let us understand meaning of various code snippets which are given below.

1. **Setting ChromeDriver executable path:**

   System.setProperty("webdriver.chrome.driver", "path/to/chromedriver.exe");

   This line sets the system property **"webdriver.chrome.driver"** to the path of the ChromeDriver executable. This is necessary for Selenium to know where to find the ChromeDriver binary on your system. Replace **"path/to/chromedriver.exe"** with the actual path to your ChromeDriver executable.
2. **Initializing the ChromeDriver:**

   WebDriver driver = new ChromeDriver();

This line creates an instance of the **ChromeDriver** class, which is an implementation of the **WebDriver** interface specific to Google Chrome. This instance is what you'll use to interact with the Chrome browser.

3. **Navigating to a URL:**

```
driver.get("https://www.google.com");
```

This line instructs the ChromeDriver to open the specified URL (**"https://www.google.com"**) in the Chrome browser. After this line executes, the browser window will load the content of the specified web page.

4. **Complete Code:**

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class Example {
public static void main(String[] args) {
        // Set the path to your ChromeDriver executable
System.setProperty("webdriver.chrome.driver", "path/to/chromedriver.exe");

        // Initialize the ChromeDriver
        WebDriver driver = new ChromeDriver();

        // Navigate to the specified URL
       driver.get("https://www. google.com");

        // Further interactions and assertions can be performed on the loaded page
        // ...

        // Close the browser window
driver.quit();
    }
}
```

5. **driver.findElement().isDisplayed():**

This line is used to check if a web element is displayed on the web page. It returns a boolean value, **true** if the element is displayed and **false** otherwise. Here's an example:

```
WebElement element = driver.findElement(By.id("exampleId"));
boolean isDisplayed = element.isDisplayed();
System.out.println("Is the element displayed? " + isDisplayed);
```

6. **driver.findElement().sendKeys():**
   This line is used to simulate typing into an input field on a web page. It's commonly used to input data into text fields. Here's an example:

   ```
   WebElement inputField = driver.findElement(By.name("username"));
   inputField.sendKeys("exampleUser");
   ```

   This code provides the basic structure to start Selenium automation using ChromeDriver. You can add more interactions and assertions within the script to simulate user behavior and perform testing on the loaded page.

7. **driver.findElement().click():**
   This line is used to simulate a click on a web element, such as a button or a link. Here's an example:

   ```
   WebElement button = driver.findElement(By.id("submitBtn"));
   button.click();
   ```

8. **assertTrue():**
   This is an assertion method commonly used in testing frameworks like JUnit or TestNG to check if a given condition is true. If the condition is false, the test will fail. Here's an example:

   ```
   boolean condition = true; // replace with your actual condition
   assertTrue("The condition is not true", condition);
   ```

9. **assertEquals():**
   This is another assertion method used to check if two values are equal. If the values are not equal, the test will fail. Here's an example:

   ```
   String expectedValue = "Hello, World!";
   String actualValue = someMethod(); // replace with the actual method returning a value
   assertEquals("Values are not equal", expectedValue, actualValue);
   ```

   To design test cases and create a test script for a registration page using Selenium with JUnit, you can follow the steps below. I'll provide basic example test script for a simple registration page.
   Below is a simple example by considering **following test cases** using Java, Selenium WebDriver, and JUnit.

Test Case Design for Registration page:

Test Case 1: Verify the Registration Page Elements

- Open the registration page.
- Check if all the required elements (e.g., input fields, buttons, and labels) are present.

Test Case 2: Perform a Successful Registration

- Open the registration page.
- Enter valid data into all required fields.
- Click on the "Register" button.
- Verify that the user is redirected to a success page or receives a success message.

Test Case 3: Perform an Unsuccessful Registration (e.g., Missing Required Field)

- Open the registration page.
- Leave one or more required fields blank.
- Click on the "Register" button.
- Verify that an error message is displayed, indicating the missing information.

Test Case 4: Perform an Unsuccessful Registration (e.g., Invalid Email)

- Open the registration page.
- Enter an invalid email address.
- Click on the "Register" button.
- Verify that an error message is displayed, indicating the invalid email format.

Step 1: Set Up Your Project

1. Open Eclipse or your preferred Java IDE.
2. Create a new Java project or use an existing one.
3. Download the Selenium WebDriver Java bindings from the official Selenium website and add the JAR files to your project.
4. Download the JUnit JAR files from junit.org and add them to your project.

   Make sure you have the necessary Selenium WebDriver and browser driver versions compatible with each other. You can download the ChromeDriver executable from the ChromeDriver downloads page. Adjust the WebDriver setup according to the browser you intend to use.

Step 2: Write the Test Script

Here's a basic example of a Selenium test script using JUnit for a registration page:

Selenium Test Script with JUnit:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.junit.After;
import org.junit.Before;
```

```
import org.junit.Test;
import static org.junit.Assert.*;

public class RegistrationPageTest {
private WebDriver driver;
private String baseUrl = "your_registration_page_url_here";

   @Before
public void setUp() {
System.setProperty("webdriver.chrome.driver", "path/to/chromedriver.exe");
driver = new ChromeDriver();
   }

   @After
public void tearDown() {
driver.quit();
   }

   @Test
public void testRegistrationPageElements() {
driver.get(baseUrl);
      // Add assertions to check if all required elements are present
assertTrue(driver.findElement(By.id("username")).isDisplayed());
assertTrue(driver.findElement(By.id("email")).isDisplayed());
assertTrue(driver.findElement(By.id("password")).isDisplayed());
assertTrue(driver.findElement(By.id("confirmPassword")).isDisplayed());
assertTrue(driver.findElement(By.id("registerButton")).isDisplayed());
   }

   @Test
public void testSuccessfulRegistration() {
driver.get(baseUrl);
      // Enter valid data into all required fields
driver.findElement(By.id("username")).sendKeys("testuser");
driver.findElement(By.id("email")).sendKeys("testuser@example.com");
driver.findElement(By.id("password")).sendKeys("test123");
driver.findElement(By.id("confirmPassword")).sendKeys("test123");
      // Click on the "Register" button
driver.findElement(By.id("registerButton")).click();
      // Verify successful registration (e.g., check for success message or redirection)
```

```
assertTrue(driver.getCurrentUrl().contains("success"));
    }

    @Test
public void testUnsuccessfulRegistrationMissingField() {
driver.get(baseUrl);
        // Leave one or more required fields blank
driver.findElement(By.id("username")).sendKeys("testuser");
        // Click on the "Register" button
driver.findElement(By.id("registerButton")).click();
        // Verify that an error message is displayed
        WebElement errorMessage = driver.findElement(By.id("error-message"));
assertTrue(errorMessage.isDisplayed());
assertEquals("Please fill in all required fields", errorMessage.getText());
    }

    @Test
public void testUnsuccessfulRegistrationInvalidEmail() {
driver.get(baseUrl);
        // Enter an invalid email address
driver.findElement(By.id("username")).sendKeys("testuser");
driver.findElement(By.id("email")).sendKeys("invalidemail");
driver.findElement(By.id("password")).sendKeys("test123");
driver.findElement(By.id("confirmPassword")).sendKeys("test123");
        // Click on the "Register" button
driver.findElement(By.id("registerButton")).click();
        // Verify that an error message is displayed
        WebElement errorMessage = driver.findElement(By.id("error-message"));
assertTrue(errorMessage.isDisplayed());
assertEquals("Invalid email address", errorMessage.getText());
    }
}
```

Step 3: Customize the Test Script

1. Replace **"your_registration_page_url"** with the actual URL of your registration page.
2. Replace **"path_to_chromedriver_executable"** with the actual path to your ChromeDriver executable.
3. Modify the element locators (**By.id**, **By.name**, **By.xpath**, etc.) based on the HTML structure of your registration page.

Step 4: Run the Test

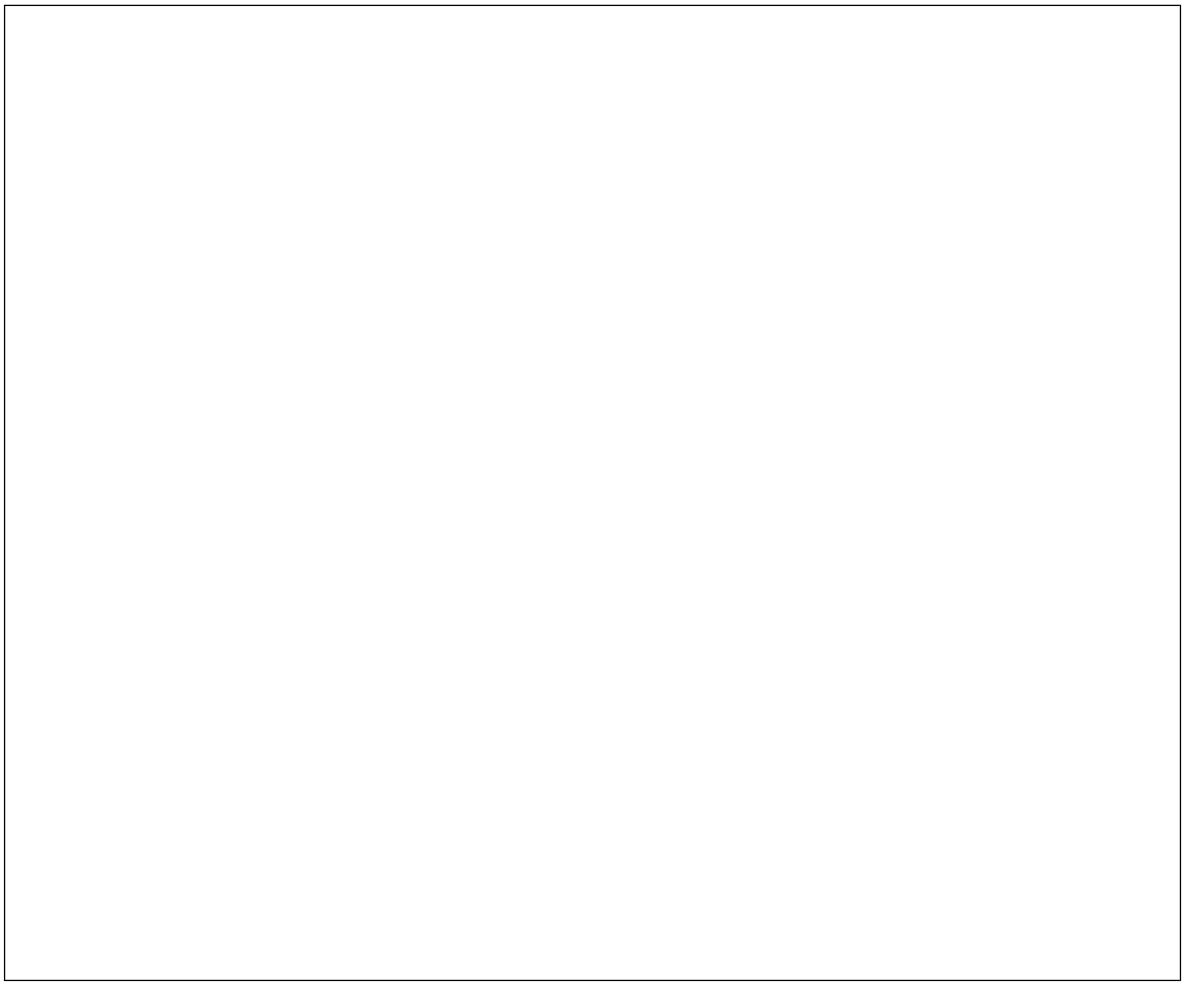
Right-click on your JUnit test class, select **Run As -> JUnit Test**. This will execute the test script, and you should see the browser interacting with the registration page.

H. Resources/Equipment Required

Sr. No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet Package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

I. Implementation:

Note: Students have to design or take registration page in which at least various 6 to 8 elements of html tags including various css property to perform this practical.



J. Practical related Exercises.

1. **Which annotation is used in JUnit to identify a method as a test method?**
   a. @TestMethod
   b. @Test
   c. @TestCase
   d. @TestMethod
2. **What command is used to launch the browser in Selenium WebDriver?**
   a. startBrowser()
   b. openBrowser()
   c. launchBrowser()
   d. driver.get()
3. **Which of the following is a correct way to assert in JUnit?**
   a. assert(condition)
   b. assertTrue(condition)
   c. check(condition)
   d. validate(condition)

4. **Which programming languages are supported by Selenium for writing test scripts?**
   a) Java, C#, Python
   b) PHP, Ruby, Perl
   c) JavaScript, TypeScript
   d) All of the above
5. **What is the role of assertions in JUnit?**
   a) To validate test data
   b) To define test cases
   c) To perform browser automation
   d) To check expected and actual results in tests

K. References / Suggestions

1. https://www.javatpoint.com/selenium-webdriver-locating-strategies
2. https://www.browserstack.com/guide/run-selenium-tests-using-selenium-chromedriver
3. https://www.guru99.com/junit-annotations-api.html
4. https://www.tutorialspoint.com/junit/junit_using_assertion.htm

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	**/25**

Signature with Date

Practical No.15 Design and run test script for a Login page and home page using Selenium tool and TestNG.

A. **Objective:** The objective is to effectively design and execute Selenium test scripts with TestNG for the Login page and home page, ensuring comprehensive testing and validation of user authentication and navigation functionalities in a web application.

B. Expected Program Outcomes (POs)

1. **Basic and Discipline specific knowledge:** Apply knowledge of basic mathematics, science and engineering fundamentals and engineering specialization to solve the *engineering* problems.
2. **Problem analysis**: Identify and analyze well-defined *engineering* problems using codified standard methods.
3. **Design/ development of solutions:** Design solutions for *engineering* well-defined technical problems and assist with the design of systems components or processes to meet specified needs.
4. **Engineering Tools, Experimentation and Testing:** Apply modern *engineering* tools and appropriate technique to conduct standard tests and measurements.
5. **Engineering practices for society, sustainability and environment:** Apply appropriate technology in context of society, sustainability, environment and ethical practices.
6. **Project Management:** Use engineering management principles individually, as a team member or a leader to manage projects and effectively communicate about well-defined engineering activities.
7. **Life-long learning:** Ability to analyze individual needs and engage in updating in the context of technological changes in field of engineering.

C. Expected Skills to be developed based on competency:

This practical is expect to develop the following skills in you apply types, levels and methods of software testing on applications.

1. Identify system with its specification and get familiar with the system.
2. Use the different software testing concepts.
3. Ability to write system specification of the systems.
4. Ability to design test case for given systems.

D. Expected Course Outcomes (Cos)

Utilize popular testing tools (e.g., Selenium, JUnit, TestNG) for creatingand executing automated test scripts.

E. Practical Outcome (PRo)

Efficiently validate the functionality and performance of a Login page and home page through automated testing using Selenium tool and TestNG.

F. Expected Affective domain Outcome (ADos)

1. Follow Testing standards and practices.
2. Maintain tools and equipment.
3. Follow safety practices.
4. Follow ethical practices

G. Prerequisite Theory: In previous practical we discussed aboutSelenium framework in details. Here in this practical we will see about TestNG Testing framework.

TestNG (Test Next Generation) is a testing framework for Java that is inspired by JUnit and NUnit. It is designed to make it easier to write and run test cases for Java applications, particularly for unit testing. TestNG provides features that go beyond what JUnit offers, and it has gained popularity in the Java testing community.

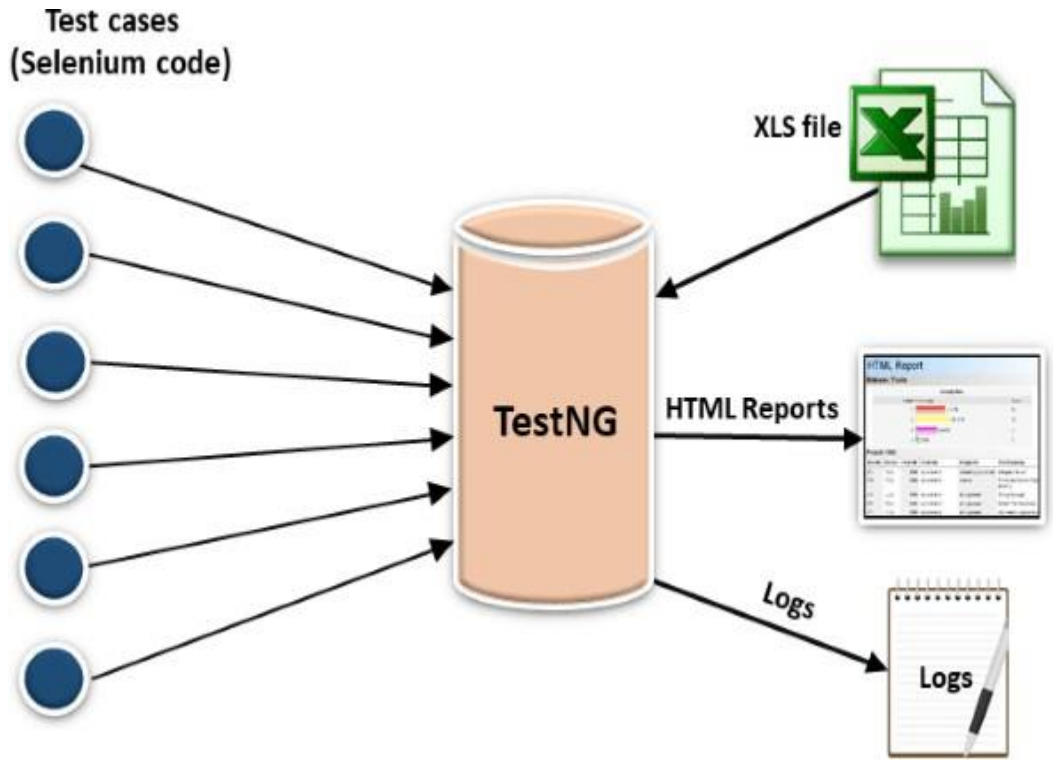
Key features and concepts of TestNG include:

Annotations: TestNG uses annotations to mark methods as test methods, configuration methods, and to define test dependencies. Common annotations include @Test, @BeforeMethod, @AfterMethod, @BeforeClass, @AfterClass, etc.

Test Configuration: TestNG allows users to configure test runs using XML configuration files. This enables users to define test suites, set parallel execution modes, specify test dependencies, and more.

Parallel Execution: TestNG supports parallel test execution, which can significantly reduce the overall test execution time. Tests can be run in parallel at the method, class, or suite level.

Groups: TestNG allows the grouping of test methods, and users can specify which groups of tests to run. This provides flexibility in running specific sets of tests based on their classifications.



Data-Driven Testing: TestNG supports data-driven testing through the use of data providers. This allows test methods to be executed with different sets of data.

Listeners: TestNG provides listeners that allow users to perform custom actions before or after test methods, suites, etc. This is useful for tasks such as logging, reporting, and handling test events.

Dependency Management: TestNG allows users to define dependencies between test methods, ensuring that certain methods are executed in a specific order.

Reporting: TestNG generates detailed and customizable HTML reports, making it easy to analyze test results.

TestNG Annotations: TestNG uses annotations to define the behavior of test methods and to perform various tasks during the testing lifecycle. Here are some of the most commonly used TestNG annotations, along with their explanations:

1. **@Test:**
    - **Purpose:** Marks a method as a test method.

- **Usage:@Test** is used to identify methods that should be executed as test cases.
- **Example:**

  java code

  @Test public void testExample() { // Test logic goes here }



2. **@BeforeMethod:**
   - **Purpose:** Executes code before each test method.
   - **Usage:** Used to set up preconditions or resources required for each test.
   - **Example:**

     java code

     @BeforeMethod public void setUp() { // Code to set up resources before each test method }
3. **@AfterMethod:**
   - **Purpose:** Executes code after each test method.

- **Usage:** Used to clean up or release resources after each test.
- **Example:**
  java code
  @AfterMethod public void tearDown() { // Code to clean up resources after each test method }

4. **@BeforeClass:**
   - **Purpose:** Executes code before the first test method in the test class.
   - **Usage:** Typically used for one-time setup activities.
   - **Example:**
     java code
     @BeforeClass public void classSetup() { // Code to run before the first test method in the class }
5. **@AfterClass:**
   - **Purpose:** Executes code after the last test method in the test class.
   - **Usage:** Used for one-time cleanup or finalization activities.
   - **Example:**
     java code
     @AfterClass public void classTearDown() { // Code to run after the last test method in the class }
6. **@BeforeTest:**
   - **Purpose:** Executes code before any test method belonging to a specified **<test>** tag in the XML file.
   - **Usage:** Used for setup activities specific to a test suite.
   - **Example:**
     java code
     @BeforeTest public void beforeTest() { // Code to run before any test method in the specified test tag }
7. **@AfterTest:**
   - **Purpose:** Executes code after all test methods belonging to a specified **<test>** tag in the XML file.
   - **Usage:** Used for cleanup or finalization activities specific to a test suite.
   - **Example:**
     java code
     @AfterTest public void afterTest() { // Code to run after all test methods in the specified test tag }
8. **@DataProvider:**
   - **Purpose:** Supplies test data to a test method.
   - **Usage:** Used in conjunction with a data provider method to feed data to a test.

- **Example:**
  java code
  @Test(dataProvider = "data") public void testDataDriven(String input) { // Test logic using the provided data } @DataProvider(name = "data") public Object[][] testData() { // Provide test data as a 2D array return new Object[][] {{"data1"}, {"data2"}}; }

9. **@Parameters:**
   - **Purpose:** Pass parameters to a test method from the XML configuration file.
   - **Usage:** Used to inject external parameters into test methods.
   - **Example:**
     java code
     @Test @Parameters("paramValue") public void testWithParameter(String paramValue) { // Test logic using the parameter value }
10. **@Listeners:**
    - **Purpose:** Specifies the listeners for a test class.
    - **Usage:** Used to attach listeners for customizing test execution.
    - **Example:**
      java code
      @Listeners(MyTestListener.class) public class MyTestClass { // Test methods and other annotations }

These annotations provide a flexible and powerful way to define the testing lifecycle and behavior in TestNG. They allow developers to customize the test execution flow and handle various aspects of test setup, execution, and teardown.
Overall, TestNG is a powerful testing framework that is widely used in the Java development community for unit testing, integration testing, and end-to-end testing of applications. It provides a rich set of features that cater to the diverse needs of software testing.

Below is an example of a simple Selenium WebDriver script with TestNG annotations. This example navigates to the Google homepage, performs a search, and asserts that the title of the search results page contains the expected text.

java code

```
import org.openqa.selenium.By;

import org.openqa.selenium.Keys;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
public class GoogleSearchTest {
    private WebDriver driver;
    private String baseUrl = "https://www.google.com/";

    @BeforeClass
    public void setUp() {
        // Set the path to the ChromeDriver executable
System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");

        // Initialize the ChromeDriver
        driver = new ChromeDriver();

        // Navigate to Google homepage
driver.get(baseUrl);
    }

    @Test
    public void testGoogleSearch() {
        // Find the search input field using its name attribute
WebElementsearchBox = driver.findElement(By.name("q"));
        // Enter the search query
searchBox.sendKeys("TestNG with Selenium");
```

```
        // Press Enter to perform the search
searchBox.sendKeys(Keys.RETURN);
        // Wait for the search results page to load
WebElementsearchResultsTitle = driver.findElement(By.cssSelector("h3"));
        // Assert that the title of the search results page contains the expected text
Assert.assertTrue(searchResultsTitle.getText().contains("TestNG with Selenium"));
    }
    @AfterClass
    public void tearDown() {
        // Close the browser window
driver.quit();
    }
}
```

Explanation of the script:

1. **@BeforeClass:**
    - This TestNG annotation is used for setup activities that need to be performed before any test method in the class.
    - In this example, it sets up the ChromeDriver and navigates to the Google homepage.
2. **@Test:**
    - This TestNG annotation marks a method as a test method.
    - In this example, the testGoogleSearch method performs a search on Google and asserts that the search results page title contains the expected text.
3. **@AfterClass:**
    - This TestNG annotation is used for cleanup activities that need to be performed after all test methods in the class have run.
    - In this example, it closes the browser window.

- To run this script, you'll need to have the Selenium WebDriver library and the ChromeDriver executable set up in your project. Adjust the path to the ChromeDriver executable accordingly.

H. Resources/Equipment Required

Sr.No.	Equipment/ Software Resources	Specification
1	Computer System	Computer system with operating system: Windows 7 or higher Ver.,macOS, and Linux, with 4GB or higher RAM
2	Software	Microsoft office or any Open-source spreadsheet package.
3	Code Editors and Tools	CODE Editor (Notepad++, VS Code, Sublime), JAVA IDE (Eclipse, NetBeans, IntelliJ)
4	Browser	Microsoft Edge, Google Chrome etc

I. Practical Implementation:





J. Practical related Questions.

1. **What annotation is used in TestNG to mark a method as a test method?**
   a. @Test
   b. @Method
   c. @TestCase
   d. @TestMethod
2. **How can you identify a web element in Selenium?**
   a. By class name
   b. By tag name
   c. By XPath
   d. All of the above
3. **What is the role of the assert statement in TestNG?**
   a. It is used to verify if a condition is true or false
   b. It is used to print messages to the console
   c. It is used to skip a test method
   d. It is used to pause the execution of a test method

4. **What is the purpose of the @BeforeTest annotation in TestNG?**
   a. It is used to specify the order of test methods
   b. It is used to mark a method as a pre-test setup
   c. It is used to define the test data
   d. It is used to generate test reports
5. **How can you group multiple test methods in TestNG?**
   a. Using @Group annotation
   b. Using @Suite annotation
   c. Using @TestGroup annotation
   d. Test methods cannot be grouped in TestNG

K. References Links

1. TestNG Tutorial: What is Annotations & Framework in Selenium (guru99.com)
2. TestNG Tutorial - Javatpoint
3. TestNG Tutorial (tutorialspoint.com)
4. TestNG Tutorial: Introduction to TestNG Framework (softwaretestinghelp.com)

L. Assessment-Rubrics

Sr. No.	Criteria	Obtain Marks
1	Test Planning	/5
2	Test Case Design	/8
3	Test Case Execution	/4
4	Correctness of Practical Implementation	/4
5	Timely Submission of Report and Answer to Questions	/4
	Total	**/25**

Signature with Date